UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola d'Enginyeria de Barcelona Est

FINAL DEGREE THESIS

Bachelor's Degree in Biomedical Engineering

MEDICAL IMAGING ANALYSIS TOOL FOR SCALABLE PATIENT-SPECIFIC STUDIES

Report and Annex

Author: Francisco Prata
Supervisor: Judit Buxadera Palomero
Co-Supervisor: Maria Ángeles Iglesias Blanco

Call: 2025 June

Abstract

This project presents the development of a modular and automated medical imaging tool for quantitative morphological analysis of anatomical structures using pre- and post-operative scans. It is designed to perform volumetric and shape-based comparisons by integrating several imaging processes—segmentation, mesh generation, registration, cropping, and statistical analysis—into a structured pipeline. The tool focuses on clinically relevant regions such as the spinal cord and psoas major muscle and supports batch processing across multiple patients. The architecture is built in Python, and is designed to enable scalability, flexibility, and easy incorporation of new functionalities. Segmentation is performed using AI-based models such as TotalSegmentator, and the tool interfaces with 3D Slicer for mesh and crop operations.

Preliminary results were obtained from two datasets of patients undergoing Anterior (ALIF) and Lateral (LLIF) Lumbar Interbody Fusion surgeries. Although limited, these studies demonstrate the potential of the tool to quantify post-surgical morphological changes such as spinal cord decompression and psoas major muscle atrophy. Overall, the project lays the foundation for a versatile image analysis platform aimed at supporting clinical research and surgical outcome assessment, offering a significant step toward automated, patient-specific volumetric and morphological evaluation imaging tool.

Resumen

Este proyecto presenta el desarrollo de una herramienta modular y automatizada de procesamiento de imágenes médicas para el análisis morfológico cuantitativo de estructuras anatómicas a partir de imágenes pre y postoperatorias. Su función principal es realizar comparaciones volumétricas y morfológicas mediante la integración de varios procesos de procesado de imagen —segmentación, generación de malla, registro, recorte— en una pipeline estructurada. La herramienta se centra en regiones clínicamente relevantes como la médula espinal y el músculo psoas mayor, y permite el procesamiento por lotes de múltiples pacientes. Su arquitectura está desarrollada en Python y está diseñada para ser escalable, flexible y permitir la incorporación sencilla de nuevas funcionalidades. La segmentación se realiza mediante modelos basados en inteligencia artificial como TotalSegmentator, y la herramienta se integra con 3D Slicer para las operaciones de malla y recorte.

Los resultados preliminares se obtuvieron a partir de dos conjuntos de datos de pacientes sometidos a cirugías de fusión intersomática lumbar anterior (ALIF) y lateral (LLIF). Aunque limitados, estos estudios demuestran el potencial de la herramienta para cuantificar cambios morfológicos postoperatorios, como la descompresión de la médula espinal y la atrofia del músculo psoas mayor. El proyecto presenta las bases de una plataforma versátil de análisis de imágenes orientada a apoyar la investigación clínica y la evaluación de resultados quirúrgicos, representando un avance significativo hacia una herramienta de imagen de evaluación morfológica y volumétrica automatizada.

Resum

Aquest projecte presenta el desenvolupament d'una eina modular i automatitzada de processament d'imatges mèdiques per a l'anàlisi morfològica quantitativa d'estructures anatòmiques a partir d'imatges pre i postoperatòries. La seva funció principal és realitzar comparacions volumètriques i morfològiques mitjançant la integració de diversos processos de processament d'imatge — segmentació, generació de malla, registre, retallada— en una pipeline estructurada. L'eina se centra en regions clínicament rellevants com la medul·la espinal i el múscul psoes major, i permet el processament per lots de múltiples pacients. La seva arquitectura està desenvolupada en Python i està dissenyada per a ser escalable, flexible i permetre la incorporació senzilla de noves funcionalitats. La segmentació es realitza mitjançant models basats en intel·ligència artificial com TotalSegmentator, i l'eina s'integra amb 3D Slicer per a les operacions de malla i retallada.

Els resultats preliminars es van obtenir a partir de dos conjunts de dades de pacients sotmesos a cirurgies de fusió intersomàtica lumbar anterior (ALIF) i lateral (LLIF). Encara que limitats, aquests estudis demostren el potencial de l'eina per a quantificar canvis morfològics postoperatoris, com la descompressió de la medul·la espinal i l'atròfia del múscul psoes major. El projecte presenta les bases d'una plataforma versàtil d'anàlisi d'imatges orientada a donar suport a la recerca clínica i l'avaluació de resultats quirúrgics, representant un avanç significatiu cap a una eina d'imatge d'avaluació morfològica i volumétrica.

Appreciations

I would like to start by thanking all the team at the Hospital de la Santa Creu i Sant Pau for granting me the opportunity to work with them and use their facilities. I began working with them during my 4-month extracurricular work experience in April 2024, and since then, I had been looking forward to returning. This project enabled me to embrace a professional work environment, and I am truly grateful for all the meetings, discussions, and conversations I had with every member of the team.

I also want to thank the Universitat Politècnica de Catalunya for granting me the opportunity to receive a technical education in a subject I am passionate about. I am convinced that all the accumulated knowledge from the past years will serve me well in the future.

Individually, I would like to thank my final degree thesis tutor, Judit Buxadera Palomero, for her trust and guidance through the technicalities of this project.

I would also like to thank Javier, Xavier, and Andreas—surgeons at the Hospital de la Santa Creu i Sant Pau—who helped me understand the anatomical and surgical complexities discussed in this paper.

I would like to give special thanks to the co-supervisor of this project, Maria Ángeles Iglesias Blanco, for constantly supervising my progress and motivating me to work harder. Without her guidance, I am convinced that this project would be a shell of itself.

Finally, I would like to thank my family for always supporting me and being by my side, even when they may not have fully understood what I was doing or the importance this project had for me.

Glossary

LIF – *Lumbar Interbody Fusion*: Surgery to fuse lumbar vertebrae using an intervertebral implant.

ALIF – *Anterior Lumbar Interbody Fusion*: LIF approach through the abdomen (anterior side).

LLIF – *Lateral Lumbar Interbody Fusion*: Minimally invasive LIF through the side via the psoas muscle.

MR – *Magnetic Resonance*: Imaging technique using magnetic fields to visualize soft tissues.

CT – *Computed Tomography*: X-ray-based imaging for detailed cross-sections of the body.

DICOM – *Digital Imaging and Communications in Medicine*: Standard format for medical images.

NIfTI – *Neuroimaging Informatics Technology Initiative*: Compressed file format for medical images.

AI – *Artificial Intelligence*: Algorithms used for automated tasks like image segmentation.

ROI – *Region of Interest*: Selected anatomical area for analysis in an image.

OOP – *Object-Oriented Programming*: Code structure based on objects containing data and methods.

IDE – *Integrated Development Environment*: Software application for writing and debugging code.

Index

Index of Figures

Index of Tables

Preface

Origin of the Project

The development of this tool actually originated from another student's thesis project [1], where a medical imaging analysis tool was created to automatically compare pre-and post-surgical images of individual patients and produce quantifiable results regarding the dural sac. The objective was to compare two types of interventions, minimally invasive spine surgery (MISS) or open surgery, and determine whether there was a quantifiable difference between the outcomes achieved by each technique. MISS involves smaller incisions and specialized instruments, while open surgery uses larger incisions and more traditional techniques.

While both techniques have their respective pros and cons, they relieve stress by removing bone and other soft tissue, thereby increasing available space for the spinal cord. In simple terms, the goal is to increase dural sac volume. Thus, **the original tool's main function was to calculate the pre- and post-operative volumes of the dural sac**. This allowed for an objective comparison of the two surgical techniques using quantitative measurements to evaluate whether one was more effective than the other. This type of tool did not exist at the time, making it a very relevant and necessary application. While this tool accomplished its primary objective of enabling quantitative comparative analysis, it did have several design limitations.

The first issue stemmed from the tool's extremely vertical and narrow code design, as the only function it could perform was the comparison of pre- and post-surgery images to calculate volume and shape change of the dural sac, making it extremely difficult to scale or adapt. Although the tool was very effective for its intended purpose, its great potential for broader applications could not be utilized. A volume and shape comparative tool could be extremely useful for surgeons to analyse all types of physiological bodies, and what became clear was that as long as we were able to segment a volume, similar steps and algorithms could be applied to perform these analyses. Furthermore, the strict structure of the code created a lot of dependencies between functions and processing steps, meaning that any change could cause the whole pipeline to collapse. This creates a situation where making improvements or adding new functionality becomes very difficult to implement.

The second limitation was related to user interaction. While most of the process was automated, there were still certain steps that required the user to manually interact with the application. Although there will always be a minimum degree of user input, it became evident that many of these interactions could be eliminated through further development. For example, during the setup phase of the analysis, the user was required to manually visualize all the series in a patient's CT scan and select the most

appropriate one for analysis. Or in another stage, the user had to manually crop a section of the spinal cord in a manner that was not particularly user-friendly.

A third major limitation was that the tool was designed to process only one patient at a time. This meant going through the same setup, image selection, cropping, and many other processes for each individual patient. While there are inevitably some aspects of the analysis that are patient-specific, for the most part, the same operations are being repeated for every patient, which seemed highly impractical. After all, to ensure fair comparisons and consistent results, the same analysis should be applied uniformly across all patients, and as is well known in research, a larger sample size yields more reliable conclusions.

In response to these limitations, we decided to build upon the pre-existing tool and develop a more robust and scalable version that would allow for improvements not only in the analysis itself but also in the range of possible applications. The goal was to create a tool that could be used not only for pre- and post-operative comparisons of spinal cord volume changes but also for analysing any other tissue of interest. The idea was to generalize the concept of volume and shape comparative analysis to a wide variety of future applications. Moreover, we aimed to make the tool as automated as possible, minimizing manual steps, and to enable the analysis of multiple patients simultaneously without the need to repeat the process individually for each one.

Motivation

Implementing all these improvements would result in a tool capable of performing high-quality morphological analyses of any tissue type in the body, which for surgeons and researchers represents an exciting prospect. By simply having pre- and post-operative images of a given structure, we could objectively assess the effectiveness of different surgical techniques or interventions through measurable changes in volume, density, or shape.

In the specific case of the *Hospital de la Santa Creu I Sant Pau*, where I have been working, they currently want to assess the impact of Lateral Lumbar Interbody Fusion (LLIF or XLIF) on the morphology and function of the psoas muscle and using this tool would be of great interest. LLIF is considered a minimally invasive procedure, as the intervertebral discal space is accessed through a lateral incision passing through the psoas muscle, minimizing tissue damage compared to traditional open surgery. However, the potential consequences of LLIF on the psoas muscle have not been fully studied yet and it is plausible to hypothesize that morphological changes such as atrophy could occur, in a quantifiable way. This presents a clear example of how the original idea behind the tool, comparing structural changes between two instances, can be extended to new investigations beyond the spinal cord.

Ultimately, the essence of this tool lies in its ability to quantitatively compare pre- and post-operative morphological changes across a tissue type. Whether it is evaluating the evolution of a tumour by analysing variations in size and density before and after treatment or studying the remodelling of a heart valve through shape changes after an intervention, the fundamental principle remains the same. By adapting the previous tool into a more robust and modular version, it is possible to lay the foundation for a wide range of clinical and research applications focused on structural and morphological analysis.

Previous Requirements

For the development of the project previous knowledge and understanding of medical image processing and segmentation was required. It was also useful to have a vast understanding of programming and system automatization.

Thanks to several subjects and projects in my biomedical engineering degree, I had already touched several of the topics and methodologies used throughout the project. This knowledge helped lay a foundation for my work and played a big role in the decision making and problem solving. Additionally, last year I completed my internship at the *Hospital de la Santa Creu I Sant Pau*, which was also focused on image processing, allowing me to further expand my knowledge and become familiar with the team.

1. Introduction

1.1. The Need for Quantitative and Accessible Imaging Tools

The basis of this project is to develop an automated medical imaging analysis tool designed for patient-specific volumetric and morphologic comparative studies between pre- and post-surgery images that is accessible to clinical practitioners or users without image processing experience. Furthermore, the aim is to implement a modular design structure that enables scalable processes, allowing the tool to be adaptable to specific needs and interests to support a wider range of studies and analyses. Currently, there are no applications with these characteristics. Those that do exist are either tailored to a specific anatomy or require a comprehensive understanding and intervention by the user.

For instance, **3D Slicer** [2] is a widely adopted open-source software for medical image visualization and processing. It provides a flexible environment for research and supports Python scripting, making it highly suitable for developing automated tools. However, in clinical context, learning how to use 3D Slicer involves a learning process and can even require programming knowledge for developing new tools, which makes it quite inaccessible for clinicians. Although full morphological analysis could be accomplished only through 3D Slicer, it is still very time consuming as it would forces the user to go patient by patient. 3D Slicer's scripting interface has been shown to fully automate functional steps, like the study by Chen et al. (2022) [3] that fully automates a morphological measurement system for the distal femur, or the original ComparativeCopilot tool [1]. This type of Python-Slicer paradigm is also used in the pipeline of this tool, as whole functional steps such as mesh generation and various crop modalities are executed within the 3D Slicer environment.

Another problem this project addresses is the lack of quantitative measurements to support the clinical evaluation of spinal surgeries. These interventions are typically evaluated purely of off qualitative measures which limits clinical research and creates a necessity for quantitative results. The tool designed in this project is capable of obtaining volumetric and morphological measurements of two significant anatomies of the intervened areas, the **spinal cord** and **psoas major muscle**, and thanks to its modular design has the ability to incorporate others. While a more in depth explanation of the surgery types and anatomies involved can be found in the section **2. Theoretical Framework**, it is important to understand that there is no tool that automatically measures spinal cord decompression (increase in volume after surgery) or psoas muscle atrophy (reduction in muscle mass and/or size). Not only does this tool do both, it can also perform statistical analyses for various patients at one time, opening the doors for clinical research.

1.2. Clinical Applications of the Designed Tool

Decompression of the spinal cord is usually obtained by one of two ways, direct decompression or indirect decompression. Direct decompression involves the physical removal of compressive bone or ligament tissue to directly relive pressure on the neural elements, while indirect decompression achieves this by restoring disc height and restoring spine alignment (These concepts are explained in depth in the section **2.Theoretical Framework**). There are studies that evaluate the effectiveness of indirect decompression after ALIF (Anterior Lumbar Interbody Fusion) by measuring dural sac cross sectional area (DSCA) [4] or quantitative clinical outcomes such as need of a second surgery for direct decompression [5], but none of them evaluate actual spinal cord volume change. Another study [6] compared a group receiving LLIF (Lateral Lumbar Interbody Fusion) with direct decompression and the other receiving LLIF with indirect decompression only, but once again only compared dural sac area together with pain and disability scores. This presents the first clinical application of the designed tool: **a volumetric and morphological analysis of the spinal cord to evaluate decompression after LIF.**

The second measurement the tool currently offers is related to the volume change in the psoas muscle pre- and post-surgery. During the LLIF approach the spinal cord and vertebrae space is accessed laterally through the psoas major muscle (explained in the **section 2.Theroetical Framework**). However few studies have been done evaluating the damage caused to this muscle, and those which have, measured leg strength to measure the recovery of the muscle [7] or morphological changes in psoas cross-sectional area [8]. Once again this shows a lack of quantitative results to properly evaluate the atrophy induced in the psoas muscle. Which presents the second clinical application of the tool: **a volumetric comparison between intervened and non-intervened psoas muscles after LLIF.**

1.3. Implementation Strategy

One of the key design choices that enables modularity, robustness and scalability is the use of **Object-Oriented Programming (OOP**). This approach enables patient data to be tracked in an organized way and allows patient-specific processing. Which solves many of the limitations identified in the original tool and adds an additional layer of functionality. The code is now structured around functional blocks and pipeline stages, which operate independently while still accessing patient information in a centralized way through the Patient class. This Independence also means that processes can be interchanged, modified or even skipped when necessary, without interrupting the overall pipeline. Additionally, this modular structure enables batch processing, as multiple patients can be processed simultaneously by simply creating additional objects within the same framework.

Another fundamental part of the design is the extraction of the region of interest (ROI) from the CT or MRI scan. This process is called segmentation and involves separating a specific anatomical structure from the rest of the image. In recent years, numerous AI models for segmentation have been developed thanks to the convolutional neural networks (CNNs) that often surpass traditional methods. **nnU-Net** [9] is a powerful deep learning framework that has redefined how many segmentation models are developed in medical imaging. This model focuses on auto-configuration, as it automatically adapts the neural network structure, pre-processing steps, training pipeline, and post-processing rules based on the properties of the dataset. **TotalSegmentator** [10] builds directly upon the nnU-Net framework and represents one of the most popular segmentation models currently available. It accepts both MRI scans but is more suited for CT images, and segments 104 anatomical structures, including 27 organs, 59 bones, 10 muscles, and 8 major vessels, all in a single pass. It achieved a mean Dice score [11] of 0.943 on the test set, significantly outperforming prior models. Importantly, TotalSegmentator is publicly available and easy to use, with implementations in both Python and 3D Slicer. While other segmentation tools were also evaluated because of their impressive performances [12] [13] the TotalSegmentator model was ultimately selected due to its exceptional performance and usability, as well as personal preference.

Considering future applications of interest to hospital surgeons and biomedical engineers, and after evaluating the current limitations of existing tools, a set of primary objectives was defined to guide the development of this project.

1.4 Objectives

The main objective of this project is to develop an automated medical imaging analysis tool designed for patient-specific volumetric and morphologic comparative studies between pre- and post-surgery images that is accessible to clinical practitioners or users without image processing experience. To reach this objective, the tool will focus on the following five criteria:

Modularity **–** Define independent processing blocks to allow for adaptable and personalized analyses.

The system was divided into pipeline phases and functional blocks, such as segmentation, meshing, or registration. Each block operates independently and can be modified or replaced without affecting the rest of the pipeline. For example, two completely different crop modalities, each using a different set procedure and set of functions, can be interchanged based on user preference. This structure ensures adaptability to various analysis types or research goals.

Scalability **–** Make the system extensible to support new analyses and adapt to any tissue or structure.

The tool was designed to be extended beyond the original use case. Although it originates from a spinal cord analysis, the modular structure allows for new processing steps to be added for different anatomical regions, making the pipeline applicable across diverse clinical settings.

Minimal user interaction – Automate repetitive tasks and streamline workflows for efficiency.

A key objective was to avoid manual steps such as Image selection, cropping, and image series visualization, by offering the option for automation. This reduces runtime and increases efficiency, making the tool more practical for users and better studied for large populations studies.

Batch processing – Enable the analysis of multiple patients simultaneously to support broader studies.

The tool supports the analysis of multiple patients at the same time, enabling large-scale studies with consistent processing parameters. This is essential for statistical robustness, reduced processing time, and also avoiding unnecessary repetition of manual tasks.

User Friendliness – Provide an intuitive structure with checkpoints and a clean, accessible interface.

Finally, the tool was designed with a clear directory structure and a clean output organization to make the tool accessible for users. The system is intended to be accessible to users with basic medical and technological knowledge who want to perform morphological analyses of physiological structures.

To test these objectives and really determine whether they are completed, the project includes the preliminary results of two possible studies using the new tool:

1. **A comparative study of direct vs indirect decompression surgeries**

2. **A study of iliopsoas muscle atrophy after LLIF surgery**

1.5 Project Scope

What the project covers

At its core, the tool performs a comparative morphological analysis using pre- and post-operative medical images. However, thanks to its modular design, it also allows users to personalize the pipeline and run only selected steps when required. It currently supports spinal cord and psoas structures, but has the potential to incorporate the analysis of any anatomical tissue or region as long as segmentation can be performed. The system is implemented entirely in Python and integrates external resources such as 3DSlicer and artificial intelligence models like

TotalSegmentator. It is primarily designed for volumetric and shape-based measurements, with the potential to incorporate additional morphological metrics obtainable from CT or MR imaging, such as density, angles, distances.

The tool currently includes the following core functionalities:

- Patient loading and configuration
- Pre/Post Image selection
- Segmentation of anatomical structures
- Mesh generation
- Image registration
- Region cropping
- Morphological and Shape analysis
- Batch processing and statistical analysis of multiple patients

The program tracks each patient's progress and results by using a structured Patient object and stores all data within organized folders. The tool is intended for clinical research and potentially for preliminary clinical planning in future versions, depending on the use case.

What the project does not cover

The designed tool does not perform medical diagnosis or automatically suggest treatments. It is not intended to replace clinical decision-making, but rather to support comparative morphological analysis and facilitate the development of research on surgical interventions or treatment effects.

The tool assumes that pre- and post-operative images are of sufficient quality and comparability for analysis. Although it's designed to be adaptable and open for future extensions, only some defined pipeline steps are implemented in this version.

2. Theoretical Framework

2.1 Relevant Anatomy

To put this project into context, there are some anatomical structures that must be introduced. First of all, it is important to recognize the vertebras that are involved in the surgeries (mostly lumbar vertebrae) as well as where the spinal canal is located and its surrounding structures. The lamina runs along the posterior part of the column closing the spinal canal and forming the foramen. Finally, the intervertebral discs sit between the vertebrae to absorb impact, ensure stability and proper spine alignment; its degeneration is one of the major causes of lower-back associated diseases.

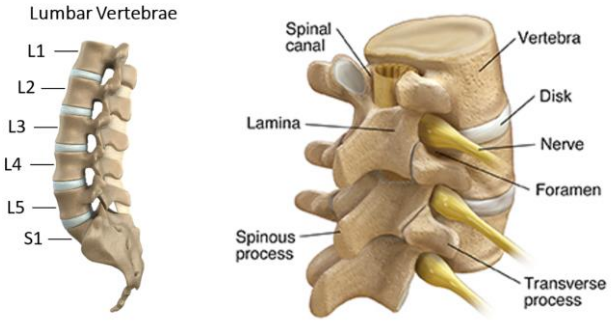


Figure 1: Visual representation of the anatomy of the lumbar vertebrae, spinal canal, lamina, foramen and intervertebral disc [14]

Inside the vertebral foramen runs the spinal cord, which is covered by a number of layers that protect it and the nerve rootlets. Both the arachnoid membrane and dura mater layers form the Dural Sac, a cylindrical structure that covers the spinal cord. Its thickness is smaller than 1 mm and even 0.3 mm in the lumbar region, which makes it practically impossible to differentiate from the spinal cord in MRI or CT scans. For this reason, the spinal cord and dural sac will be considered as the same, as this does not affect the results.

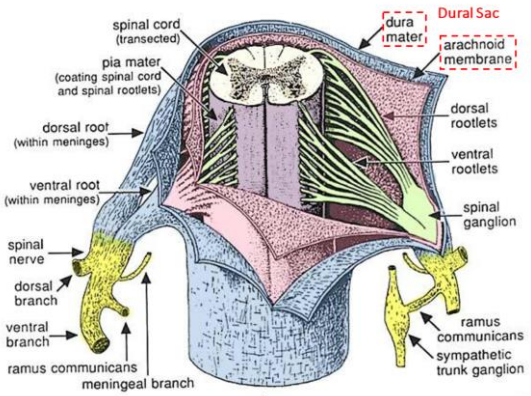


Figure 2: Visual representation of the anatomy of the spinal cord and dural sac with its components: dura mater and arachnoid membrane. [15]

Finally, the iliopsoas muscle is one of the most important muscles that overlie the vertebral column, as it is the main stabilizer in the core of the body. The psoas major is the elongated part of the muscle that attaches the lumbar vertebras (T12 – L5) to the femur [16]. The lumbar plexus is a complex of nerves that is intimately associated with the psoas major, as its branches emerge within the muscle. The Iliac Crest forms the lateral part of the pelvis and serves as an insertion point for various muscles, ligaments and fascia. These anatomies are relevant to certain interventions that insert the spinal cord and intervertebral space laterally, as the complex anatomy can interfere with the surgery.

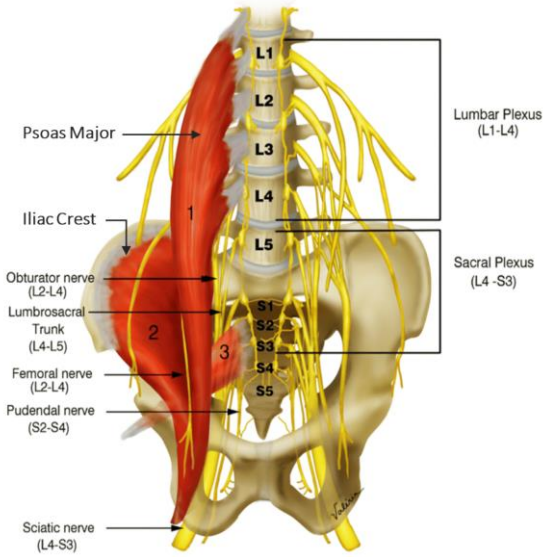


Figure 3: Visual representation of the anatomy of the psoas major muscle, iliac crest and lumbar plexus, and their connection to the lumbar vertebrae region. [17]

2.2 Lumbar interbody fusion Surgery Types

Lumbar interbody fusion (LIF) is a surgical technique used to stabilize the spine by joining (fusing) two or more vertebrae. This is achieved by inserting screws and rods into the vertebrae, as well as removing damaged intervertebral discs and implants (cage or spacer) into the disc space, restoring height and alignment. This technique is typically used to treat conditions such as disc degeneration, spinal instability, and nerve compression.

Currently, there are five main approaches to LIF [18]; posterior lumbar interbody fusion (PLIF), transforaminal lumbar interbody fusion (TLIF or MI-TLIF), oblique lumbar interbody fusion/anterior to psoas (OLIF/ATP), anterior lumbar interbody fusion (ALIF) and lateral lumbar interbody fusion (LLIF). There is no clear evidence for one approach being better than another, as they each possess their own advantages and disadvantages depending on the case.

Some of these techniques can be performed using minimally invasive (MIS) approaches, which uses smaller incisions to minimize damage of the surrounding soft tissue. For instance, the LLIF and OLIF approaches where specifically designed for MIS, but requires more complex procedures and specialized instrumentation. While these minimally invasive routes reduce trauma to the back muscles and often improve recovery, they introduce other risks. For example, the transpsoas route used in LLIF may cause temporary or permanent damage on the psoas major muscle.

2.2.1 Direct Decompression vs Indirect Decompression

In many cases, the aim is not only to stabilize the spine but also to decompress the spinal cord and nerve roots, either by directly removing bone or ligament, or indirectly by restoring disc height and alignment with an intersomatic cage.

- **Direct Decompression** usually involves a laminectomy or laminotomy, a procedure that involves removing part of the vertebral lamina to directly enlarge the spinal canal. This provides a more immediate relief by physically removing compressive structures and is frequently used to complement cage-based fusion techniques. [19]
- **Indirect Decompression** relies on the re-expansion of the disc space increasing foraminal and canal dimensions, relieving pressure on the dural sac and nerve roots without removing compressive structures.

2.2.2 ALIF and LLIF approaches

This project specifically focuses on **Anterior Lumbar Interbody Fusion (ALIF)** and **Lateral Lumbar Interbody Fusion (LLIF).**

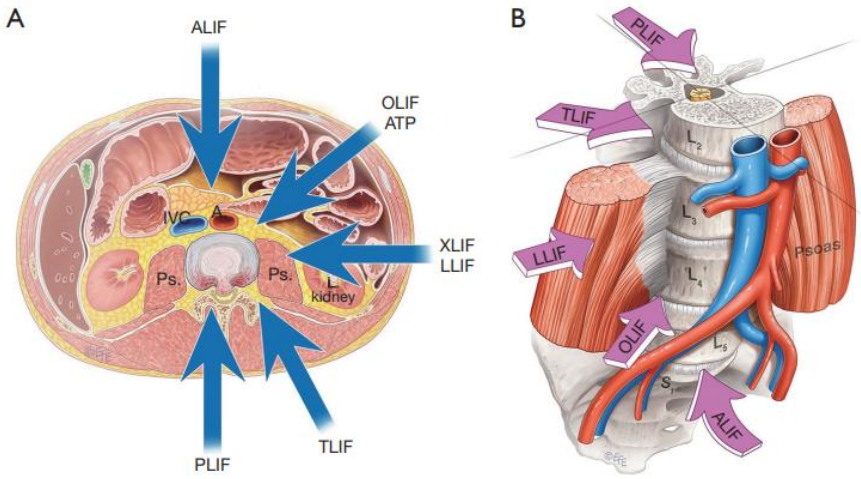


Figure 4: Visual representation of the different types of incisions during LIF surgery. As seen in image B, the ALIF approach enters through the L5-S1 level, while the LLIF enters laterally through the psoas muscle. [18]

The ALIF approach, the disc is accessed from the front of the body, by making an incision through the abdomen and between major blood vessels. It is particularly effective in the realm of discogenic lower back pain as it is most suitable for vertebrae levels L4-L5 and L5-S1 due to vascular anatomy interfering in superior levels. This surgery does not usually include direct decompression (Laminotomy or Laminectomy) as it does not access posterior structures, so in this project all the ALIF cases evaluated form part of a study on indirect decompression.

The second approach is LLIF (Also known as XLIF) [20], where the disc is accessed from the side and passes through the iliopsoas major muscle. It is suited for conditions that require access to the interbody disc space from T12 to L4-L5, but due to the location of the iliac crest L5-S1 level is not convenient. Additionally, the psoas muscle contains branches of the lumbar plexus, which adds a layer of complexity to the MIS approach. This surgery usually relies on indirect decompression, but in cases of severe central canal stenosis, it may be combined with a posterior laminectomy.

3. Materials

This section describes the resources and data used for the development of this project. The laptop used for the development and tests of the tool is a HP EliteBook x360 1030 G2, without GPU and with 8 GB of RAM. Because this is not a high-end computer with advanced graphical capacities, the tools efficiency and performance would improve significantly with a more powerful and dedicated GPU. The tool has been developed entirely in Python language in the Visual Studio Code IDE because of personal preference. However certain steps require tools from 3D Slicer and the code launches the application to perform them. The TotalSegmentator module must also be installed as well as the external Python libraries specified in the reuirementx.txt from the GitHub repository. (Explained in **Annex 4: GitHub Repository**)

Two datasets were used for both testing and obtaining results. The first group was formed by 19 patients that underwent ALIF surgery while the second contained 17 patients that went through the LLIF approach. These scans where provided by the team in the *Hospital de la Santa Creu I Sant Pau.*

ALIF Dataset

This group was formed of 12 males and 7 females, with an average age of 58 years. Only 3 patients of the ALIF group had a multilevel intervention with both L4-L5 and L5-S1 levels involved (all the rest were performed only in L5-S1). While some patients had complimentary procedures performed, none of them included a laminectomy (or laminotomy) making them all indirect decompression cases. All these cases disposed of both pre- and post-operative CT scans, however some scans presented corrupted files that only allowed 17 patients to be included in the results.

LLIF Dataset

The second group of patients consisted of 6 males and 11 females with a mean age of 67,4 years. Of the 17 patients, 4 of them had a multilevel intervention with one of the patients having 4 levels intervened in a single surgery. The most common intervened level was the L3-L4 level and few included the L4-L5 region due to the access limitations of the LLIF approach. There were 5 patients that included a laminectomy, forming the LLIF + Direct Decompression group while the others made up the LLIF + Indirect Decompression group. 13 of the patients where intervened on the left side, while 4 on the right (affecting the left or right psoas muscle respectively). Finally, once again even though all these cases had both pre- and post-surgery CT scans available, 4 patients contained corrupted files reducing the number of available LLIF cases to 13.

4. Methodology – System Design

The overall structure of the tool is divided into three distinct phases to improve the workflow and enhance usability, given the high number of processing steps and functional components involved. The first phase is the **Input Phase**, responsible for loading patients and organizing input data within the tool's workspace. Although it may not be immediately apparent, one of the most time-consuming steps for clinicians is the creation of a structured database that can later be processed efficiently. This is partly due to the heterogeneity of medical images, which can vary significantly depending on the acquisition device. Additional time-consuming tasks include the need to rename files, ensure proper anonymization, and maintain consistent data structuring while managing file sizes.

This is followed by the **Setup Phase**, where the configuration of the desired analysis is determined, and finally, the **Execution Phase**, where the actual analysis takes place.

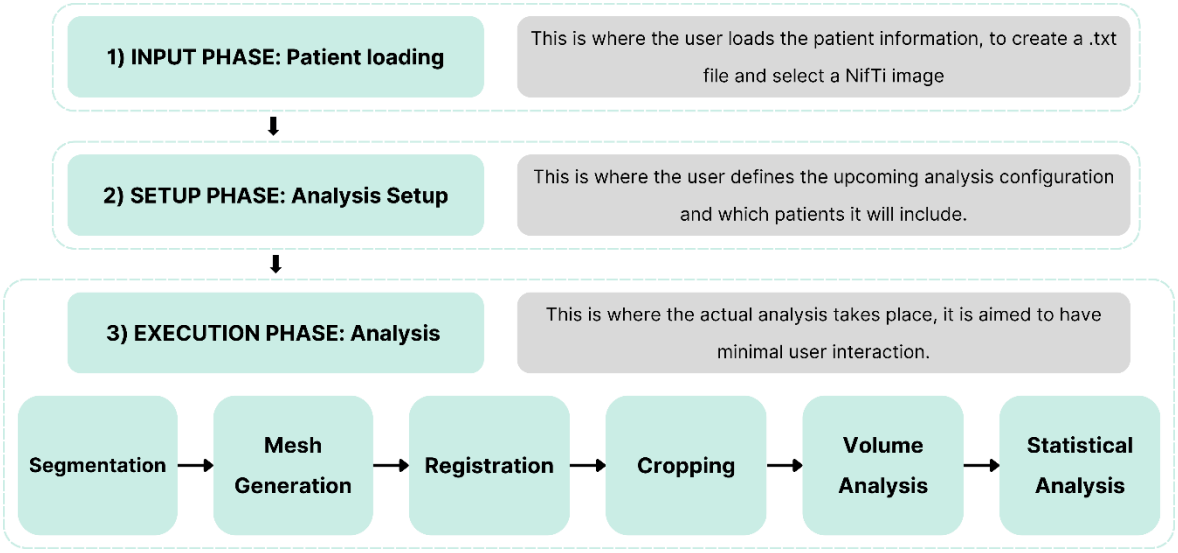


Figure 5: Visual overview of the main pipeline phases. The bottom of the image shows the specific analysis steps for the execution phase.

The purpose of this structured approach is to provide users a clear and intuitive workflow that guides them through the various stages of the tool without requiring an in depth understanding of the code itself. There is also a recurrent effort to maintain the flow of data between these phases as logical and structured as possible, promoting reusability while minimizing redundancies. This is achieved through the creation of Patient and Visit classes (explained in the section **5.1 Patient and Visit Classes**), where patient metadata and files are stored as attributes, as well as TXT and JSON files where information is physically stored for reusability.

4.1 Input Phase: Patient Loading

The primary function of the Input Phase is to load patient-specific data and select the pre- and post-operative images that will be used in the subsequent analysis. This phase establishes the foundation for all the following processing steps, ensuring that the necessary data is accessible and structured in the correct manner for further analysis. Another important role of this phase is to reduce file sizes by selecting a single image per visit and converting DICOMs to compressed NifTi files.

Due to the variability of patient data and the need for some specific information that cannot be extracted directly from imaging files, some minimal input is required. Specifically, the user must provide the following data:

Type of surgery performed: This is used to categorize patients into groups based on the surgical technique performed (e.g ALIF, LLIF, OPEN).

Vertebral levels involved: The specific vertebral levels where the surgery was performed (e.g L3-L4, L4-L5). This data is essential for targeting segmentation and locating the expected morphological change.

Patient ID: To ensure anonymity, the user assigns a numerical ID which is then used throughout the pipeline for tracking.

Raw Image paths: The user must indicate the location of the raw image data specifying which folders contain PRE and POST visits by naming them PRE and POST. These folders can contain any type of files or structures, as the code later organizes and extracts the necessary images.

4.1.1. Patient Loading Modes

The process of loading patient specific information can be done in two ways: **Batch Loading** and **Individual loading**. Both modes execute through *high level orchestration functions*[1] called ***batch_patient_prep()*** and ***individual_patient_prep()***, which manage data extraction, file organization and image selection of each patient. The detailed implementation of these functions will be further elaborated in the section **Modular Blocks and Functional Design**.

Batch Loading

If user selects Batch Loading, the function ***batch_patient_prep()*** is executed. This function expects two inputs:

- **A CSV/Excel File containing all patient-specific metadata**
- **A path to the raw data folder containing all patient images**

The excel file is the key component of the input structure for batch processing, as it allows the loading of many patients at once. The format of this excel must follow a certain template, as seen in the Table below:

NHC	PatientID	VisitType	CaseType	Levels
(Patient NHC)	(Any number)	(Pre folder/Post foler)	(Surgery Type)	(SupVert-InfVert) (Use / for multilevel)
NHC1	1	PRE/POST	LLIF	L4-L5
NHC2	2	PRE/POST	LLIF	L3-L4/L4-L5
NHC3	3	PRE/POST	ALIF	L3-L4
NHC4	4	PRE/POST	ALIF	L3-L4

Table 1: Bath Input Template for Patient Metadata, indicating required fields and their respective formats.

[1] High level orchestration functions coordinate multiple underlying functions. This way, long functions that can create dependencies and reduce modularity can be avoided.

Table 1 shows the necessary structure and contents of the Excel file for the Batch Loading of patients to work correctly. However, the first column introduces an additional data field not mentioned before, Numero Historial Clinico (NHC), which is a unique identifier used solely for batch loading to match patient folders in the raw data directory. For this to work, each patient folder inside the raw data directory must have their own NHC as name. This still keeps anonymity of Patients, as it is only used for the identification and loading phase, as later on the desired Patient ID is used for all processes eliminating the use of the NHC.

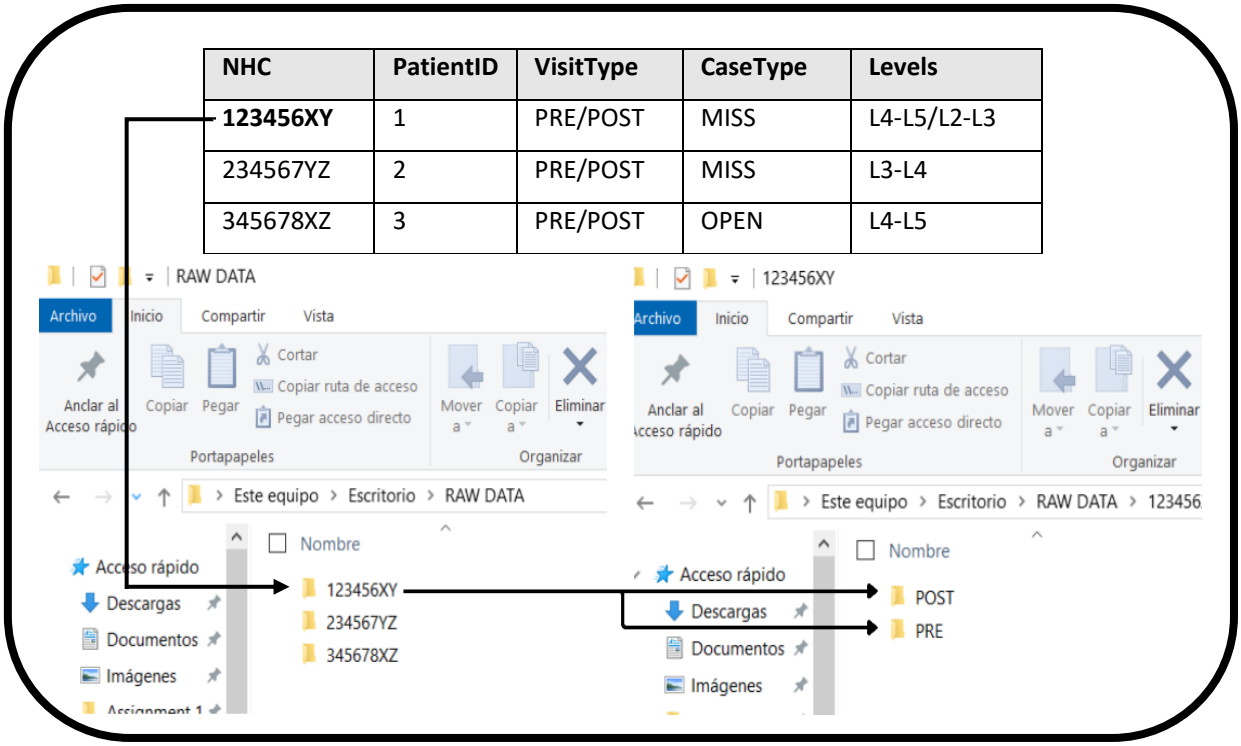


Figure 6: Visual example of patient folder identification using NHC number. The top table shows the excel template with patient metadata. The arrows indicate how the NHC number is used to identify the pre and post image folders.

In practice, the tool iterates through each row of the Excel file, identifying the corresponding patient folder in the raw data directory using the NHC value. This allows the user to organize a large data set with relative ease and without having to manually select directory paths for each patient. It also classifies the patients in groups by Case Type. This case shows a classification of patients who underwent MISS (Minimally Invasive Spine Surgery) and OPEN (Traditional Open surgery).

Individual Loading

If the user selects Individual Loading, the function ***individual_patient_prep()*** is executed. This function is intended for specific occasions such as testing or when a patient has corrupt files and must be reloaded. It gathers the same information than in batch loading without the need of an Excel file, as the user manually inputs the required data:

- Patient ID
- Surgery type
- Vertebral levels
- Visit types (PRE/POST)
- Raw image path

This mode does not require the use of the NHC as the user manually introduces the raw data directory. The idea is that manually introducing information is not an issue when processing a single case, but becomes repetitive and inefficient when loading more patients, which is why the Excel file method was designed for batch loading.

4.1.2. Image selection

After loading patient metadata, the next step in the Input Phase is image selection. This step is necessary because a single CT or MRI scan usually contains multiple image series, each with different characteristics, resolutions, and orientations. These series occupy a significant amount of unnecessary storage space, as the tool only uses a single pre- and post-operative image throughout its whole pipeline.

To maintain data consistency and anonymity, all selected images are saved in **NIfTI format** (.nii.gz). If the input data is provided in **DICOM format**, it is automatically converted during the loading phase. For research and development purposes (such as this project), anonymity is crucial, as it must comply with the Organic Law on the Protection of Personal Data and Guarantee of Digital Rights [21], which aims to preserve patient privacy.

Selecting the optimal series is important to ensure consistency and the best possible quality. The tool offers two modes of image selection: Manual selection and automatic selection.

Manual Selection

The user is presented with a preview of all the series within the specified PRE and POST folders for each patient. They must manually select the appropriate series for their analysis based on image quality, clinical relevance, or any factor deemed valuable for the user. This is the original mode for image selection which clearly presents an efficiency problem if processing a batch of patients, as the user is required to go through multiple images for each pre- and post-operative visit, and repeat the process for multiple patients.

```
🧠 Selecting series for Patient 9 — PRE
1. Lumbar  1.0  MPR  columna multi — DICOM — Shape: (512, 512) — Voxel size: (0.344, 0.344, 1.0) — Num slices: 507
2. Lumbar  2.0  MPR  sag — DICOM — Shape: (917, 512) — Voxel size: (0.477, 0.477, 2.0) — Num slices: 86
```

Figure 7: Snapshot of the displayed image characteristics for manual image selection.

Figure 8: Examples of displayed images during manual image selection.

Automatic Image Selection

Automatic Image Selection reduces user control, it radically improves efficiency and reduces user interaction. A predefined criteria is established to simulate the quality inspection that a user would normally do, primarily based on two factors. The first is the total anatomical layers contained by the image, which is also understood as the total number of voxels, as more voxels usually means more information. And the second criterion is the resolution of the image, which is essentially the size of these voxels, as a better resolution should mean better quality. The combination of a high number of voxels and a low voxel size (high resolution) would establish a high quality image for the subsequent analysis. To follow this criterion the necessary information can be extracted from the image metadata, and then this two stage selection can be applied.

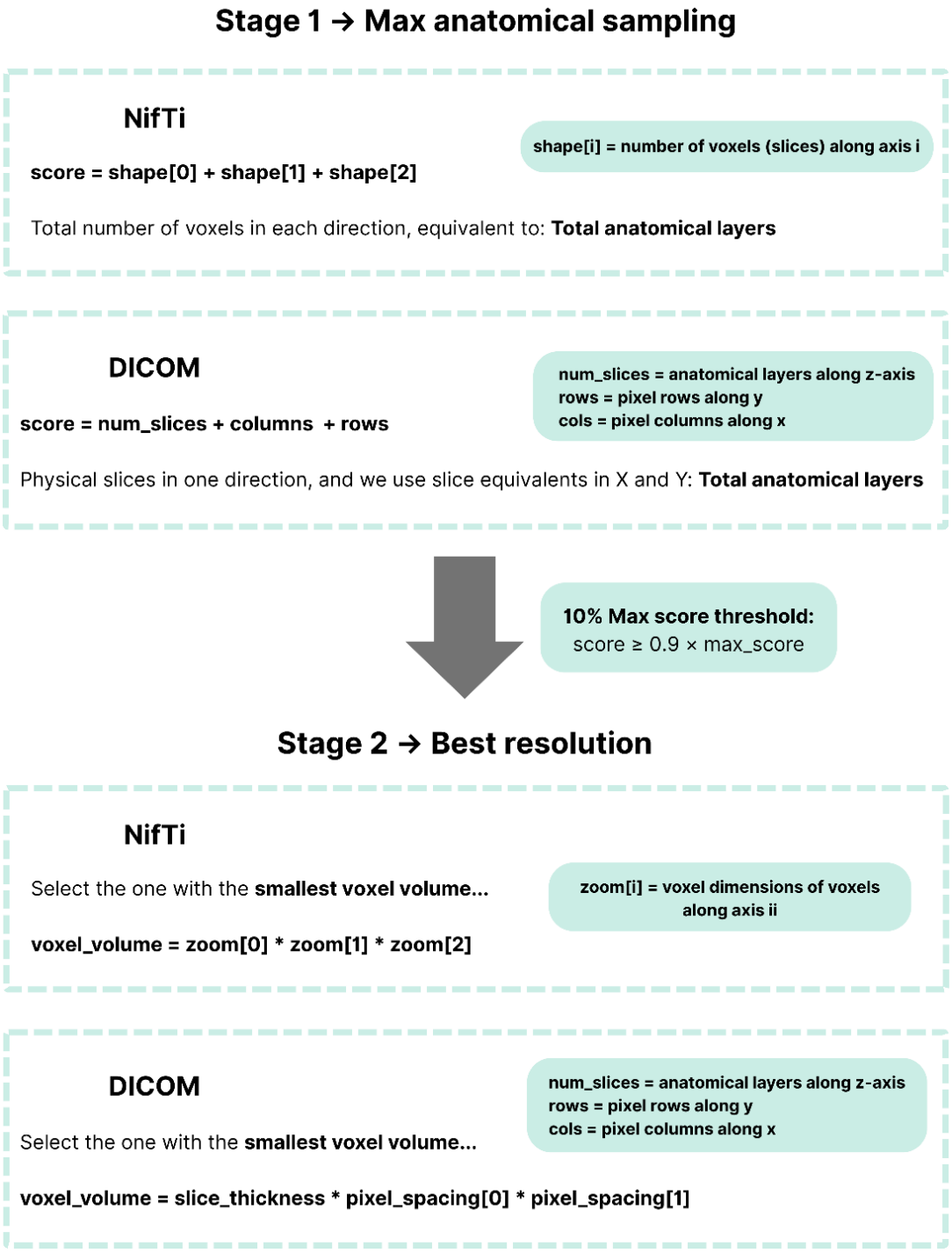


Figure 9: Visual representation of the automatic two stage selection process.

This automated selection process is performed by the function ***automatic_image_selection()*** which applies this two stage selection algorithm. First it selects the images with the highest number of anatomical layers by accessing the image metadata, both for NifTi and DICOM series. Finally, from the highest scored series it selects the image with the lowest voxel volume which is equivalent to the best resolution.

4.1.3. Data Organization and Folder Structure

Regardless of the loading mode or image selection method, the tool organizes patient data into a standardized folder structure. This structure ensures that data is consistently organized and accessible during the Setup and Execution Phases.

Before any patient data is loaded, a **Results** folder is created within the main directory. This folder serves as the central repository for all patient data, selected images and future analysis results. If this folder already exists, new Patients are loaded to this existing directory. In cases where a patient folder already exists for a given ID, the user can choose to overwrite the existing data or skip the loading process. This overwrite prompt is one of many progression checks implemented to prevent accidental data loss or duplication, and for them to function consistent structures are essential for a correct data management throughout the pipeline.

The structure is as follows:

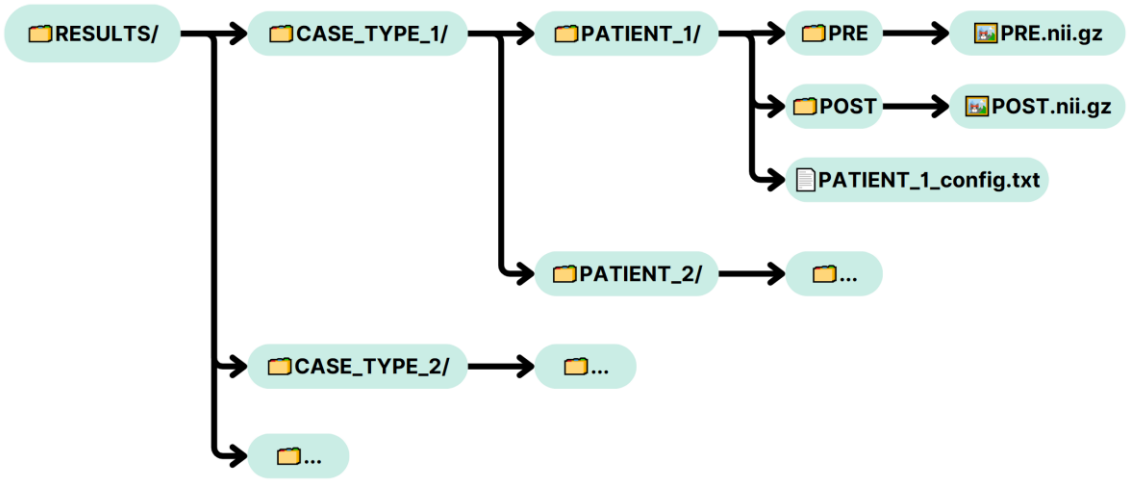


Figure 10: Visual representation of the folder structure created during the loading phase. Selected images are saved as NifTi files and a txt file with loaded patient metadata is also saved in patient folders.

Data Structure and Contents

Results Folder: The main directory containing all patient data, organized by case type (e.g., MISS, OPEN, LLIF).

Patient Folder: Each patient is assigned a unique folder named using their **Patient ID**. This folder is created during the Input Phase and serves as the centralized location for all data related to that patient throughout the pipeline.

PRE/POST Subdirectories: Each patient folder contains subdirectories named PRE and POST (or as specified during the loading phase). After patient loading these folders store the selected pre- and post-operative images in **NIfTI format**, ensuring data anonymity and format consistency.

Configuration File (config.txt): A text file containing the loaded metadata for each patient. This includes the assigned Patient ID, surgery type, vertebral levels, and image paths. This configuration file is essential for ensuring that each patient's data is accurately referenced during the Setup and Execution Phases.

4.1.4. Loaded Patients Excel File

The code automatically generates an Excel File containing all loaded patients and their current progression. By default, this file is located in the desktop and is updated whenever a new patient is loaded or whenever an analysis step is executed. This serves as an accessible and understandable way of tracking patient progression and improves reusability for different analysis.

Patient_ID	Case_Type	evels_Intervene	PRE_NIfTI	POST_NIfTI	PRE_Seg	POST_Seg	PRE_Mesh	POST_Mesh	Registration	Crop
21	XLIF	L3, L4	✓	✓	✓	✓	✓	✓	✓	✓
22	XLIF	L4, L5	✓	✓	✓	✓	✓	✓	✓	✓
23	XLIF	L3, L4, L4, L5	✓	✓	✓	✓	✓	✓	✓	✓
24	XLIF	L3, L4	✓	✓	✓	✓	✓	✓	✓	✓
25	XLIF	L3, L4	✓	✓	✓	✓	✓	✓	✓	✓
26	XLIF	L2, L3	✓	✓	✓	✓	✓	✓	✓	✓
27	XLIF	L2, L3	✓		✓		✓			
28	XLIF	L3, L4	✓	✓	✓	✓	✓	✓	✓	✓
29	XLIF	L3, L4	✓	✓		✓		✓		
30	XLIF	L3, L4	✓	✓		✓		✓		
31	XLIF	L3, L4	✓		✓		✓			
32	XLIF	L3, L4, L4, L5	✓	✓	✓	✓	✓	✓	✓	✓
33	XLIF	L4, L5	✓	✓	✓	✓	✓	✓	✓	✓
34	XLIF	L3, L4	✓	✓	✓	✓	✓	✓	✓	✓
35	XLIF	L1, L2, L2, L3, L	✓	✓	✓	✓	✓	✓	✓	✓
36	XLIF	L2, L3	✓	✓	✓	✓	✓	✓	✓	✓
37	XLIF	L3, L4	✓	✓	✓	✓	✓	✓	✓	✓

Table 2: Example Excel file showing the completed steps for each patient. The file is updated every time the user returns to the main menu.

4.2. Setup Phase: Analysis Setup

The Setup Phase is designed to gather all necessary information for the desired analysis. Specifically, it allows the user to:

- Let the user choose which patients to analyze.
- Let the user define which analysis steps to run.
- Load or create Patient objects accordingly.
- Configure parameters like segmentation or crop mode.
- Log this setup in a structured way for the execution phase.

The objective is for all user interaction to occur during this phase, allowing the following Execution Phase to proceed as an automated pipeline.

4.2.1. Patient Selection

This phase begins by presenting the user with a list of previously loaded patients, each represented by a TXT configuration file created in Phase 1. As explained in the Input Phase, each loaded patient should have a predetermined folder structure with PRE and POST visit subdirectories, and a TXT configuration file. This file contains metadata such as patient ID, case type, and vertebral levels and is used to identify patients for analysis in the Setup Phase. If the TXT file is not found, the patient cannot be selected.

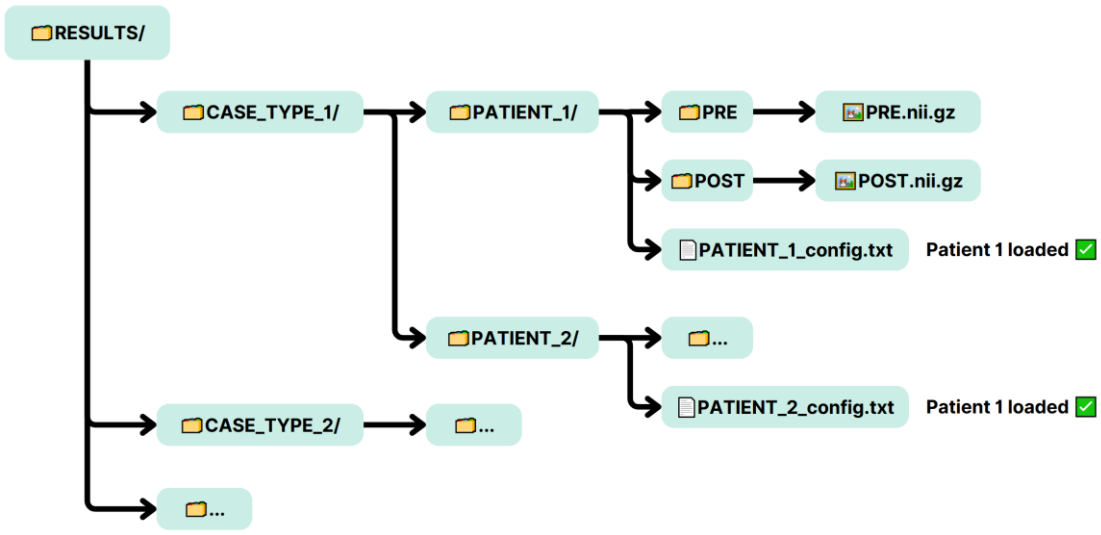


Figure 11: Diagram of the folder structure containing the loaded patients during phase 1. The code uses the configuration .txt file as an identifier; if this file is not found, the patient cannot be selected for analysis.

The system lists all patient .txt configuration files found, loaded previously during Phase 1. The user can then choose individual patients or select “All”.

4.2.2. Step Selection

Once patients are selected, the user is given the choice to select the analysis steps that will be performed during the Execution Phase. By default, the tool executes all steps as they are all necessary to obtain the final analysis results, but the option is given for users who want to experiment with the tool.

The available steps, as defined in the execution phase include:

- **Segmentation**
- **Mesh generation**
- **Registration**
- **Crop Section**
- **Patient Volume Analysis**
- **Statistical Analysis**
- **All**

This modular selection allows for partial re-analysis if the process is interrupted and helps prevent redundant computation (e.g., skip Segmentation if already done). The selected steps, together with the selected patients are passed on to the final phase to initiate execution.

4.2.3. Analysis Recycling – JSON Checkpointing

A core feature of the Setup Phase is determining whether a patient has already undergone part of the analysis process and whether reprocessing would be redundant. To facilitate the recycling of previous analysis, a JSON checkpointing mechanism is incorporated to store patient analysis progress.

At the end of the Setup Phase, a patient object is created where all patient metadata and directories to the saved files are stored in a structured and efficient way. However, to not lose this information a JSON file is also created (from the object) as a checkpoint where the object's information is saved. As the patient moves through the pipeline new files are added to the Patient folder, and so both the patient Object and JSON file are updated. This allows for constant saving of progress.

This allows for a progress check during the Setup phase, as once the user has selected the patients for analysis, the tool searches for each patient's JSON file. If the JSON file doesn't exist the code understands this is a newly loaded patient and it is added to the new_patients list. However, if the JSON file does exist this means the patient has already gone through some degree of analysis giving therefore re-using the patient object is an option. The user is given three options:

1. **Overwrite Existing Progress:** Delete all previous files and results using the ***clean_patient_analysis_files()*** function and treat the patient as a newly loaded patient.
2. **Use existing progress:** Continue from where the analysis was left off, and reuse the previously obtained results by using ***load_from_json()*** function.
3. **Skip Analysis:** Exclude the patient from further analysis if the existing results are sufficient.

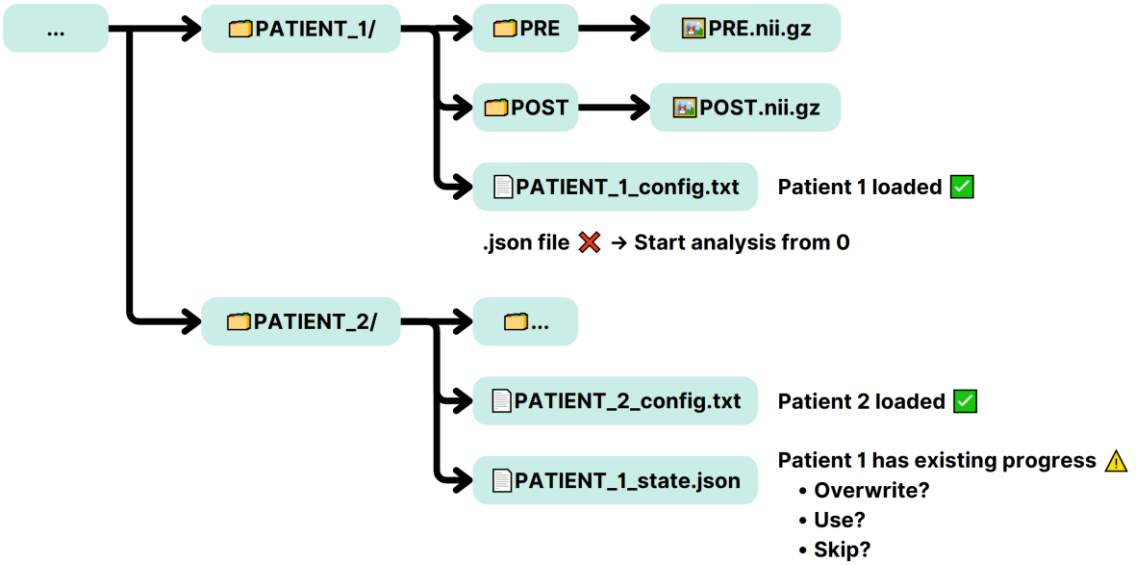


Figure 12: Example of the folder structure showing the .txt config file and the resulting JSON checkpoint.

4.2.4. Configuration Setup

After patient and step selection, the user proceeds to define specific configurations for the selected analysis steps. If the user decides to use existing progress of existing patients, these will not be included in the re-configuration. The first and most important is segmentation, as it lays the foundation for subsequent analysis steps.

Segmentation Configuration

If the segmentation step is selected, the user will have to set up the segmentation parameters. This configuration process is handled by the ***get_segmentation_config()*** function, and involves the selection of the following parameters:

- **Segmentation tool:** Currently, the code uses TotalSegmentator, but other AI models could be incorporated, specialized for different anatomies or image modalities.
- **Imaging Modality:** TotalSegmentator admits both CT and MRI scans, but works best for CT. The user must specify the modality to avoid inconsistencies in segmentation results.

- **ROI:** The main anatomical structure that the user wants to analyse. Currently only spinal cord and iliopsoas are implemented, but future expansion could include additional structures.
- **Other Anatomical Structures:** This option allows the user to include the segmentation of other regions that are not directly used in analysis. These can be visualized manually but are not analysed.
- **Fast segmentation:** This enables a lower-resolution segmentation mode that speeds up the process, useful for test runs or batch processing.

These parameters are saved in the patient object as attributes, as they are seen as patient-specific.

Crop Modality

The modularity of the tool allows for different ways of executing the analysis steps. The cropping step can be performed in different modes depending on the desired level of control or automation. Although the specifics will be explained in the execution phase, here the user can choose the preferred method. The tool currently offers three Crop modalities:

- **Scissor Crop:** The original cropping modality, a manual circular cropping.
- **Plane Crop:** Define two planes to crop regions above and below, allowing for more targeted analysis.

Once patient selection and configuration are completed, the Setup Phase returns the selected patients, analysis steps and crop modality, passing them into Phase 3 for automated execution.

Statistical Analysis Setup

The user can select the type of statistical analysis depending on the intended study. Different statistical analyses can be designed and added into the pipeline for different purposes. The current version includes:

- **Comparative Surgery Spinal Cord:** This is a comparative statistical volumetric and morphologic analysis specifically designed for spinal cord.
- **Psoas Atrophy Analysis:** This statistical analysis is designed to compare intervened psoas segments with its non-intervened pair.

4.3. Execution Phase: Analysis Execution

The execution phase is the core of the analysis tool, where all imaging processes are automatically carried out based on the previous phases. It receives as input the list of Patient objects and the selected analysis steps defined during the Setup Phase. The goal of this phase is to perform the selected analysis steps with no user interaction, ensuring consistency and batch processing capability. It follows a typical medical image process as the pre- and post-operative images undergo:

- **Segmentation**
- **Mesh Generation**
- **Registration**
- **Copping**
- **Patient Volumetric Analysis**
- **Statistical Analysis**

These steps form the general processing workflow, but the actual execution might vary depending on the selected morphology to undergo analysis (ROI) and the configuration defined in the previous phase. For example, certain ROIs may require a different type of cropping or registration strategy. The pipeline is intentionally designed to be adaptable, supporting anatomical flexibility and scalability.

This version of the tool builds upon the previous implementation focused on the spinal cord. That implementation was tailored specifically to the spinal cord structure and was limited to a single configuration. The new modular system is designed to allow the analysis of new anatomical structures as well as new configurations to improve results. This is demonstrated by incorporating the iliopsoas muscle as a possible ROI.

4.3.1. Segmentation

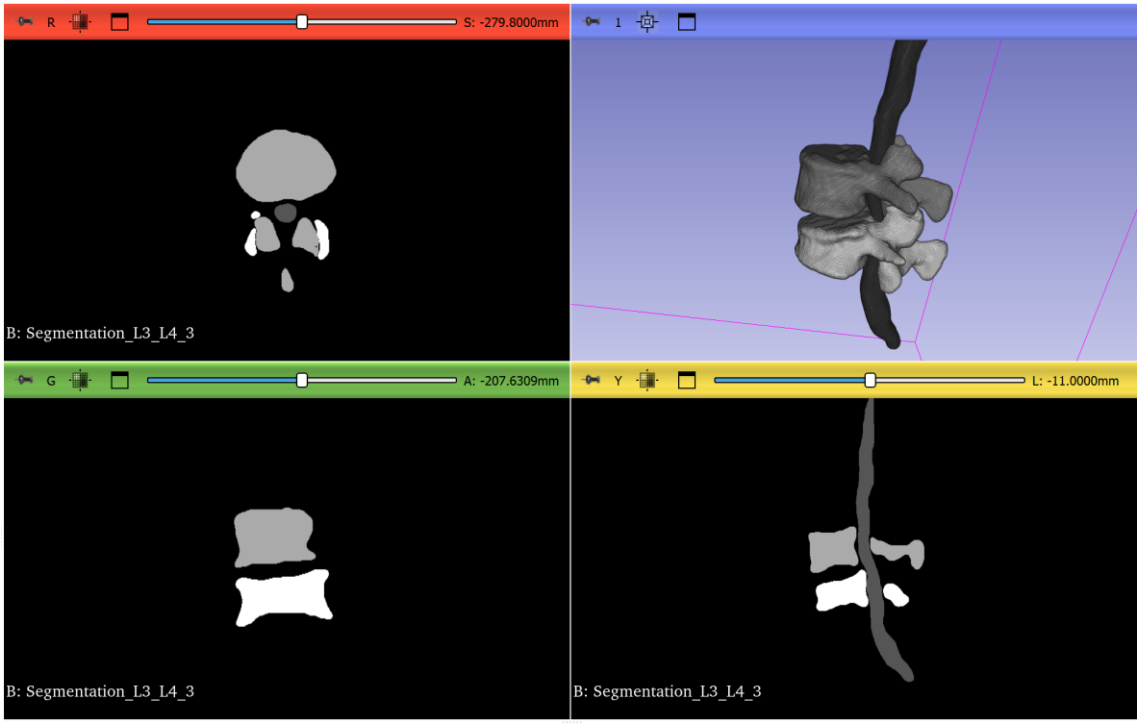


Figure 13: Example of the segmentation of the spinal cord together with the L3 and L4 vertebras.

Segmentation involves extracting the selected anatomy (Spinal Cord or Iliopsoas) and the vertebras of the level intervened from the pre- and post-operative volumes. There are many segmentation tools designed for this process but currently only TotalSegmentator has been implemented in the pipeline. This tool requires the user to define parameters such as structures of interest, image modality, output paths, etc. but thanks to the loading and setup phases this is executed automatically. The ROI subset to be segmented will be different for each patient depending on its predefined metadata and the configuration set up by the user.

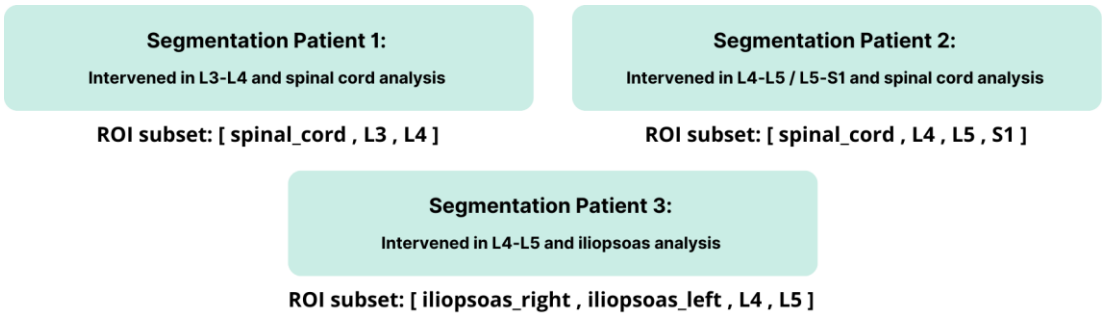


Figure 14: Examples of different ROI subsets the pipeline can manage.

Segmentation post-processing

TotalSegmentator returns an individual binary mask for each structure of the ROI subset. To simplify the post-processing process the different segments are combined, obtaining a single mask where the individual masks are encoded. Each individual segment is given a unique value so that they can still be distinguished in the future. This process is done by the functions:

- ***combine_segmentations_spinal_cord()***
- ***combine_segmentations_iliopsoas()***

Spinal Cord Segmentation

Segment	Value
spinal_cord	1
Vertebrae_X	2
Vertebrae_Y	3

Iliopsoas Segmentation

Segment	Value
iliopsoas_right	1
iliopsoas_left	2
Vertebrae_X	3
Vertebrae_Y	4

Figure 15: Visual representation of the values given to each binary mask after segmentation.

Then the combined segmentation undergoes smoothening, by applying a low-pass Guassian filter over all segments. This is a necessary step for images obtained by algorithms such as the ones applied by TotalSegmentator, as results tend to contain noise and have a pixelled finish. The smoothening technique helps to mitigate over-segmentation and is essential to improve the quality of results by making the segmentation more coherent, less fragmented and less noisy.

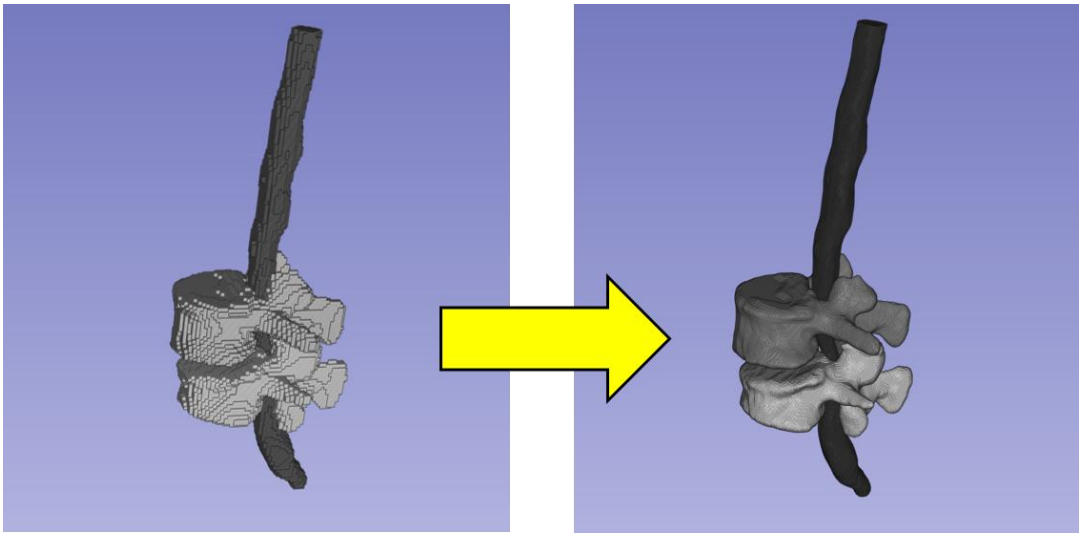


Figure 16: Visual representation of the smoothening algorithm applied in post-segmentation process.

4.3.2. Mesh Generation

This step of the analysis consists in converting the acquired segmentations into a 3D Mesh. The difference between volumes/segmentations and 3D meshes is that the first are represented in voxels, while meshes are a combination of structured vertices (points in space) that are attached forming surfaces in between them. This makes meshes easier to manipulate in space, different to Volumes which are more suited to quantitative analysis. This conversion is necessary as the following registration algorithm consists of superposing the pre- and post-operative segments, and can only be done with 3D mesh elements.

To complete the mesh generationprocess a python script is automatically executed through 3D Slicer. While this software can be used for many purposes one of them is for mesh generation using internal Slicer functions. These functions rely on a variation of the Marching Cubes algorithm, a well-established method for surface reconstruction from voxel data [22]. 3D Slicer's **vtkDiscreteMarchingCubes** algorithm is specifically designed to handle discrete (labelled) voxel values, which is ideal for medical segmentation masks.

The mesh generation is completed by the MeshGeneration.py script where each anatomical label is transformed into a polygonal mesh (.obj file) representing its outer boundary. After the mesh process the combined segmentation is converted into individual 3D meshes and saved into a Mesh folder in the Patient directory.

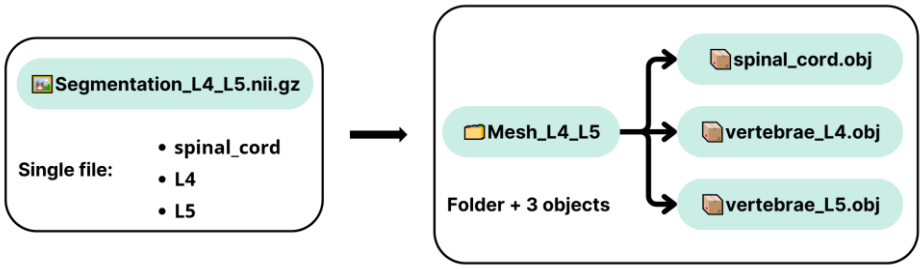


Figure 17: Visual representation of how the segmentation goes from a single NifTi file to a folder containing a group of mesh objects.

4.3.3. Registration

This is the first step that involves combining both pre- and post- operative data. Up until this point, each visit was processed independently, with its own segmentation and mesh outputs. Registration brings both volumes (or derived meshes) together by spatially aligning them using registration techniques. This ensures that the corresponding anatomical structures are overlaid properly, which is critical for detecting the morphological and volume changes caused by surgical intervention.

The registration process applies the Iterative Closest Point (ICP) algorithm [23] to achieve spatial alignment, by computing a transformation matrix that maps the post-operative anatomy over the pre-operative one (or vice versa). This matrix is derived by maximising the overlap between a source object and a mobile object. Once the optimal alignments are found, the resulting transformation matrix can be applied to any related structure.

During the development of the original tool multiple challenges were encountered when trying to register spinal cord segments directly. Most CT or MR scans do not cover the same regions in each visit, which can generate problems in the maximal coincidence of the pre- and post-operative spinal cord segments. For example, the spinal cord is a very long structure, meaning that if one of the images only covers the Lumbar Vertebras and another covers part of the Thoracic Vertebras, the size discrepancy will affect the superposition.

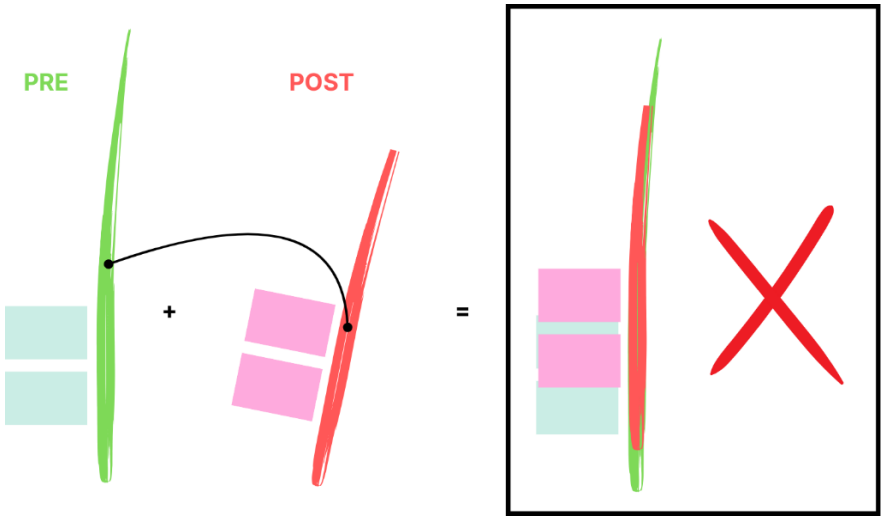


Figure 18: Visual representation of misalignment if spinal cords are cut-off at different length. After registration the discrepancy between the positions of the vertebras shows a tendency for great error.

Something similar occurs during psoas registration as this inconsistency is also seen as most scans cut off the psoas somewhere around the femur head region but not always in the same point. This creates an inconsistency between the two structures which makes the registration incorrect.

To overcome these issues, the final registration method uses vertebrae as reference points. These rigid bony structures are anatomically stable and consistently segmented, making them ideal for spatial alignment. Specifically, the process registers vertebrae pairs corresponding to the operated levels and applies the resulting transformation matrix to the surrounding soft-tissues structures (e.g., spinal cord, iliopsoas muscles). This is one of the main reasons why the segmentation includes the vertebras apart from the selected ROI, as they are an essential part of the registration process.

Some of the advantages of Vertebra-Based Registration are:

Anatomical consistency: Vertebrae tend to remain unchanged between scans and can be reliably identifies and segmented.

Region-specific accuracy: Using intervened-level vertebrae the registration is optimized exactly where the surgical changes are expected.

Robustness: The same methodology can be applied for any tissue surrounding the vertebras, as clearly demonstrated with the iliopsoas muscles.

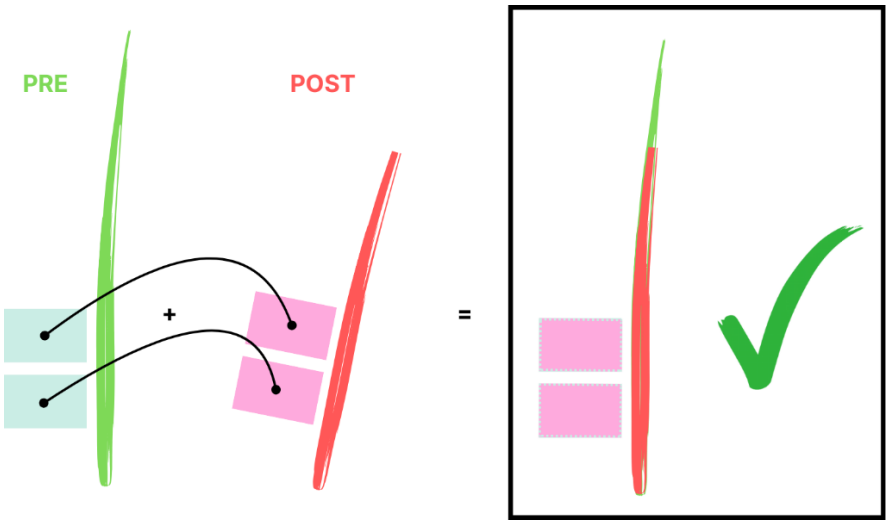


Figure 19: Visual representation of the designed registration process. By aligning the vertebras, the spinal cord will align "indirectly".

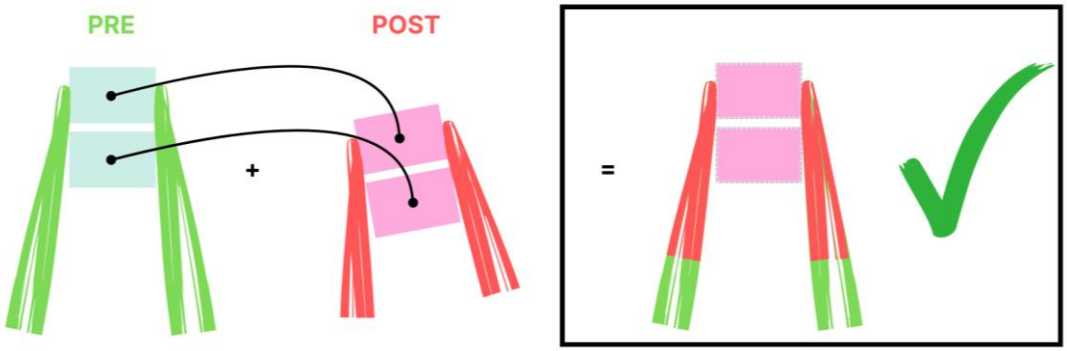


Figure 20: The same process can be applied for the iliopsoas muscles, as their insertions on the bone will not change.

4.3.4. Cropping

Once the selected anatomy has undergone the registration process, a cropping step is introduced to focus the analysis on a specific region. This ensures that the following morphological analysis is performed on a consistent anatomical section for both visits. Depending on the user's configuration the cropping may be:

- **Scissor Crop** (semi-automatic)
- **Plane Crop** (semi-automatic)

Scissor Crop

This Crop process was developed entirely in the previous version of the tool, and was integrated into the pipeline as a function called ***scissor_crop_process()***. This process is done through 3D Slicer and its module SegmentEditor [32], where the scissor tool is used to edit the registered segmentations.

Similar to the mesh process, a python script called ScissorCrop.py is executed through the 3D Slicer python interpreter, automatically setting the scene for the user to perform the crop. Both pre- and post-operative fragments are shown at the same time, as the crop action is performed once for both segments, ensuring that the delimited limits for the upper and lower boundaries are the same for both structures. The original NifTi image and the pair of vertebrae in the intervened level are also included in the visualization to facilitate visualization and as a reference point for the cut.

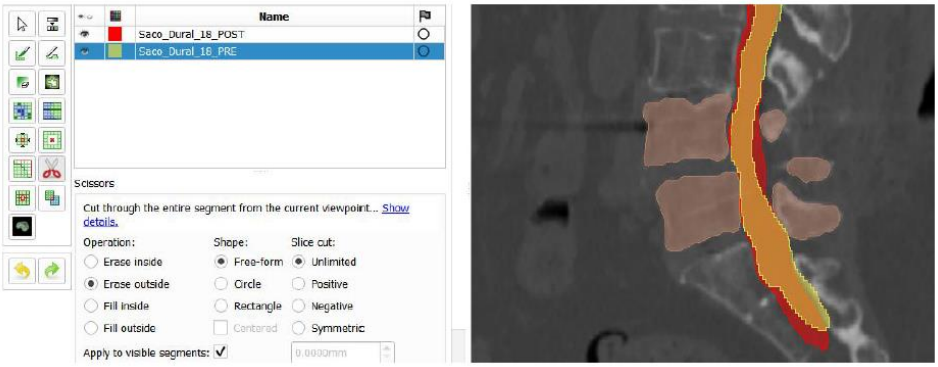


Figure 21: Snapshot of the 3D Slicer scene using the scissor crop modality.

Plane Crop

The Plane Crop process is a newly designed crop modality that allows the user to perform a crop using two planes instead of having to draw a region with a scissor tool. Although it is not fully automatic, it is a more practical way of performing the crop if dealing with an individual or small batch of patients.

Similar to the scissor crop, the function ***plane_crop_process()*** initiates a subprocess where a python script called PlaneCrop.py is executed through the 3D Slicer python interpreter. This automatically sets the scene with the two pre- and post-operative segments, one of the original NifTi images and one of the pairs of vertebras in the intervened level. This crop follows a more intuitive methodology than its predecessor, as the user must select two points for each plane and the upper and lower boundaries will be automatically generated in a more consistent manner.

For each patient and level, a slicer scene will automatically load showing the two registered spinal cord segments. Then a window will emerge indicating the user to place two dots which will draw a line for the superior plane. Then the user must click “OK” and do the same for the inferior plane. This planes remove all segments of the ROI that surpass these limits. Both planes will be perpendicular to these two dots, so they must be placed either on the sagittal view or coronal view. In the case of spinal cord the most convenient is the sagittal view.

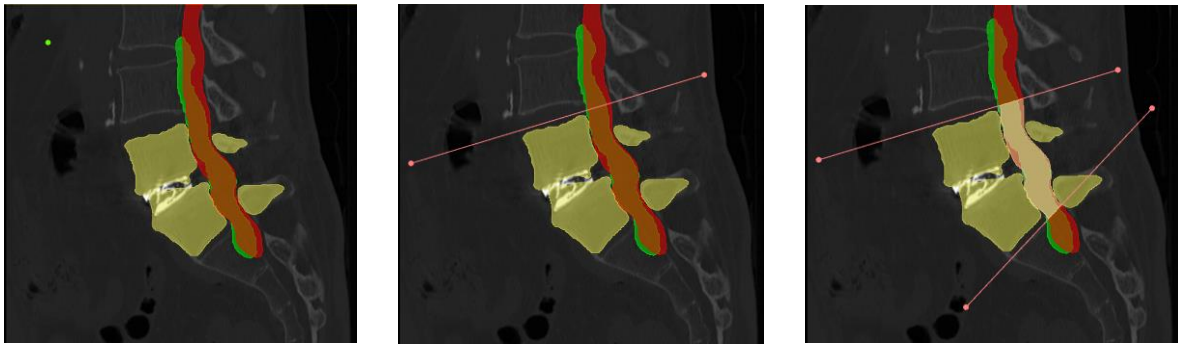


Figure 22: Representation of the plane crop from left to right. Pre-operative spinal cord in Green, post-operative in Red. Vertebras of intervened level showed for reference in yellow.

The same tool can be applied for the psoas muscle crop if the registration works effectively. For this anatomy the ideal view to perform the cut is the coronal view, as it allows the best visualization of the ROI.

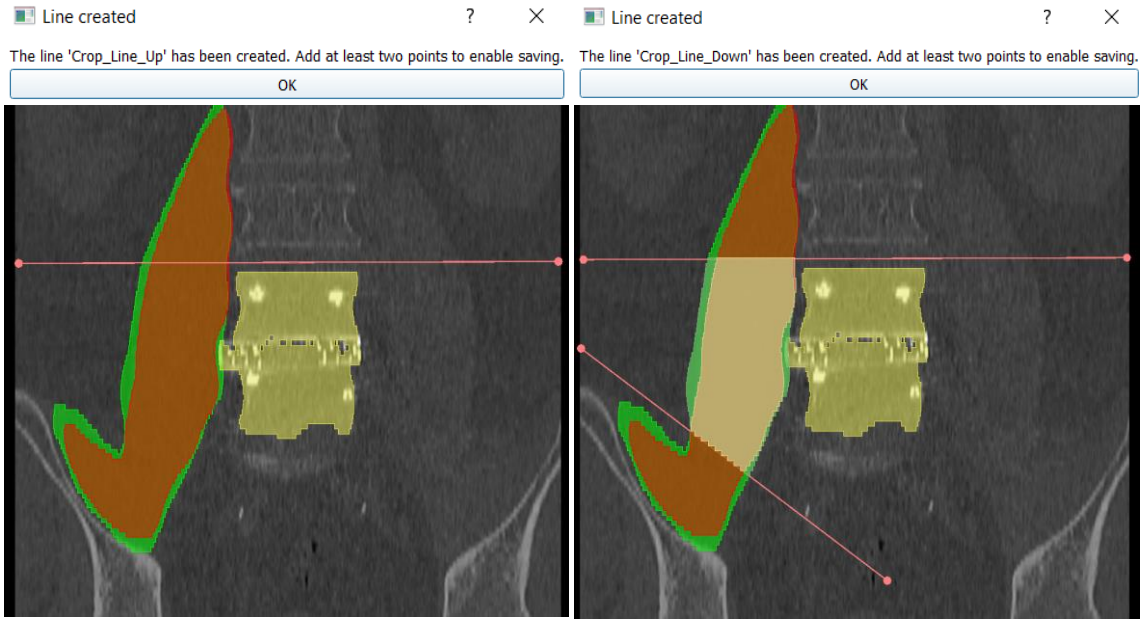


Figure 23: Snapshots of the 3D Slicer scene using a coronal view to perform the plane crop on the psoas muscle.

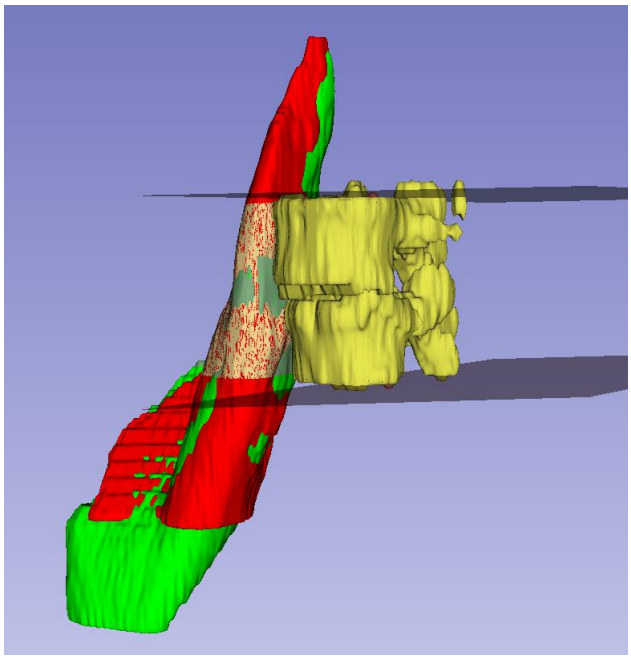


Figure 24: The dimensional view of the plane crop performed, using 3D Slicer.

4.3.5. Patient Volumetric Analysis

This step is responsible for extracting quantitative results from the segmented and cropped volumes for each patient. This phase translates the processed image data for each patient into interpretable results in the generated Results Excel file, which reflect the impact of surgical intervention. This is a crucial step as it is where the quantifiable measurements are obtained.

The patient volumetric analysis is performed by the function ***volume_value_from_patient()*** and it consists of calculating the total volume of the segmented structure before and after surgery. It computes the percentage of increase or decrease offering a straightforward quantification of the volume change, to detect surgical effects such as decompression (for dural sac) or atrophy (for psoas major). After all patients undergo the individual calculations, all the results are then saved in an Analysis Results Excel file. Basic statistical parameters (mean and standard deviation) are obtained of the processed cases that underwent the analysis and are saved on the same sheet. However depending on the selected ROI a specific analysis process is executed that prepares the data for the final statistical analysis that goes into more depth.

Spinal Cord Individual Morphological Analysis

After the basic volumetric analysis, if the selected ROI is the spinal cord, the function calls the function ***spinal_cord_shape_analysis()*** that performs the individual morphological analysis. The data of the patient specific analysis is saved in the patient folder together with the generated plots. These methodologies incorporated from the original tool can be seen in Figures X and Y. The full methodological details and validation of the methods applied are explained in [1].

The results of each individual morphological analysis are saved in the patient specific folders, which establish a consistent structure that allows the final comparative statistical analysis between surgeries.

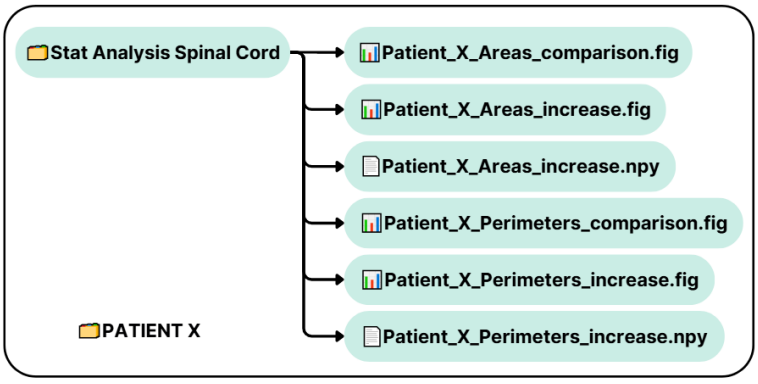


Figure 25: Visual representation of the result files saved in each patient folder.

Individual Psoas Patient Analysis

This analysis follows the same structure as the spinal cord analysis, as after the basic volumetric analysis is obtained for each patient and saved in the Analysis Results excel file, the function ***psoas_atrophy_analysis( )***identifies the intervened and non-intervened psoas and saves the results accordingly. This is possible because when loading the patient metadata the user can indicate which the intervened side is by introducing the case type as XLIFR (XLIF Right) or XLIFL (XLIF Left). Because this is an analysis that is tailored for the XLIF approach the code expects that if the psoas analysis wants to be performed the intervened side must be indicated in this manner.

NHC	PatientID	VisitType	CaseType	Levels	
123456XY	1	PRE/POST	**XLIFL**	L2-L3	→ **Intervened Psoas = left_Iliopsoas**
234567YZ	2	PRE/POST	**XLIFL**	L3-L4/L4-L5	→ **Intervened Psoas = left_Iliopsoas**
345678XZ	3	PRE/POST	**XLIFR**	L4-L5	→ **Intervened Psoas = right_Iliopsoas**

Figure 26: Example of how the tool automatically identifies the intervened psoas. The table represents the input batch Excel file.

After this the function can automatically save the volumetric analysis of each level and each side in separate files. This creates a consistent structure that allows the final statistical analysis involving multiple patients to be executed.

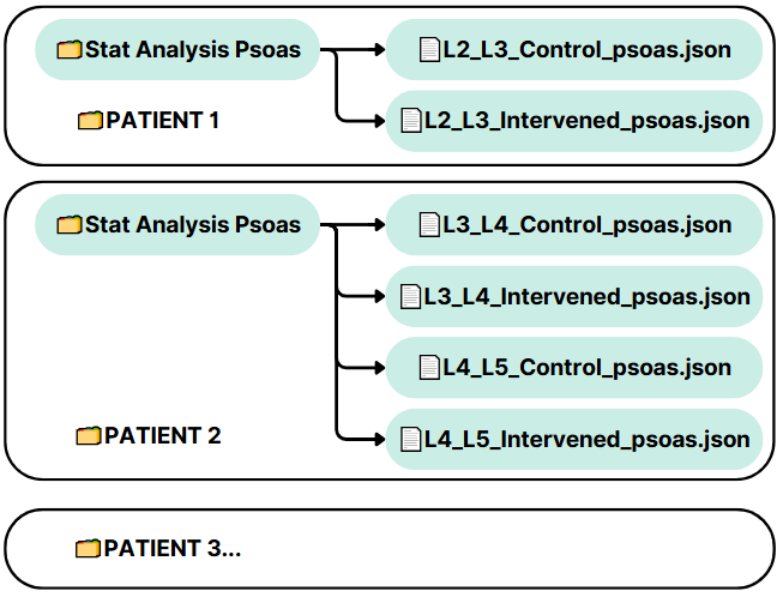


Figure 27: Visual representation of where the psoas results files are located within the patient folders.

4.3.6. Statistical Analysis

Comparative Surgery Spinal Cord Analysis

While the original tool was focused only on compering MIS vs OPEN cases, the updated pipeline allows for comparisons between any cases of interest. During the Setup Phase, if this type of analysis is selected, the user can decide which two found case types they want to compare. In the Evaluation of Results section, a variety of comparative analyses are performed to show the power of the modular tool, as it opens the doors to many possible comparisons between surgery approaches.

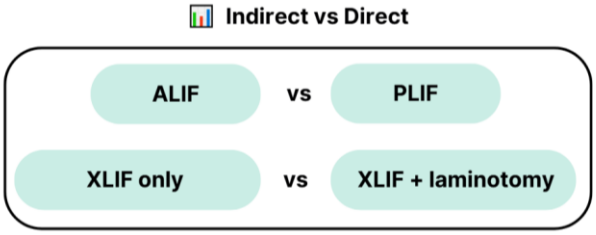


Figure 28: Visual representation of the possible indirect vs. direct comparative studies.

Psoas Atrophy Analysis

This analysis was specifically developed to obtain a comprehensive statistical comparison between the intervened and non-intervened psoas. The analysis produces both quantitative metrics which are saved in the excel results file and graphical figures. The metrics saved in the Excel for both Intervened and Control (non-intervened) groups are:

- **Mean:** average percent volume change between PRE and POST scans
- **Standard Deviation:** variability in percent volume change
- **Median:** central value of the distribution, robust to outliers
- **Interquartile Range (IQR):** spread of the middle 50% of values

Then for each patient the difference (Intervened-Control) is calculated and then the following metrics:

- **Mean Difference:** average relative change between paired levels
- **Std Dev of Difference:** variability of this relative change

And finally, the **Cliff's Delta** is calculated. This is a non-parametric measure that indicates how often a value from the control group is larger than the intervened group. It ranges from -1 to +1, with a negative value indicating that the control group tend to be larger and a positive indicating they tend to be smaller (since the code does Intervened - Control)

4.3.7. Step Completion Checking and Pipeline Flow

A crucial design choices that makes the pipeline efficient during Execution Phase includes a step validation checking mechanism using individual patient JSON files to track progress along the pipeline. Before starting any of the analysis steps for a patient, the pipeline checks whether that step has already been completed by using the information stored in the patient JSON file. If a step is already complete, the system skips it to avoid redundant computations. This features allows incomplete analyses to resume exactly where they were left off and supports batch processing.

The pipeline processes each step in order and within each step it iterates over all the patients selected. This step first approach allows for manual interventions (e.g., cropping) to be done all at once, as all the patients will have to undergo the step at the same time. After each step is successfully completed the patient JSON file is updated, ensuring the checkpoint reflects current progress.

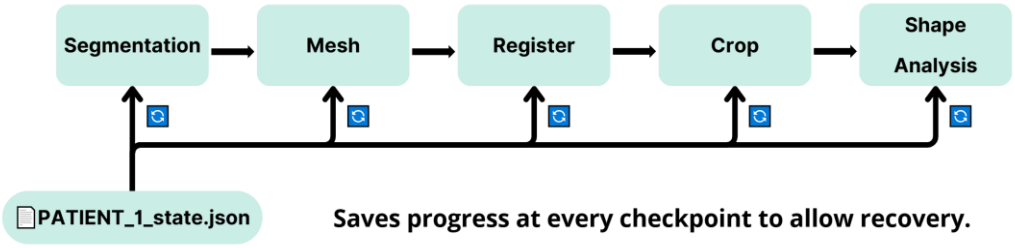


Figure 29: Visual representation of the JSON checkpoint mechanism.

So, the content of the Patient object is constantly being updated onto the JSON file to preserve progress, containing the metadata and directories of all patient specific files:

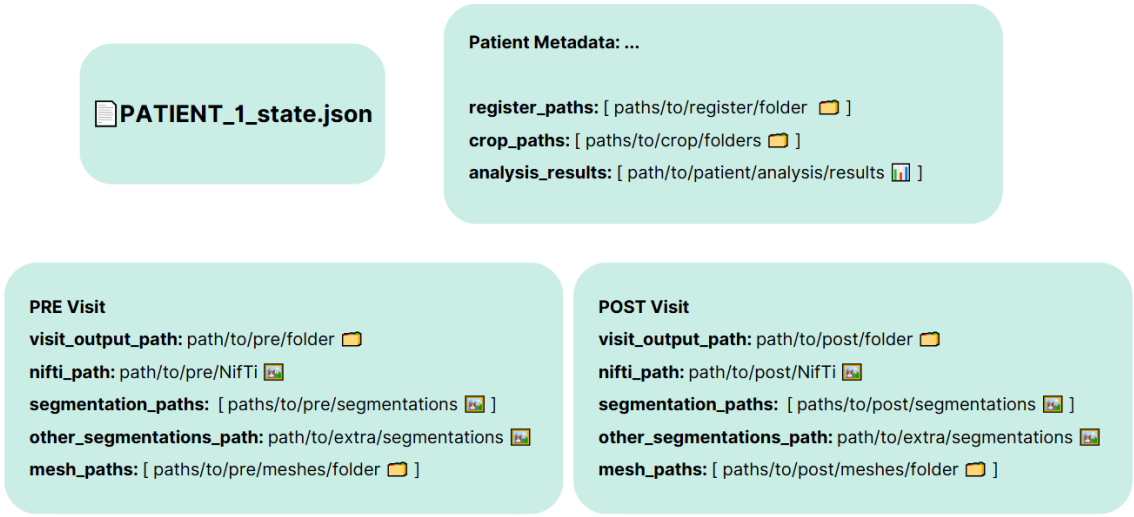


Figure 30: Example of JSON file contents.

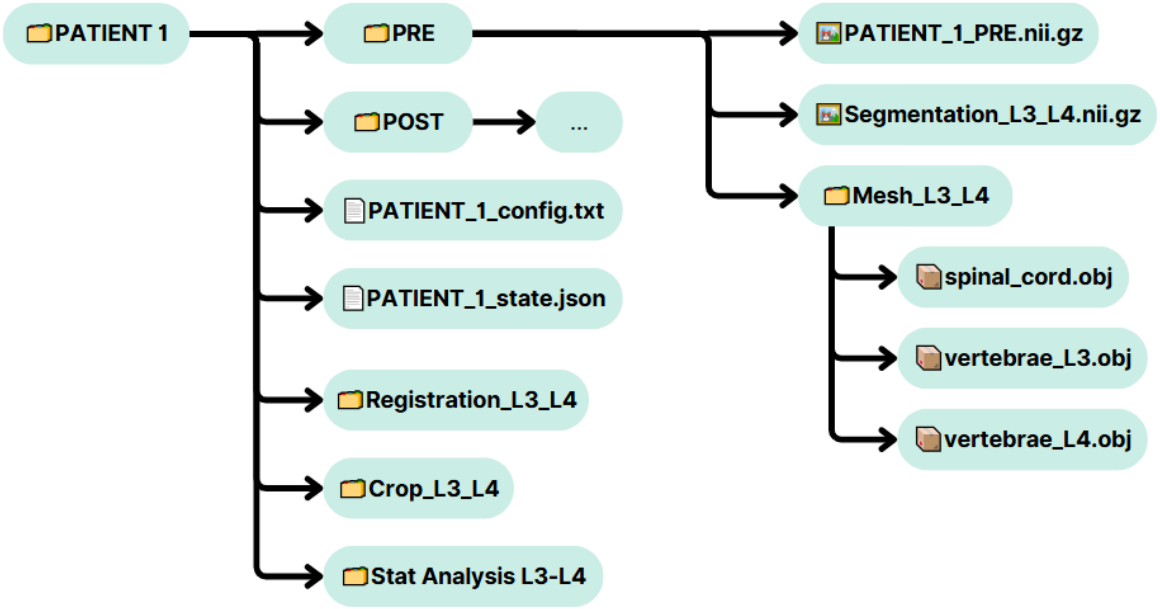


Figure 31: Display of the same patient's folder contents.

5. Code Structure

5.1 Patient and Visit Classes

The creation and use of objects have already been introduced during the Setup Phase and Execution Phase. However, a dedicated explanation of the reasoning behind this Object Oriented Programming (OOP) approach is valuable. The design and use of **Patient** and **VisitData** classes offer many practical and structural advantages, as it allows for the encapsulation of both data (attributes) and behaviour (methods) in the same element.

Some of the advantages of the Object-Based Structure are:

- **Localized Patient-Specific Storage**
  Instead of having to store patient metadata (e.g., patient ID, vertebral levels, etc.) and paths (e.g., path to segmentation) across separate variables or files, all relevant information is encapsulated within a single Patient object.
- **Efficient Multi-Patient Handling**
  By having each case as a distinct object batch processing becomes straightforward. A list of Patient objects can be iterated through, systematically executing analysis steps for each one independently.
- **Consistent and Predictable Data Structure**
  Defining a Patient class along with its visit objects, provides a predetermined structure to managing input data, outputs and eventually final results. Even though the physical data is saved in nested patient folders, the processing logic does not need to reference these directories directly. Instead by accessing the object, all necessary paths and metadata are centrally accessible.
- **Traceability and Reusability via JSON Checkpoints**
  The pipeline incorporates a checkpointing mechanism by copying each Patient object in the form of a JSON file. This acts as an up to date snapshot of the objects state, enabling reuse of previously analysed cases.

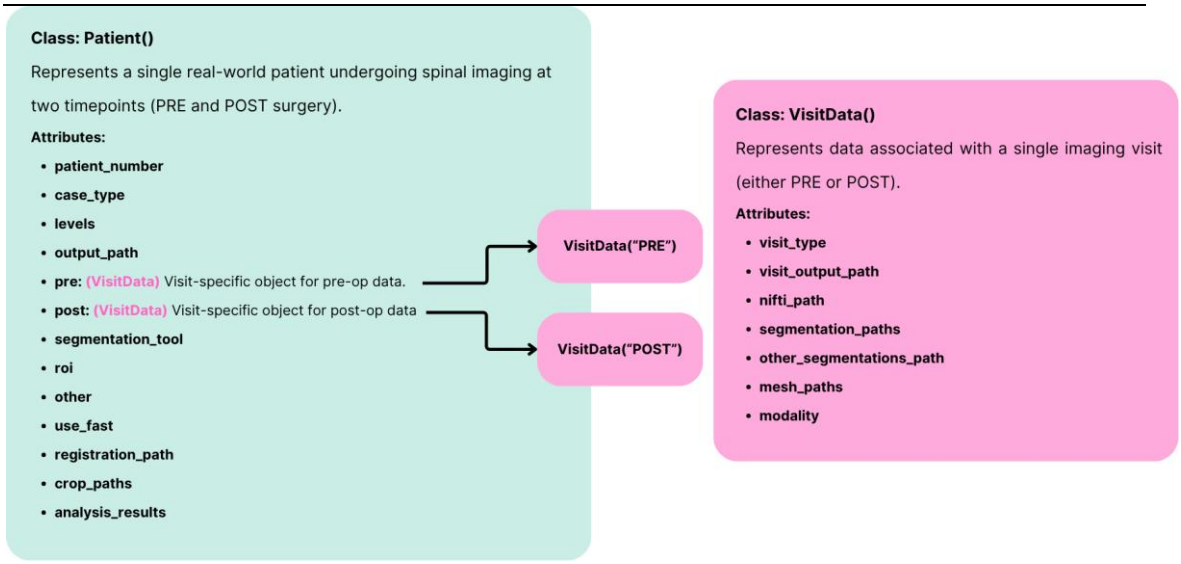


Figure 32: Logical representation the Class structures and all its attributes. The Patient object class contains two VisitData instances.

The project uses a two-level object architecture:

- The **Patient class** stores global metadata, configuration options, and analysis results, and it also holds two embedded VisitData objects, one for PRE-operative and one for the POST-operative visit.
- The **VisitData class** manages all visit-specifc data, such as the selected NifTi file and its segmentations and its generated meshes.

This distinction is necessary because certain characteristics are shared by both visits while some are independent to the specific pre- or post-operative visit. For instance, most of the patient Metadata is shared by both visits, and after the analysis the Registerd and Cropped segments combine both visits meaning it's also patient-specific; meaning all of these would be stored in the higher-level **Patient** object. Whereas the segmentation and meshing outputs are unique for the PRE and POST visits, being stored in the **VisitData subclass**.

It's worth noting that although **VisitData** is referred to as a subclass for organizational purposes, in reality it is an independent class by itself. However, it is directly associated with the Patient class: a **VisitData** object always exists within a **Patient** object, meaning a Visit cannot be processed on its own and is always associated with a Patient.

For a detailed breakdown of all functions and attributes in each class go to **Annex 1: Patient and Visit Class Attributes and Methods.**

5.2 Modular Blocks and Functional Design

A crucial step in creating the modular pipeline was dividing all pre-existing and new functions into blocks or **modules**. This was done to avoid vertical structured scripts with many dependencies between functions, allowing the pipeline to become modular and scalable. The following modules where created:

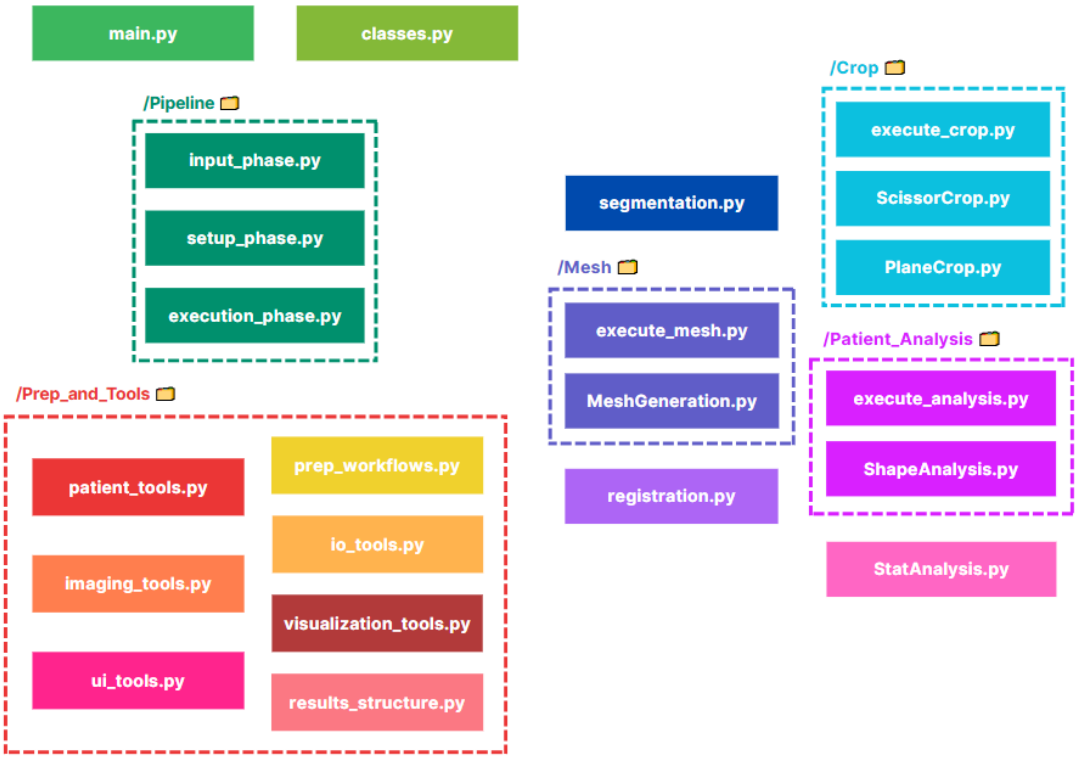


Figure 33: Generic overview of all the modular blocks that form the code.

All functions are separated in the following modules as scripts. Some of them are simply used as a way of defining functions in an ordered way, while other scripts contain higher level orchestrator functions that call other tool-like functions to complete a given process. The whole code is separated in:

- **5 core control modules**: *main.py, classes.py, input_phase.py, steup_phase.py and execution_phase.py.*
- **6 functional modules** (for each execution step): *segmentation.py, execute_mesh.py, registration.py, execute_crop.py, execute_analysis.py and execute_StatAnalysis.py.*
- **7 tool and helper packages:** *prep_workflows.py, patient_tools.py, imaging_tools.py, visualization_tools.py, ui_tools.py and results_structure.py.*

5.2.1. Core Control Modules

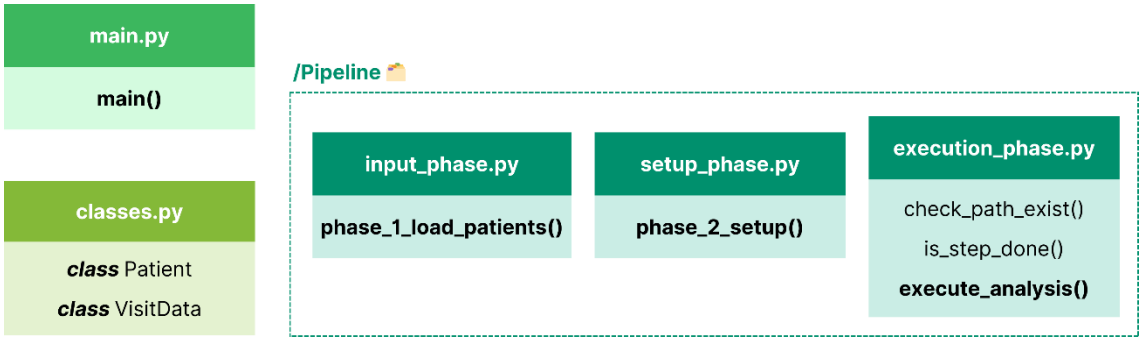


Figure 34: Visual representation of all the core control modules.

These five scripts form the Core Control Modules, responsible for orchestrating all the other blocks in the pipeline. The ***main.py*** script is the entry point and navigation interface, while the ***classes.py*** script defines the Patient and VisitData object structures**.** Then inside the **Pipeline** folder ***input_phase.py***, ***setup_phase.py***, and ***execution_phase.py*** organize the three phases of processing. These functions serve as the tool's backbone for correct functioning.

An important concept used throughout the whole pipeline was the use of high-level orchestrator functions to coordinate many individual modular functions to complete complex processes. This design choice focuses all the dependencies within a few functions but still allows the code to be modular and scalable, as additional functions can be implemented without affecting the general pipeline. This also allows many of the functions to be re-used when necessary, as a same function can serve at different points of the pipeline.

The Core Control Modules scripts define their own high-level orchestrator functions and import most of the helper functions from the Functional and Tools modules. This is because these modules control the pipeline and it is important to keep a clean pipeline structure for correct functioning.

For a code structure overview including functions defined and flowcharts to understand the workflow of the function go to **Annex 2: Code Structure of Core Control Modules.**

5.2.2. Functional Modules

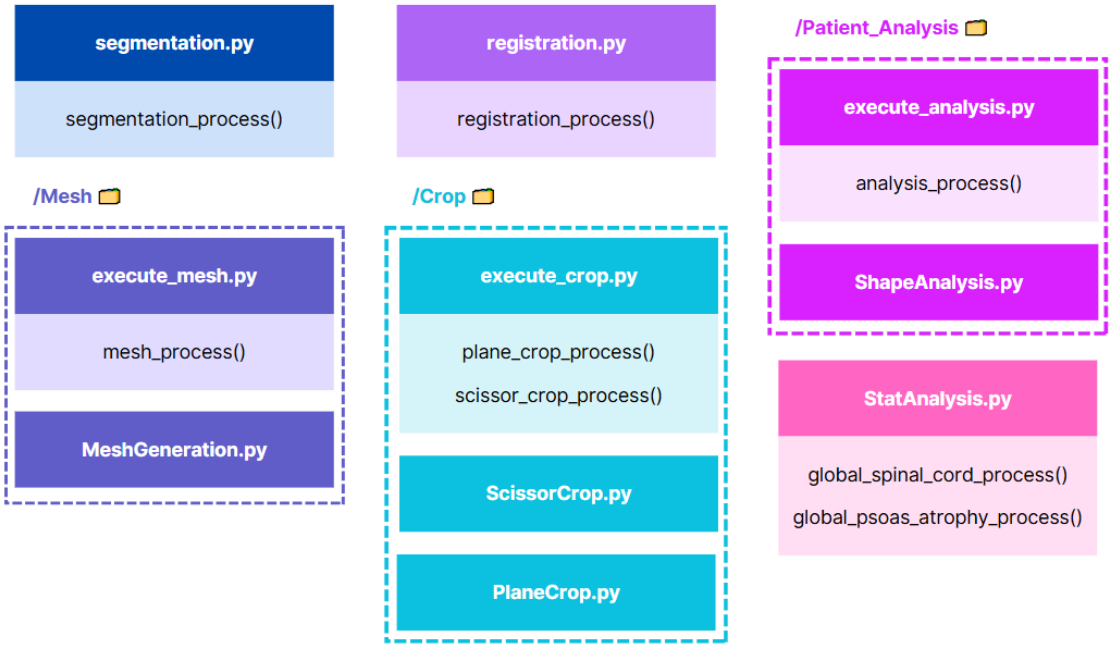


Figure 35: Visual representation of all the functional modules.

These are the six functional modules that form the execution phase. The details of what each stage does have already been explained, however the functions that carry out the operations are defined in this section. These blocks have been designed to admit the inclusion of new processes that allow the tool to be adapted for different purposes. This is demonstrated by having different process for psoas and spinal cord or as different crop modalities. The individual modules also make them independent from each other, as one step can be executed individually or all at once to complete the full analysis. This is possible thanks to the Patient object storing all information, which makes it the only necessary input for most of the functions , eliminating the need for passing information between steps. For these reasons all of these functions do not return anything in the pipeline, but rather physically store files within the patient folder.

Segmentation Module

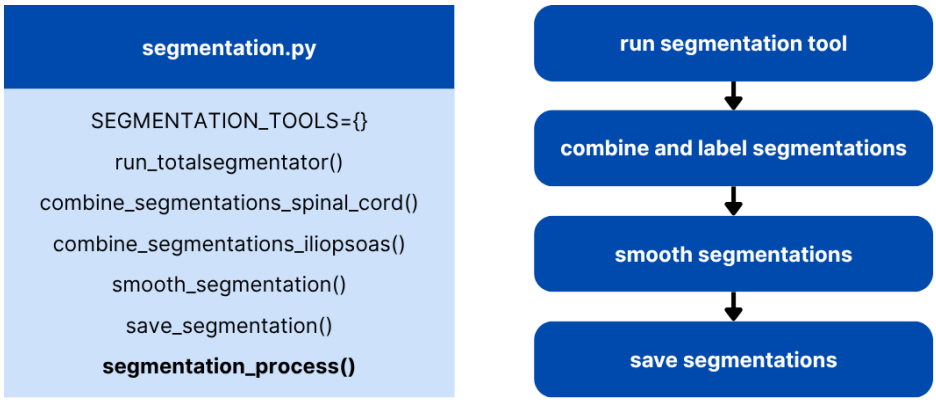


Figure 36: Visual representation of the segmentation module.

This is the first functional block of the analysis that carries out the segmentation of the desired anatomical structures. The whole process is coordinated by the high-level orchestration function ***segmentation_process()***. It currently supports two main targets:

- **Spinal Cord + Vertebrae**
- **Iliopsoas Muscles + Vertebrae**

The available segmentation models and their ROI options are stored in the ***SEGMENTATION_TOOLS = {}*** dictionary. As of now only **TotalSegmentator** is configured, with CT and MR configurations.

The incorporation of both targets shows how adding a variety or ROIs is possible and the pipeline is designed to allow this. The same applies for the segmentation tool.

Mesh Generation Module

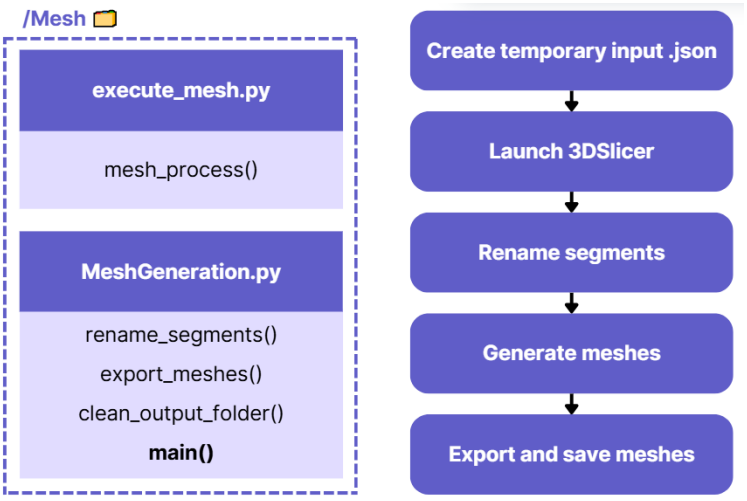


Figure 37: Visual representation of the mesh generation module.

This module is composed of two scripts ***execute_mesh.py*** and ***MeshGeneration.py***. The first is in charge of triggering the mesh generation externally by launching 3D Slicer and running a Python script (***MeshGeneration.py***) using ***subprocess***. A temporary ***.json*** file with patient specific parameters is generated and passed into the suprocess as input for the following script.

The ***MeshGeneration.py*** script processes a NifTi segmentation file and exports the anatomical meshes (.obj files). It uses Label dictionaries depending on the selected ROI:

- ***SPINAL_CORD_LABELS = {},*** for spinal cord labelling
- ***PSOAS_LABELS = {},*** for psoas labelling

Registration Module

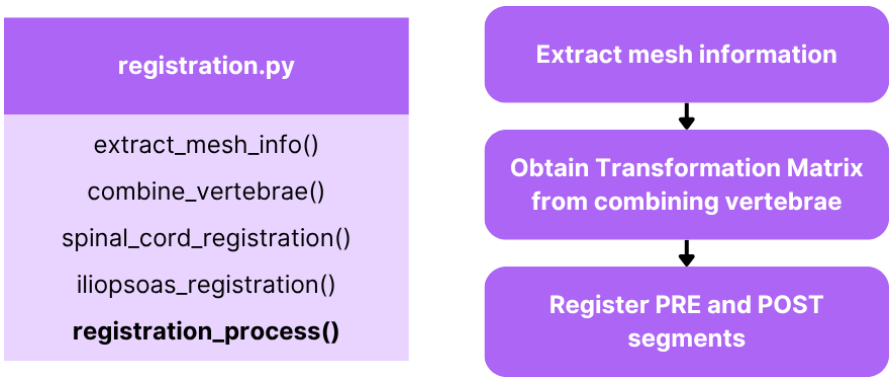


Figure 38: Visual representation of the registration module.

This block performs the registration process described in the **4.3.3. *Registration*** section. Once again, the only input expected is the Patient object, as from there all the mesh files are accessible.

Crop Module

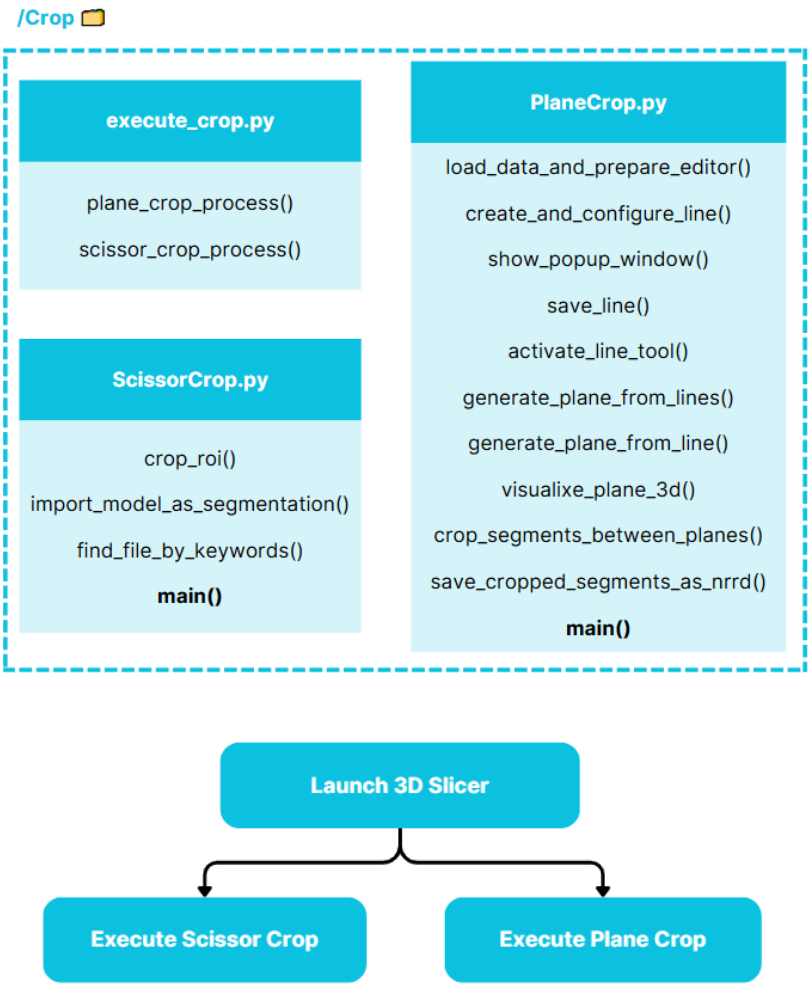


Figure 39: Visual representation of the crop module.

This module has three scripts, one being the controller that launches the 3D slicer environment and then executes a crop script depending in the modality. Then the ***ScissorCrop.py*** or ***PlaneCrop.py*** allow the user to perform the crop by automatically showing the crop segments and instructions of use. This module can easily incorporate new crop modalities by simply adding an additional crop script or high-level orchestrator function.

Patient Volumetric Analysis Module

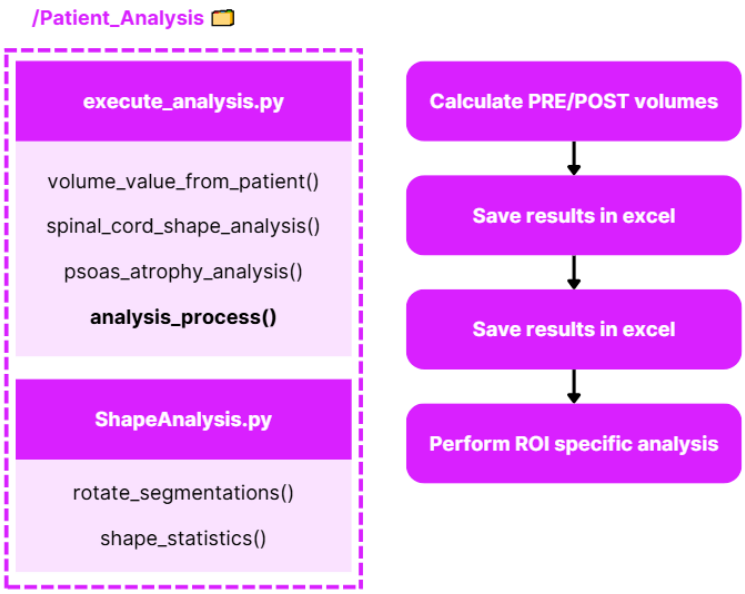


Figure 40: Visual representation of the patient volumetric analysis module.

This block executes the quantitative analysis after registration and cropping. The whole patient analysis process is controlled by the high-level orchestrator function **analysis_process()**. For all ROIs it performs the volumetric analysis (function ***volume_value_from_patient()***) and saves results in the results excel file. Then depending on the selected ROI it initiates the ***psoas_atrophy_analysis()*** or does the individual ***spinal_cord_shpe_analysis()***. The ***ShapeAnalysis.***py script contains specific functions used for the morphological analysis of the spinal cord.

Stat Analysis Module

Figure 41: Visual representation of the stat analysis module.

This final block is the first that is not executed per patient or visit, as it is a group-level analysis that does a statistical comparison of many patients at once. The ***global_spinal_cord_process()*** is the high level orchestrator that performs the comparative morphological analysis of the two selected groups. The ***global_psoas_atrophy_process()*** has the same responsibility for the intervened vs control analysis of the iliopsoas muscle.

4.2.3. Prep_and_Tools Modules

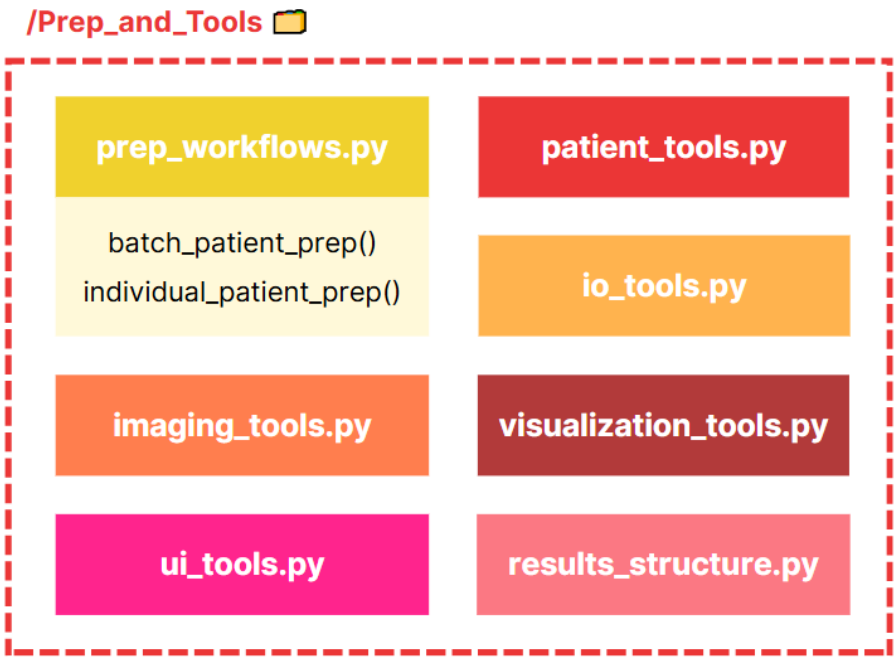


Figure 42: Visual representation of the all the prep and tools modules.

The following table shows an overview of the modules inside the Prep_and_Tools folder. Apart from the two high-level orchestrator functions defined in the ***prep_workflows.py*** script all the other modules contain helper functions and tools used throughout the whole pipeline. For a more detailed view each function is explained in **Annex 2: Prep_and_Tools Functions.**

Module	Focus	Main Roles
prep_workflows.py	Input Phase Orchestration	Converts raw data to usable config + NIfTI
patient_tools.py	File Logic	Organizes folders, finds patients, handles cleanup
io_tools.py	I/O Helpers	Load/copy files, print folder contents
imaging_tools.py	Image Metadata	Convert & list DICOM/NIfTI, auto-selection
visualization_tools.py	Previews	Display NIfTI or DICOM for selection
ui_tools.py	CLI Interaction	Ask users for choices, paths, configs
results_structure.py	Results Saving	Create folders, export Excel stats, save figures

Table 3: Overview of the scripts included in the Prep_and_Tools folder.

5. Results and Evaluation

5.1. Results

In the results section, the main results obtained with the tool are shown: volumetric change of the spinal cord after the ALIF approach (Table 4), after the LLIF approach (Table 5) and the volumetric change of the psoas major muscle after LLIF approach (Table 6)

5.1.1. ALIF Approach – Volumetric Change of the Spinal Cord

CASE TYPE	PATIENT ID	LEVEL	PRE [cm³]	POST [cm³]	INCREMENT [%]
ALIF	1	L5-S1	16.32	16.40	0.49
ALIF	12	L5-S1	7.93	10.68	34.72
ALIF	13	L4-L5	12.96	12.84	-0.91
ALIF	13	L5-S1	7.94	8.68	9.37
ALIF	14	L5-S1	18.58	20.78	11.82
ALIF	15	L4-L5	12.90	13.65	5.84
ALIF	16	L5-S1	5.11	5.55	8.57
ALIF	17	L4-L5	17.31	15.21	-12.12
ALIF	18	L5-S1	9.50	10.27	8.18
ALIF	19	L4-L5	20.49	17.49	-14.67
ALIF	2	L5-S1	16.82	18.23	8.42
ALIF	3	L4-L5	12.32	16.10	30.65
ALIF	4	L4-L5	7.67	10.50	36.80
ALIF	5	L5-S1	9.06	9.76	7.65
ALIF	6	L5-S1	7.23	6.07	-16.08
ALIF	7	L5-S1	12.36	14.05	13.72
ALIF	8	L5-S1	12.05	11.91	-1.14
ALIF	9	L5-S1	13.27	15.01	13.17

Table 4: Results of the Volumetric Change of the Spinal Cord after the ALIF Approach.

5.1.2. LLIF Approach – Volumetric Change of the Spinal Cord

CASE TYPE	PATIENT ID	LEVEL	PRE [cm³]	POST [cm³]	INCREMENT [%]
LLIF DIRECT	21	L3-L4	7.00	11.01	57.59
LLIF DIRECT	28	L3-L4	6.82	7.36	7.98
LLIF DIRECT	36	L2-L3	10.38	14.13	36.19
LLIF INDIRECT	22	L4-L5	19.30	19.57	1.43
LLIF INDIRECT	23	L3-L4	6.87	9.90	44.19
LLIF INDIRECT	23	L4-L5	8.14	12.08	47.10
LLIF INDIRECT	24	L3-L4	7.52	8.69	15.54
LLIF INDIRECT	25	L3-L4	6.01	7.98	32.70
LLIF INDIRECT	26	L2-L3	9.07	9.80	8.02
LLIF INDIRECT	32	L3-L4	8.97	9.21	2.66
LLIF INDIRECT	32	L4-L5	8.65	9.31	7.61
LLIF INDIRECT	33	L3-L4	8.03	9.02	12.24
LLIF INDIRECT	33	L4-L5	8.49	9.65	13.69
LLIF INDIRECT	34	L3-L4	7.97	11.51	44.37
LLIF INDIRECT	35	L1-L2	9.09	11.92	31.05
LLIF INDIRECT	35	L2-L3	8.09	13.85	71.20
LLIF INDIRECT	35	L3-L4	8.87	15.68	76.68
LLIF INDIRECT	35	L4-L5	7.92	11.88	50.01
LLIF INDIRECT	37	L3-L4	9.72	11.57	19.07

Table 5: Results of Volumetric Change of the Spinal Cord after the LLIF Approach.

5.1.3. LLIF Approach – Volumetric Change of the Psoas Major Muscle

CASE TYPE	PATIENT ID	SIDE	PRE [cm³]	POST [cm³]	INCREMENT [%]
LLIFR	21	Left-Iliopsoas_Left_psoas	46.96	63.26	34.70
LLIFR	21	Right-Iliopsoas_Right_psoas	31.54	44.73	41.84
LLIFL	22	Left-Iliopsoas_Left_psoas	77.83	71.41	-8.24
LLIFL	22	Right-Iliopsoas_Right_psoas	116.93	110.61	-5.41
LLIFL	23	Left-Iliopsoas_Left_psoas	0.00	3.70	nan
LLIFL	23	Right-Iliopsoas_Right_psoas	0.00	3.40	nan
LLIFL	24	Left-Iliopsoas_Left_psoas	50.64	66.32	30.97
LLIFL	24	Right-Iliopsoas_Right_psoas	45.94	47.39	3.15
LLIFL	25	Left-Iliopsoas_Left_psoas	30.16	28.95	-4.02
LLIFL	25	Right-Iliopsoas_Right_psoas	48.15	48.81	1.37
LLIFR	26	Left-Iliopsoas_Left_psoas	48.43	40.58	-16.22
LLIFR	26	Right-Iliopsoas_Right_psoas	36.99	29.92	-19.13
LLIFL	28	Left-Iliopsoas_Left_psoas	37.57	52.29	39.18
LLIFL	28	Right-Iliopsoas_Right_psoas	41.77	53.44	27.94
LLIFL	32	Left-Iliopsoas_Left_psoas	34.70	47.46	36.77
LLIFL	32	Right-Iliopsoas_Right_psoas	44.28	64.35	45.32
LLIFL	33	Left-Iliopsoas_Left_psoas	56.58	71.60	26.54
LLIFL	33	Right-Iliopsoas_Right_psoas	58.56	77.90	33.02
LLIFR	34	Left-Iliopsoas_Left_psoas	38.01	33.70	-11.35
LLIFR	34	Right-Iliopsoas_Right_psoas	51.68	39.71	-23.17
LLIFR	35	Left-Iliopsoas_Left_psoas	57.95	56.67	-2.22
LLIFR	35	Right-Iliopsoas_Right_psoas	80.65	73.96	-8.30
LLIFL	36	Left-Iliopsoas_Left_psoas	83.01	87.16	5.00
LLIFL	36	Right-Iliopsoas_Right_psoas	116.73	121.44	4.04
LLIFL	37	Left-Iliopsoas_Left_psoas	53.26	41.96	-21.23
LLIFL	37	Right-Iliopsoas_Right_psoas	70.80	66.02	-6.75

Table 6: Results of Volumetric Change of the Psoas Major Muscle after the LLIF Approach.

5.2. Evaluation of Results

5.2.1. ALIF Approach – Indirect Decompression of the Spinal Cord

Mean Increment (POST-PRE) [%]	Standard Deviation [%]
8.03	**15.06**

Table 7: Generic results of the Indirect Decompression of the Spinal Cord (ALIF Approach)

The ALIF approach showed a mean increment in spinal cord volume of 8.03% with a standard deviation of 15.06%. This initially indicates that **spinal cord volume increases after ALIF surgery** demonstrating that Indirect Decompression does occur.

t-statistic	p-value
2.26	**0.037**

Table 8: One sample t-test

Both the t-statistic and p-value indicate that the null hypothesis is rejected (p-value < 0.05), and that **the mean increment is significantly different than 0 with 95% confidence**.

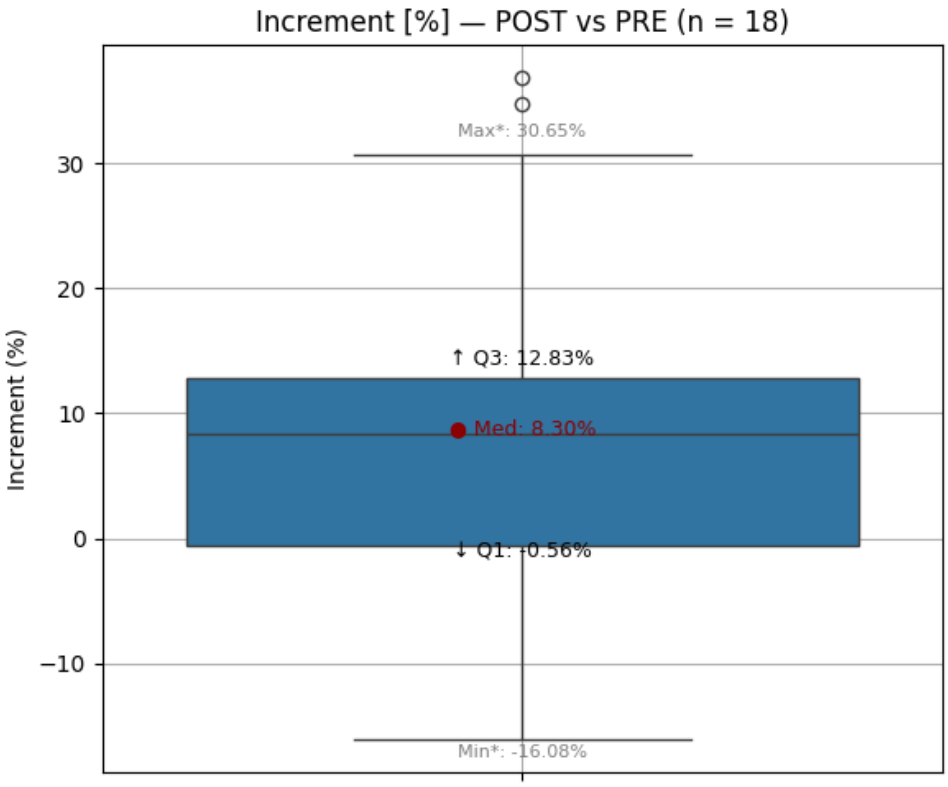


Figure 43: Boxplot showing the distribution of results of the ALIF Approach.

Median [%]	Q1 [%]	Q3 [%]	Min [%]	Max [%]
8.30	-0.56	12.83	-16.08	30.65

Table 9: Numerical results of the distribution of the ALIF approach.

The median is 8.30 % increment with Q1 = - 0.56 % and Q3 = 12.83%. Looking at the distribution of data most of the results appear to be within the positive range, indicating once again that **most cases experience spinal cord decompression** after the ALIF approach.

5.2.2. LLIF Approach – Indirect and Direct Decompression of the Spinal Cord

Case Type	Mean Increment (POST-PRE) [%]	Standard Deviation [%]
LLIF DIRECT	33.92	24.88
LLIF INDIRECT	29.85	23.73

Table 10: Generic results of the Direct and Indirect Decompression of the Spinal Cord (LLIF Approach)

Both LLIF approaches present similar increments in volume. The LLIF + Direct Decompression group showed a mean increment of 33.92% with a standard deviation of 24.88% and the Indirect Decompression group a mean increment of 29.85% with a standard deviation of 23.73%. This initially indicates that **spinal cord volume increases significantly after LLIF surgery** with no noticeable differences between the Direct and Indirect decompression groups.

t-statistic (Indirect)	p-value (Indirect)
5.03	0.00015

Table 11: One sample t-test of indirect decompression (LLIF)

The t-statistic and p-value can only be calculated for the indirect decompression group as the sample size for the direct group is not sufficient. Because p-value < 0.001 the null-hypothesis can be confidently rejected, **statistically proving strong evidence for an increase in spinal cord volume after LLIF with indirect decompression**.

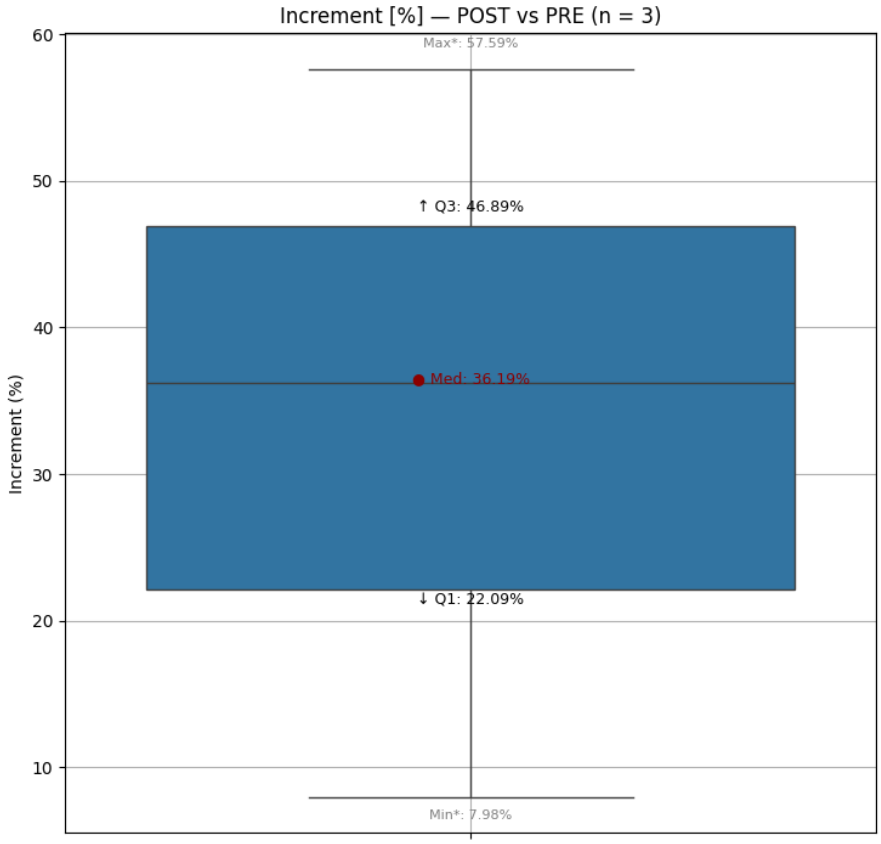


Figure 44: Boxplot showing the distribution of results of the LLIF Approach with direct decompression.

Median [%]	Q1 [%]	Q3 [%]	Min [%]	Max [%]
36.19	**22.09**	**46.89**	**7.98**	**57.59**

Table 12: Numerical results of the distribution of the LLIF Approach with direct decompression.

Looking at the distribution of the direct group, the median shows 8.30 % increment with Q1 = - 0.56 % and Q3 = 12.83%, indicating once again that **most cases experience significant spinal cord decompression.** However due to the small sample size this does not represent a comprehensive understanding of the direct group data distribution.

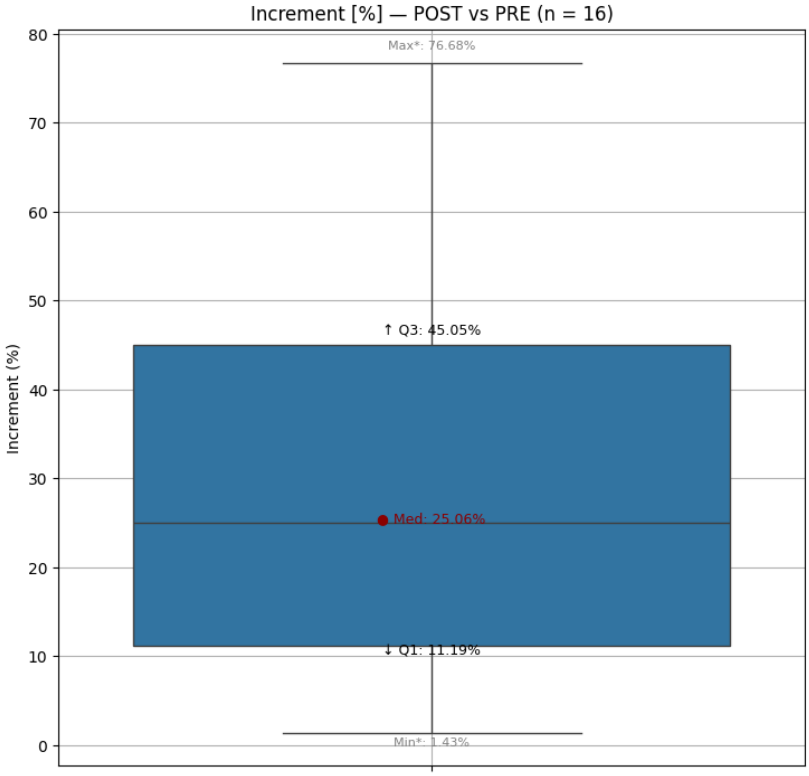


Figure 45: Boxplot showing the distribution of results of the LLIF Approach with indirect decompression.

Median [%]	Q1 [%]	Q3 [%]	Min [%]	Max [%]
25.06	**11.19**	**45.05**	**1.43**	**76.68**

Table 13: Numerical results of the distribution of the LLIF Approach with indirectdirect decompression.

On the other hand the distribution of Increments of the indirect decompression group once again proves a significant volume increase of the spinal cord after LLIF surgery. The median is 25.06%, the Q1 = 11.19% and Q2 = 45.05% showing how **most cases show positive increments in volume**.

5.2.3. Comparative Surgery Morphological Analysis of the Spinal Cord

This is one of the statistical analysis modules that has been incorporated with the pipeline that shows a comparative figure of the mean increments of cross-sectional area and perimeter through the height of the spinal cord section of two surgery types.

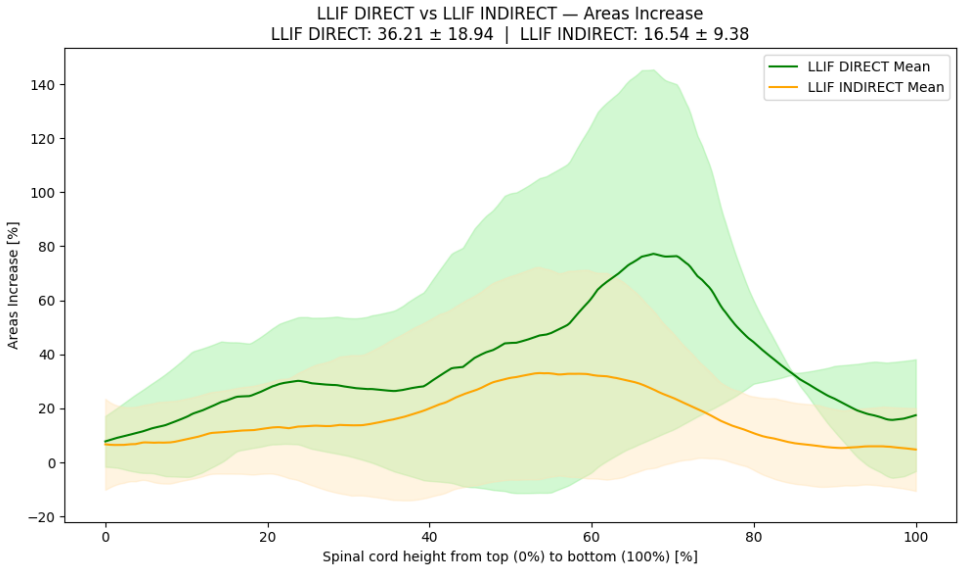


Figure 46: LLIF Direct vs. Indirect Decompression Sectional Area Comparison.

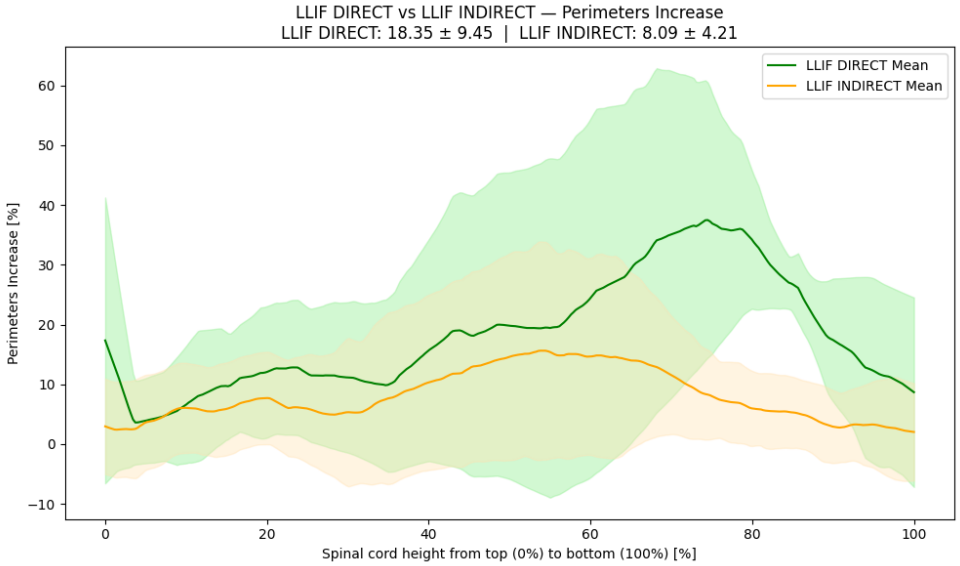


Figure 47: LLIF Direct vs. Indirect Decompression Perimeter Comparison.

Once again both groups show significant increase with the direct group slightly higher than the indirect group. However because of the low sample size the results of the direct group should not be used to take conclusions.

However, analysing the indirect group there is an evident maximum point of area and perimeter increase around 50% of the spinal cord height. These results could be an indicator that the indirect decompression is effectively coming from the intersomatic cages being installed between the vertebras as they are located approximately in the middle of the cropped section.

5.2.4. LLIF Approach – Iliopsoas Muscle Atrophy Analysis

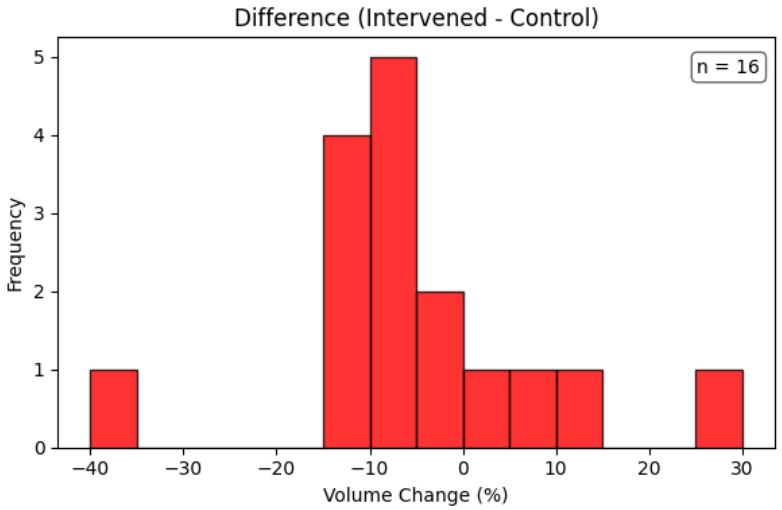


Figure 48: Histogram of the LLIF psoas muscle atrophy results.

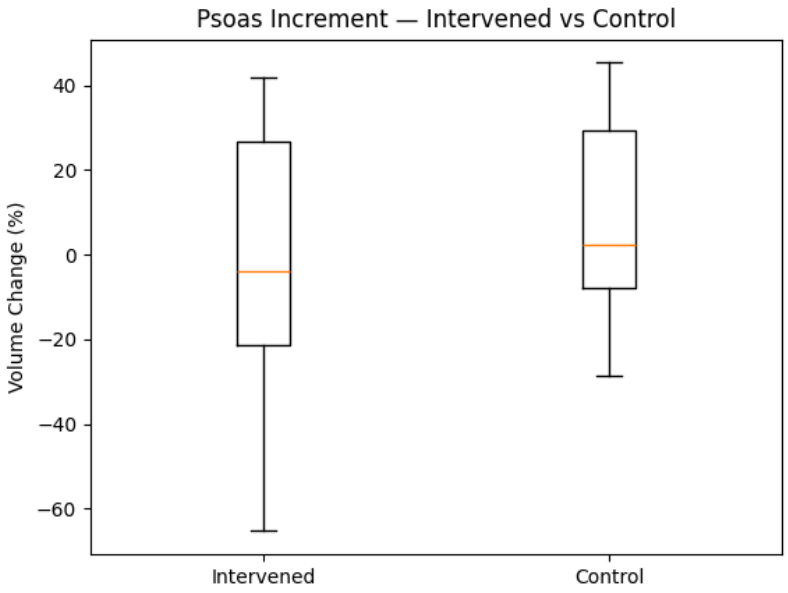


Figure 49: Boxplots comparing psoas increment of intervened vs. control samples.

Group	N	Mean	Std Dev	Median	IQR
Intervened	17	**0.08**	29.48	**-4.02**	47.77
Control	16	**7.42**	21.84	**2.26**	37.09

Table 14: Results of the generic distribution.

The results of the psoas atrophy analysis show some initial trends between the intervened and control muscle groups. The intervened group, consisting of 17 levels, presented an average volume change close to zero (mean = 0.08%) and a negative median (–4.02%). In contrast, the control group (16 levels) showed a positive mean (7.42%) and a positive median (2.26%). However, both groups presented high variability, showing standard deviations (Intervened = 29.48%; Control = 21.84%) and high IQR values (Intervened = 47.77%; Control = 37.09%). **These results show that there is no significant volume change in either group.**

Mean Diff (I - C)	Std Dev Diff	Cliff's Delta
-5.15	**13.29**	**-0.15**

Table 15: Intervened vs. Controlled extracted parameters.

The mean difference between groups (Intervened – Control) was –5.15% with a standard deviation of 13.29% and the non-parametric Cliff's Delta was –0.15. These results **indicate a small but systematic tendency for the control group to maintain higher volume increases than the intervened group.**

Side	t-statistic	p-value
Intervened	1.10	0.296
Control	1.51	0.160

Table 16: One sample t-test for Intervened and Controlled psoas.

However, results also show that **statistically neither groups shows a significant change from 0%** (p-value > 0.05). Due to the high variability the change noticed could be due to chance.

t-statistic	p-value
-0.10	**0.921**

Table 17: Two sample t-test (Intervened vs Control)

The two sample t-test between the control and intervened group present a numerical value of whether the means of the two groups are significantly different. In this case **the difference between groups is not significant**.

5.3. Comprehensive Overview of Results

5.3.1. Indirect Decompression of the Dural Sac after LLIF and ALIF approaches

After evaluating the results obtained for spinal cord decompression, they do seem to present a strong sentiment that spinal cord **indirect decompression does effectively occur in both ALIF and LLIF approaches** without the need of an additional laminectomy or laminotomy. Both surgery approaches accounted for an adequate number of samples that showed consistent results that back this statement.

Although ALIF presented a lower 8.03% mean increment in spinal cord volume (compared to LLIF: 29.85% mean increment in spinal cord volume) both the distribution of the data and one sample t-test coincide that most values present significant increments larger than 0 with high confidence level.

The same can be said with even more confidence for the group that underwent LLIF surgery with no additional direct decompression. The t-test scores show **that increase in spinal cord volume is extremely probable** and that getting a negative increment is very unlikely. The morphological analysis of this group also presents valuable information, as it's noticeable that the maximum point of area and perimeter increase is in the middle of the spinal cord segment. This aligns with the hypothesis that the decompression is coming from the insertion of the intersomatic cage, by restoring discal height and proper alignment, indirectly relieving pressure off of the spinal cord.

5.3.2. Indirect vs Direct Decompression of the Dural Sac after LLIF approach

Unfortunately, a proper comparison was not established due to the low sample size of the LLIF interventions with an additional direct decompression surgery. One of the main reasons for this is image quality, as of the 25 cases I was granted only 19 where accessible. The other 6 cases (5 direct and 1 indirect) either had corrupt or missing DICOM files. This is a common issue in the realm of medical image processing as these are heavy files that can be easily mismanaged and loss of information can happen.

However, the results obtained can be used as a preliminary idea of what to expect from such a study. Mean spinal cord volume increment in LLIF with direct decompression was of 33.92% while the indirect decompression group showed 29.85% increment, implying that **there seems to be no major difference between the two groups.** This type of results is also present in other studies [6] where it is concluded that when the approach is properly selected, indirect decompression through LLIF can be just as effective.

5.3.3. Analysis of Psoas Atrophy after LLIF Surgery

While most of the extracted results seem to show a trend towards some atrophy in the intervened group such a statement cannot be made with confidence. Instead, this study shows that **there are no significant changes in volume increment between intervened and non-intervened iliopsoas muscles after LLIF intervention**.

This statement is more accurate because apart from the means and distributions of both groups being close to a 0% increment, the high variability in both standard deviation and IQR meant that the small changes observed do not represent an accurate measure. This is also backed by the t-test as both their p-values indicate that neither group show significant change from 0%. Furthermore, when comparing the two samples the p-value once again shows how there is not a clear difference between the two groups.

Unfortunately, the interpretation of these results is limited by the multiple sources of variability introduced throughout the imaging and processing pipeline. Patient positioning, scan resolution, segmentation accuracy, and registration fidelity can all have an impact in the volume measurements. The reality is that a 20% growth in muscle mass for whichever group is highly unlikely indicating inflated or implausible measurements.

This presents a harsh reality that is, **properly measuring psoas atrophy is a complex process that involves many stages prone to errors**. The first one being inconsistencies with the original scans, as a muscle is a dynamic structure that will change its shape and size depending on the level of contraction or elongation. Any changes in patient position when receiving the scan will affect the psoas segment extracted through the imaging and processing pipeline.

As an attempt of reducing these systematic errors, the relative difference between the intervened and control psoas (i.e., Intervened – Control) was calculated as it is a more robust and interpretable measure. Because both groups are processed using the same tools, segmentations, and source image, systemic biases and errors likely affect both in similar ways. So by pairing these differences within the same patient and same image, many of those artefacts effectively **cancel out**, revealing a truer estimate of relative atrophy. In this context, the - 5.15% mean difference with a lower variability (SD = 13.29%) stands out as the most reliable indicator of intervention-related atrophy. Additionally, the negative Cliff's Delta (-0.15) is another measure that dismisses the absolute values but rather looks at what group tends to be larger than the other. These measurements conclude that **although there are no significant changes in volume increment between intervened and non-intervened groups there seems to be a slight tendency for intervened psoas to experience atrophy.**

6. Discussion

After evaluating the results of the clinical applications of the designed tool, it's also important to assess whether the tool reached the initial objectives set. After all the presented results are just a demonstration of how this tool can be applied in a clinical context, but don't evaluate the effectiveness of the new pipeline. To adress this several quantitative and qualitative measures have been calculated and are discussed in this section.

The objectives set at beginning of this project where:

- ***Modularity***
  Define independent processing blocks to allow for adaptable and personalized analyses.
- ***Scalability***
  Make the system extensible to support new analyses and adapt to any tissue or structure.
- ***Minimal user interaction***
  Automate repetitive tasks and streamline workflows for efficiency.
- ***Batch processing***
  Enable the analysis of multiple patients simultaneously to support broader studies.
- ***User Friendliness***
  Provide an intuitive structure with checkpoints and a clean, accessible interface.

6.1. Evaluation Using Quantitative Performance Metrics

The processing time of certain phases can help evaluate whether the designed code is fulfilling its objectives. An estimation of the time taken during the input phase for both batch loading and individual loading, as well as using automatic or manual image selection is shown in the table below.

	Manual Image Selection	Automatic Image Selection
Individual Patient (9 series)	5 min	3 min
Batch of 5 Patients (58 series)	14 min	8 min

Table 18: The first row shows an example of the times taken to load a single patient with manual or automatic image selection. The second shows the times taken to load a batch of 5 patients.

In both individual and batch loading modes using automatic image selection is faster than using the manual mode thanks to the fact that images do not need to be displayed and the user does not

need to interact with the interface, reducing both the time taken to select the image and user interaction. These results also show how loading patients in batch is also less time-consuming than individually. If the time taken for and individual patient with 9 DICOM series is of 5 minutes (manual) and 3 minutes (automatic) one could assume that loading 5 patients with 45 DICOM series of equal characteristics would take around 25 minutes and 15 minutes respectively. However, these results show that loading 5 patients with a total of 58 DICOM series took 14 minutes using manual selection and only 8 minutes using automatic selection. This is almost half of the processing time for the same number of patients, demonstrating how batch processing is working effectively. Furthermore, in terms of user interaction, using the batch loading mode does not require the patient to manually introduce each patient's metadata as it is all extracted from the Batch Input Excel file, reducing both user interaction and processing time significantly

The rest of the pipeline's run time is primarily composed of the segmentation step. This can take up to 7 minutes per patient depending on the levels intervened and the segmentation mode (fast mode segmentation is faster but has lower resolution). However, after the segmentation, from mesh generation to the results, using the plane crop modality, the whole process takes between 2 and 3 minutes per patient depending on the levels intervened. This means that if the user has previously executed the segmentation step, results of 10 patients can be obtained in around 20 minutes. Splitting the workflow in this manner is recommended because many times segmentations can fail due to corrupt files or lack of memory space and it is highly practical to first have all the segmentations which take up a lot of processing time and then execute all the other following steps. These run times present a compelling case for this tools powerful batch processing capabilities.

Another quantifiable measurement that shows the effectiveness of this tool's file management is the **weight of the saved files**. This is a significant problem when dealing with large datasets, especially considering the type of files used to store CT or MR scans. These images contain a lot information and the better the resolution the more storage space they occupy. It is also remarked how a single patient scan can contain up to 9 DICOM series, which means there is a lot of redundant information considering only one of them will be used by the tool. After the Input Phase where an image is selected for each pre- and post-surgery visit, the total storage space of the ALIF dataset was around 7.9 Gb, while the original database containing more than a single series per visit weighed 19.3 Gb. This metric shows the importance including this image selection as it can reduce the weight of the files used by more than half. This metric shows how the tool improves usability as unnecessary storage space is reduced as much as possible.

6.2 Evaluation of Design Principles Through Qualitative Improvements

This updated patient-centered, object-oriented approach solves many of the limitations identified in the original tool and allows the addition of new modules and functions in a user-friendly way. Previously, each level was interpreted as a separate case, but with the refactored code structure a single patient can have many levels thanks to the storage of information through the Patient and Visit classes. One of the LLIF cases even presented intervention in 4 separate levels, and the pipeline processed each individually while still saving all progress in the single patient. Another crucial incorporation that demonstrated the ability to scale this tool is the inclusion of the psoas major volumetric analysis. The addition of this ROI demonstrates how the tools OOP structure welcomes new functionalities and is even designed to facilitate them. This not only re-enforces the scalability of the tool but also the modularity. This project shows different crop modes, image selection methods and patient loading modes, as well as encouraging many other additions to be implemented. For instance, a framework to include various AI segmentation models is already implemented, as well as encouraging the design of an automatic crop modality, to eliminate user interaction entirely in the execution phase.

6.3 Current Limitations and Future Work

Because of the defined objectives the focus of this project was not to perfect the methodologies of analysis, which is why no validations were performed for any for the analysis steps. While the spinal cord registration methods and morphological analysis where previously validated in the original tool's paper [1], this project does not include this type of analysis. Some of the side of effects of the lack of a validation were seen in the registration process where some cases resulted in a incorrect spatial alignment. In the case of the spinal cord, because ALIF cases always include the S1 this is a structure that TotalSegmentator struggles to segment properly, especially with interference of intersomatic cages. And in the case of the psoas major muscle, aligning these structures can introduce problems that did not exist with the spinal cord and the original tool based its registration process for the spinal cord.

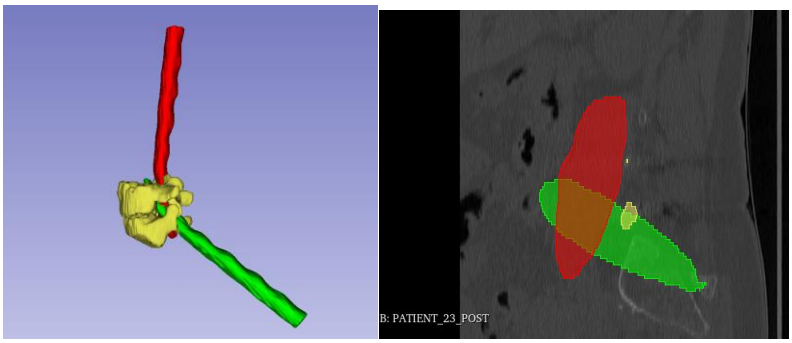


Figure 50: Example of registration errors. Left snapshot shows erroneous spinal cord registration. Right Image shows improper alignment of psoas muscles.

The incorporation of the psoas muscle to a framework that was initially designed for the spinal cord also showed certain inconveniences. For instance, while the per vertebrae level analysis of the spinal cord decompression makes sense, it doesn't for the psoas. To measure atrophy, it is more convenient to look at the full volume of the psoas (or as much of it as possible). The reality is a specific psoas registration, crop and analysis (not per-level) should also be implemented in the pipeline for more representative results. The reality is that due to time constraints many of the spinal cord procedures were adapted to the psoas, which served to demonstrate the potential of the tool but not to take conclusions from the obtained results.

Finally, the most important improvements that would elevate this tools user-friendliness and also reduce user interaction are the implementation of a graphical-interface as well as the development of an automatic crop modality. The Interface would give a lot to this tool as it would allow clinicians to use the code without needing to write a single line of code. Currently the pipeline still works through the terminal, and while it has messages and icons to facilitate its usage, it is not the same as having a dedicated user interface. Surgeons have tried the tool and are familiar with how the pipeline works, but it is evident that for them to extract its full potential an interface has to be designed. And the automatic crop is an essential part of the automatization of the execution phase, as it would allow the user to setup the analysis and obtain results in a completely automatic way. Both of these improvements will be developed in the upcoming weeks.

7. Conclusions

This project has successfully presented the development of a tool capable of automatically extracting quantifiable metrics for volumetric and morphologic analysis across multiple patients at the same time. The implemented modular design allows users to select the anatomical structure of interest, such as the spinal cord or psoas major muscle, and tailor the execution pipeline to specific analysis requirements.

The limited ALIF and LLIF datasets that served for obtaining the preliminary results evaluated in this work should not be considered representative of a full clinical evaluation and should not be taken as reference. However, they illustrate the potential value that such tool can provide in clinical applications.

Despite certain limitations, the tool demonstrates to have reached most of its objectives. The modular design has proven effective in enabling scalability, supporting batch-processing, and minimizing user intervention. Nonetheless, the current reliance on terminal-based interaction instead of a usable graphical-interface possess a barrier for clinical practitioners with no programming experience.

Ultimately, the outcomes of this work underscore the value of integrating proper modular architecture and automation in a medical pipeline. This tool sets a foundation for a much larger project that includes many more anatomies and functionalities to truly create a robust and powerful volumetric and morphologic comparative analysis tool. This project opens the door to many more clinical applications that would benefit from this type of quantifiable volumetric metrics and analysis.

Sustainability analysis and ethical implications

Sustainability analysis

This project generated minimal environmental impact as there were no real waste products or physical consumptions. However the use of a personal laptop and the transportation to the hospital facilities could be taken into account.

The transportation consisted of 3.5 km by metro there and back once a week for meetings with the team. According to emission factors, metros generate 0.0237 kg CO_2/km per person [24]**.** Assuming 1 day per week over 14 weeks**,** the estimated total CO_2 emissions from commuting amount to approximately **2.323 kg**.

The laptop used during development was a HP EliteBook x360 1030 G2 with a 57 Wh battery capacity, lasting roughly 5 hours per cycle. Based on an estimated total usage of 560 hours, this equates to approximately 112 cycles, resulting in an energy consumption of **6.384 kWh**. Using the Spanish electricity mix value of 0.26 kg CO_2/kWh [25], the emissions from computer use are estimated to be **1.659 kg**.

In total, the environmental impact of this project amounts to approximately **3.982 kg of CO_2**, which is relatively low considering the duration and scope of the work.

Ethical Considerations

This project also takes into account key ethical principles related to patient data and medical imaging. First, a recurrent effort is made for all patient imaging data to be anonymized prior to processing, ensuring full compliance with data protection regulations and privacy standards. There are steps implemented in the tool that specifically intended to respect patient privacy. Additionally, no personal information was used or stored at any stage of the project.

Second, the project relies on CT imaging data, which involves exposure to ionizing radiation. This not ideal and can even be a limitation for future research if the tool is dependent on this scan modality. For this reason it is important to incorporate MR scans in the pipeline. However it is important to remark that no additional scans were performed for this work, as imaging was previously acquired as part of standard clinical practice.

Economic analysis

This section presents an estimate of the cost associated with this Project. In terms of materials the only resource used throught the development was a personal laptop, which is not included in this budget as it was already in my possession. Similarly, the tools used during development including Python libraries, medical software such as 3D Slicer, and segmentation models like TotalSegmentator, are free and open-source, thus not generating any additional expenses.

The only cost to consider is the value of the work performed. Although no fixed schedule was followed, the total time dedicated to the project was around 40 hours a week. Since the project began in April and 14 weeks have elapsed, the total estimated time spent amounts to approximately 560 hours.

According to salary references [26] for software engineers in Spain, the average annual salary is around 54.000 €/year which corresponds to an hourly rate of approximately **25.96 €/h**. Applying this rate to the estimated hours worked brings the total cost to **14,537.60 €.**

Bibliografía

[1] S. M. Barroso, «Herramienta semiautomática para la cuantificación volumétrica y morfológica del saco dural intervenido en casos de descompresión medular,» UPC, 2024.

[2] A. Fedorov, R. Beichel, J. Kalpathy-Cramer, J. Finet, J.-C. Fillion-Robin, S. Pujol, C. Bauer, D. Jennings, F. M. Fennessy, M. Sonka, J. Buatti, S. R. Aylward, J. V. Miller y Pieper, «3D Slicer as an Image Computing Platform for the Quantitative Imaging Network,» *Magnetic Resonance Imaging,* vol. 30, 2012.

[3] Z. Chen, Y. Wang, X. Li, K. Wang, Z. Li y P. Yang, «An automatic measurement system of distal femur morphological parameters using 3D slicer software,» *Bone,* vol. 156, 2022.

[4] D.-H. Lee, D.-G. Lee, J.-S. Hwang, J.-W. Jang, D.-H. Maeng y C.-K. Park, «Clinical and radiological results of indirect decompression after anterior lumbar interbody fusion in central spinal canal stenosis,» *Journal of Neurosurgery: Spine,* vol. 34, nº 4, 2021.

[5] P. B. Derman, D. D. Ohnmeiss, A. Lauderback y R. D. Guyer, «Indirect Decompression for the Treatment of Degenerative Lumbar Stenosis,» *International Journal of Spine Surgery,* vol. 15, nº 6, 2021.

[6] W. Limthongkul, C. Thanapura, K. Jitpakdee, P. Praisarnti, V. Kotheeranurak, W. Yingsakmongkol, T. Tanasansomboon y W. Singhatanadgige, «Is Direct Decompression Necessary for Lateral Lumbar Interbody Fusion (LLIF)? A Randomized Controlled Trial Comparing Direct and Indirect Decompression With LLIF in Selected Patients,» *Neurospine,* vol. 21, nº 1, 2024.

[7] R. Takatori, T. Ogura, W. Narita, T. Hayashida, H. Tonomura, Y. Mikami, M. Nagae, K. Ikoma y T. Kubo, «Leg Muscle Strength After Lateral Interbody Fusion Surgery Recovers Over Time After Temporary Muscle Weakness,» *Clinical Spine Surgery,* vol. 32, nº 3, 2019.

[8] A. Hiyama, H. Katoh, S. Nomura, D. Sakai, M. Sato y M. Watanabe, «Radiographs assessment of changes in the psoas muscle at L4-L5 level after single-level lateral lumbar interbody fusion in patients with postoperative motor weakness,» *Journal of Clinical Neuroscience,* vol. 90, 2021.

[9] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen y K. H. Maier-Hein, «nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation,» *Nature Methods,* vol. 18, nº 2, 2021.

[10] J. Wasserthal, H. C. Breit, M. T. Meyer, M. Pradella, D. Hinck, A. W. Sauter y e. al., «TotalSegmentator: Robust Segmentation of 104 Anatomic Structures in CT Images,» *Radiology: Artificial Intelligence,* vol. 5, nº 5, 2023.

[11] Y. Varun, «Understanding Dice Coefficient,» 2022 . [En línea]. Available: https://www.kaggle.com/code/yerramvarun/understanding-dice-coefficient. [Último acceso: 2025 5 23].

[12] J. Haubold, G. Baldini, V. Parmar, B. M. Schaarschmidt y e. al., «BOA: A CT-Based Body and Organ Analysis for Radiologists at the Point of Care,» *Investigative Radiology,* vol. 59, 2024.

[13] M. U. Saeed, N. Dikaios, A. Dastgir, G. Ali, M. Hamid y F. Hajjej, «An Automated Deep Learning Approach for Spine Segmentation and Vertebrae Recognition Using Computed Tomography Images,» *Diagnostics (Basel, Switzerland),* vol. 13, 2023.

[14] Unknown, «Anatomy of a normal spine,» Saint Luke's Health System, [En línea]. Available: https://www.saintlukeskc.org/health-library/anatomy-normal-spine. [Último acceso: 5 5 2025].

[15] D. W. Moore, «Spinal Cord Anatomy,» 30 7 2024. [En línea]. Available: https://www.orthobullets.com/spine/2004/spinal-cord-anatomy. [Último acceso: 23 4 2025].

[16] M. A. Siccardi, M. A. Tariq y C. Valle., «Stat Pearls,» National Library of Medicine, 8 8 2023. [En línea]. Available: https://www.ncbi.nlm.nih.gov/books/NBK535418/#:~:text=The%20psoas%20muscle%20is%20among,to%20form%20the%20iliopsoas%20muscle.. [Último acceso: 1 5 2025].

[17] J. Filho, R. V. Leao, N. Horvat, P. Helito, D. Amaral, P. Viana, I. Louza y M. Bordalo Rodrigues, «What abdominal radiologists should know about extragenital endometriosis-associated neuropathy,» *Abdominal Radiology,* vol. 45, nº 10.1007/s00261-018-1864-x, 2020.

[18] R. J. Mobbs, K. Phan, G. Malham, K. Seex y P. J. Rao, «Lumbar interbody fusion: Techniques, indications and comparison of interbody fusion options including PLIF, TLIF, MI-TLIF, OLIF/ATP, LLIF and ALIF,» *Journal of Spine Surgery,* vol. 1, nº 1, 2015.

[19] M. Clinic, «Spinal decompression: Laminectomy & foraminotomy,» 25 5 2021. [En línea]. Available: https://mayfieldclinic.com/pe-decompression.htm. [Último acceso: 2025].

[20] B. M. Ozgur, H. E. Aryan, L. Pimenta y W. R. Taylor, «Extreme Lateral Interbody Fusion (XLIF): a novel surgical technique for anterior lumbar interbody fusion,» *The Spine Journal,* 2006.

[21] J. d. Estado, «Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales,» Boletín Oficial del Estado, 2018.

[22] W. E. y. C. H. E. Lorensen, «Marching cubes: A high resolution 3D surface construction algorithm,» *ACM SIGGRAPH Computer Graphics,* vol. 21, nº 4, 1987.

[23] J. Zhang, Y. Yao y B. Deng, «Fast and Robust Iterative Closest Point,» *IEEE Transactions on Pattern Analysis and Machine Intelligence,* vol. 44, nº 7, 2020.

[24] TMB, «Mobilitat sostenible,» TMB, [En línea]. Available: https://www.tmb.cat/ca/coneix-tmb/qualitat-i-mediambient/mobilitat-sostenible. [Último acceso: 10 06 2025].

[25] Gencat, «Factor de emisión de la energía eléctrica: el mix eléctrico,» Gencat, 2024. [En línea]. Available: https://canviclimatic.gencat.cat/es/actua/factors_demissio_associats_a_lenergia/. [Último acceso: 09 06 2025].

[26] R. Andrés, «Sabemos cuál es el salario de un ingeniero de software en España: comparamos Europa, Reino Unido y EE.UU.,» Xataka, 12 3 2025. [En línea]. Available: https://www.xataka.com/empresas-y-economia/sabemos-cual-salario-ingeniero-software-espana-comparamos-europa-reino-unido-eeuu.

Annex 1: Patient and Visit Class Attributes and Methods

Patient class Overview

This class represents the complete unit for a single patient.

Attributes	Type	Description
patient_number	int	Unique numeric ID assigned to the patient.
case_type	str	Describes the surgical classification (e.g., "MISS", "OPEN").
levels	list[str]	List of vertebral levels intervened in the procedure (e.g., ["L3", "L4"]).
output_path	str	Path to the folder where all outputs related to this patient are stored.
pre	VisitData	Visit object storing data for the pre-operative session.
post	VisitData	Visit object storing data for the post-operative session.
segmentation_tool	str	Name of the segmentation tool selected (e.g., "total_segmentator").
roi	str	Region of interest selected for analysis (e.g., "spinal_cord").
other	list[str]	Additional anatomical labels selected for segmentation.
use_fast	bool	Whether the segmentation was configured to run in fast mode.
registration_paths	list[str]	Paths to registered meshes between pre and post sessions.
crop_paths	list[str]	Paths to mesh files cropped along anatomical or geometric planes.
analysis_results	list[str]	Paths to analysis results folders containing figures and metrics.

Table 19: Overview of Patient class attributes.

Methods	Description
log_registration(path_or_list)	Stores the paths to registration outputs.
log_crop(path_or_list)	Records the locations of cropped mesh outputs.
log_analysis(results_dict)	Logs computed results from shape or volumetric analysis.
summary()	Prints the current status of the patient, including metadata and saved outputs.
to_dict()	Converts the entire patient object to a dictionary for serialization.
from_dict(data)	Reconstructs a Patient object from a saved JSON dictionary.
save_to_json()	Saves the object to a JSON file as a checkpoint of pipeline progress.
load_from_json(output_path, patient_number)	Loads an existing patient JSON from disk and reconstructs the object.

Table 20: Overview of Patient class methods.

VisitData class Overview

This class represents a single unit visit. Typically each patient will contain two VisitData objects.

Attributes	Type	Description
visit_type	str	Indicates whether this is the "PRE" or "POST" imaging session.
visit_output_path	str	Path to the folder where outputs from this specific visit are stored.
nifti_path	str or None	Path to the selected NIfTI file used in segmentation.
segmentation_paths	list[str]	List of paths to the final (combined and smoothed) segmentation masks.
other_segmentations_path	str or None	Folder containing all other segmentation masks that were not combined.
mesh_paths	list[str]	Paths to the mesh output directories generated from segmentations.
modality	str or None	Imaging modality used in this visit (either "ct" or "mr").

Table 21: Overview of VisitData class attributes.

Methods	Description
log_registration(path_or_list)	Stores the paths to registration outputs.
log_crop(path_or_list)	Records the locations of cropped mesh outputs.
log_analysis(results_dict)	Logs computed results from shape or volumetric analysis.
summary()	Prints the current status of the patient, including metadata and saved outputs.
to_dict()	Converts the entire patient object to a dictionary for serialization.
from_dict(data)	Reconstructs a Patient object from a saved JSON dictionary.
save_to_json()	Saves the object to a JSON file as a checkpoint of pipeline progress.
load_from_json(output_path, patient_number)	Loads an existing patient JSON from disk and reconstructs the object.

Table 22: Overview of VisitData class methods.

Annex 2: Code Structure of Core Control Modules

Code Structure Overview: main.py

main.py is the entry point of the pipeline. It presents a main menu where the user can either load patients, setup analysis, or run the analysis. It controls the execution flow between the three phases. It also incorporates functions from the **Prep_and_Tools** folders to set the global output directory, slicer directory and shows available patients both on the terminal as well as updating the Patient List excel.

Functions used in main.py:

Function	Purpose	Location
select_main_directory()	Select working folder (where Results are saved)	Prep_and_Tools.ui_tools
select_slicer_path()	Select 3D Slicer executable path	Prep_and_Tools.ui_tools
get_user_selection()	CLI prompt for user menu selection	Prep_and_Tools.ui_tools
find_patients()	Discover existing .txt config files	Prep_and_Tools.patient_tools
show_patients()	Print summary of all discovered patients	Prep_and_Tools.patient_tools
export_patient_list_to_excel()	Export list of config files to Excel	Prep_and_Tools.results_structure
phase1_load_patients()	Phase 1: Load and prepare patient configs	pipeline.input_phase
phase2_setup()	Phase 2: Configure analysis steps + patients	pipeline.setup_phase
execute_analysis()	Phase 3: Run pipeline steps for each patient	pipeline.execution_phase

Table 23: Overview of functions used in main.py.

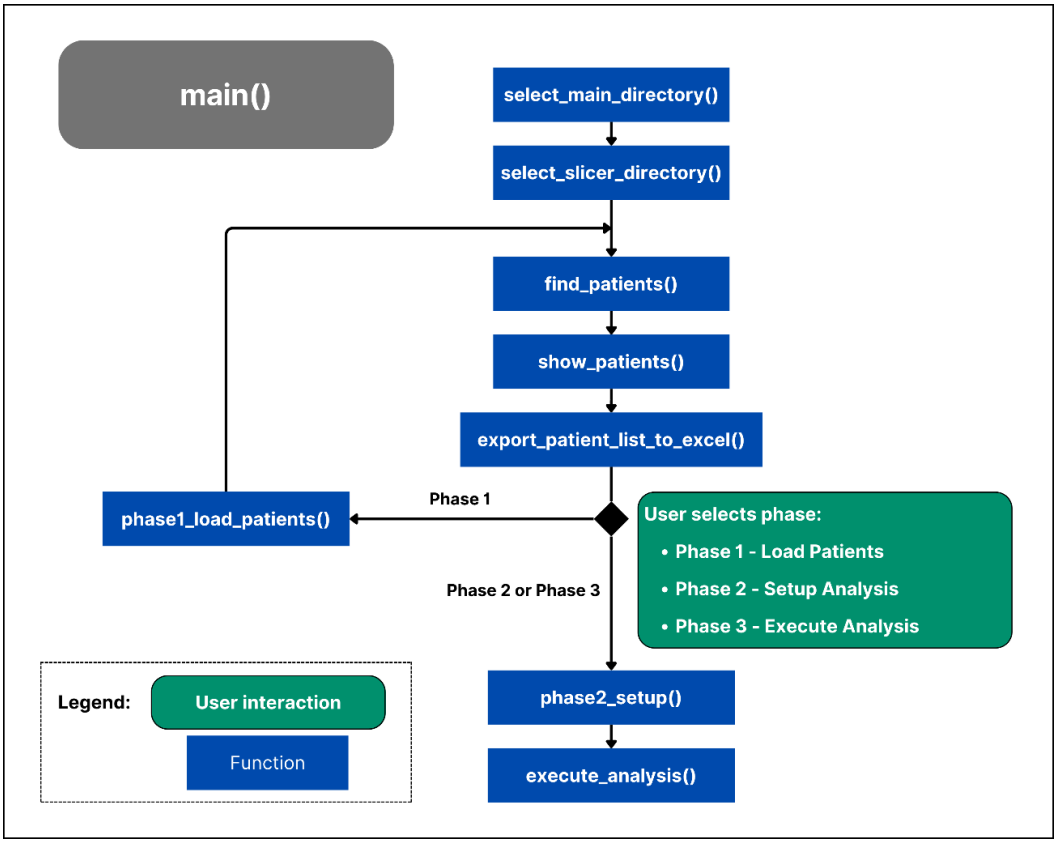


Figure 51: Flowchart of main() using the functions explained in table 23.

Code Structure Overview: input_phase.py

Each phase has its own high level orchestration function that dictates the general flow for the entire phase. The first is ***phase1_load_patients()*** *that* uses functions within the **Prep_and_Tools** folder. The general flow of the function is dictated by the type of loading mode, by executing ***batch_patient_prep()*** or ***individual_patient_prep()*** functions. These are also high level orchestration functions stored in the ***prep_workflows.py*** script. They use many helper functions from the **Prep_and_Tools** folder that perform specific tasks such as extract information from the batch input excel, organize patient files, or converting DICOM files to NifTi. The ***automatic_image_selection()*** function is also used in both batch and individual loading if selected by the user.

Functions used in phase1_load_patients() :

Function	Purpose	Location
get_user_selection()	Displays menu for selecting input mode (Batch, Individual)	Prep_and_Tools.ui_tools
batch_patient_prep()	Parses a CSV file and prepares multiple patients at once	Prep_and_Tools.prep_workflows
individual_patient_prep()	Manually prepares a single patient by collecting info from CLI	Prep_and_Tools.prep_workflows

Table 24: Overview of functions used in phase1_load_patients()

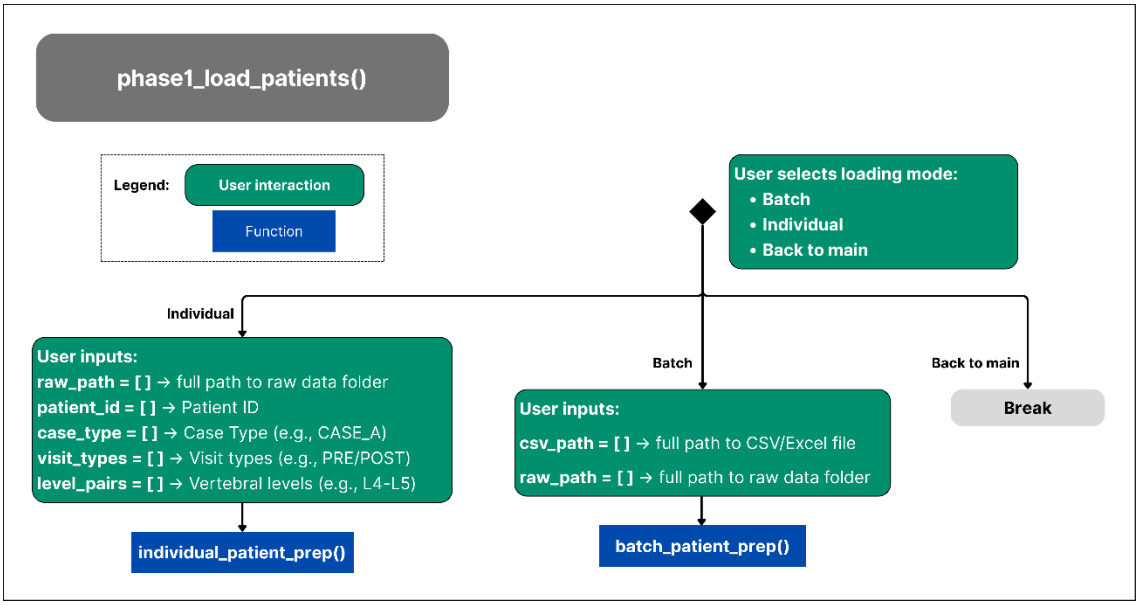


Figure 52: Workflow of phase1_load_patients() showing the user inputs required for batch_patient_prep() and individual_patient_prep() functions.

As seen in figure X, once in the input phase there are three possible actions the user can take. The option Batch and Individual are the two possible loading modes, and the Back to main option simply takes the user back to the main menu.

Code Structure Overview: setup_phase.py

This is the analysis setup phase where the user selects patients, configures the steps of the analysis pipeline and prepares them for execution. The higher level orchestrator function ***phase2_setup()*** calls functions from the **Prep_and_Tools** folder and creates or re-instates Patient objects for the selected patients.

Functions used in *phase2_setup()*:

Function / Method	Purpose	Location
get_user_selection()	CLI selection menu (e.g., for patients, steps, options)	Prep_and_Tools.ui_tools
get_segmentation_config()	Prompt for tool, ROI, modality, etc.	Prep_and_Tools.ui_tools
load_case_config()	Load patient metadata from config .txt	Prep_and_Tools.patient_tools
clean_patient_analysis_files()	Delete previous results (segmentations, meshes, etc.)	Prep_and_Tools.patient_tools
create_analysis_results_excel()	Create or append Excel for analysis summary	Prep_and_Tools.results_structure
Patient.load_from_json()	Load existing patient state object	classes.Patient
Patient.save_to_json()	Save updated patient to JSON	classes.Patient
Patient()	Create new patient object from config	classes.Patient
patient.pre.log_nifti()	Log NIfTI path for PRE visit	VisitData (inside Patient class)
patient.post.log_nifti()	Log NIfTI path for POST visit	VisitData (inside Patient class)
patient.pre.log_modality()	Save scan modality for PRE visit	VisitData (inside Patient class)
patient.post.log_modality()	Save scan modality for POST visit	VisitData (inside Patient class)

Table 25: Overview of functions used in phase2_setup()

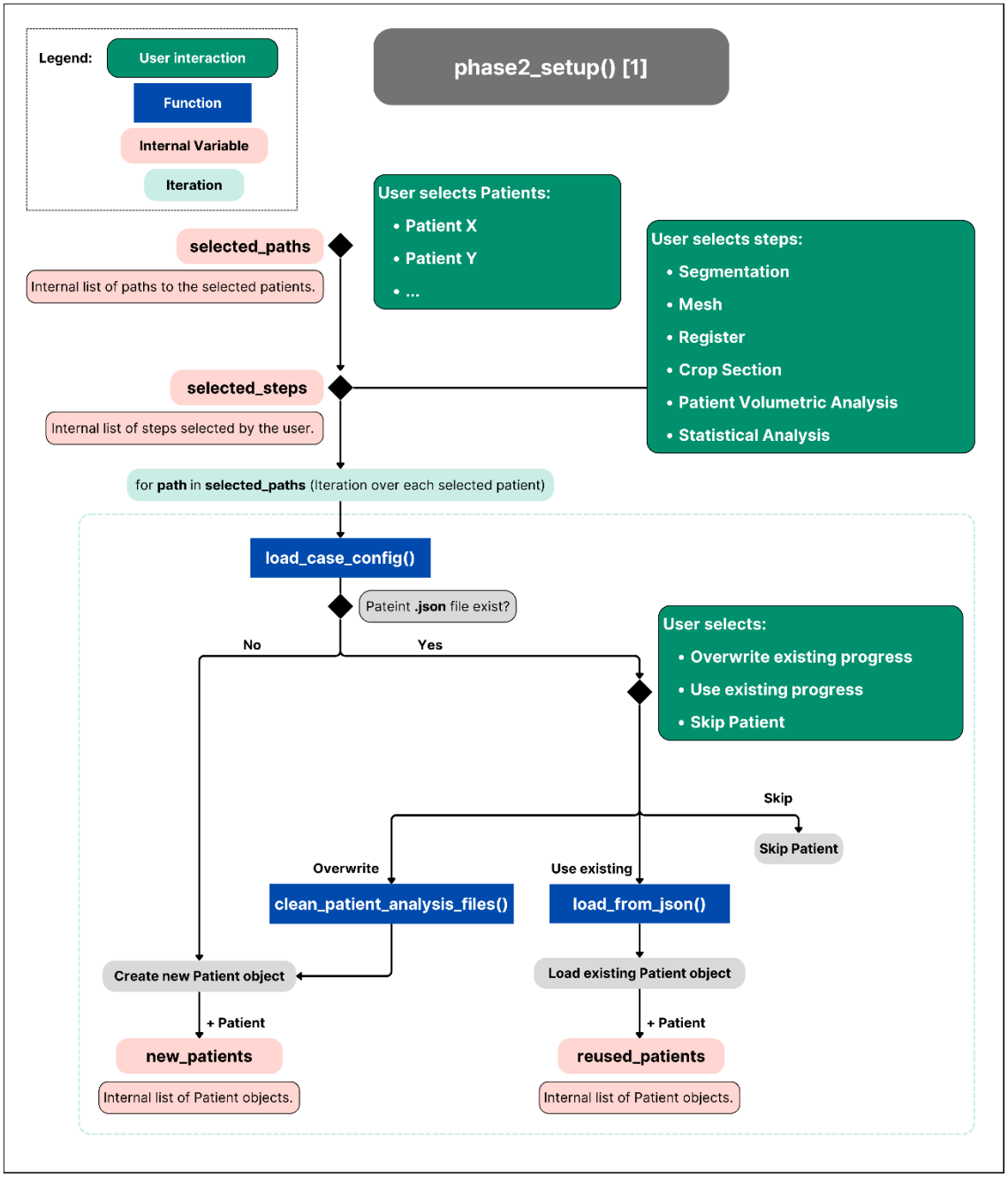


Figure 53: Flowchart 1 of phase2_setup()

Figure X shows how the code creates two separate lists for new patients and reused patients. The new patients are simply new objects which are added to the list, while the reused ones need to go through the process of re-loading the existing progress or overwriting and cleaning the patient folder from pre-existing progress. If the user selects to overwrite the patient then it will be added to the **new_patients** list, while reused objects will have their own internal list **reused_patients**.

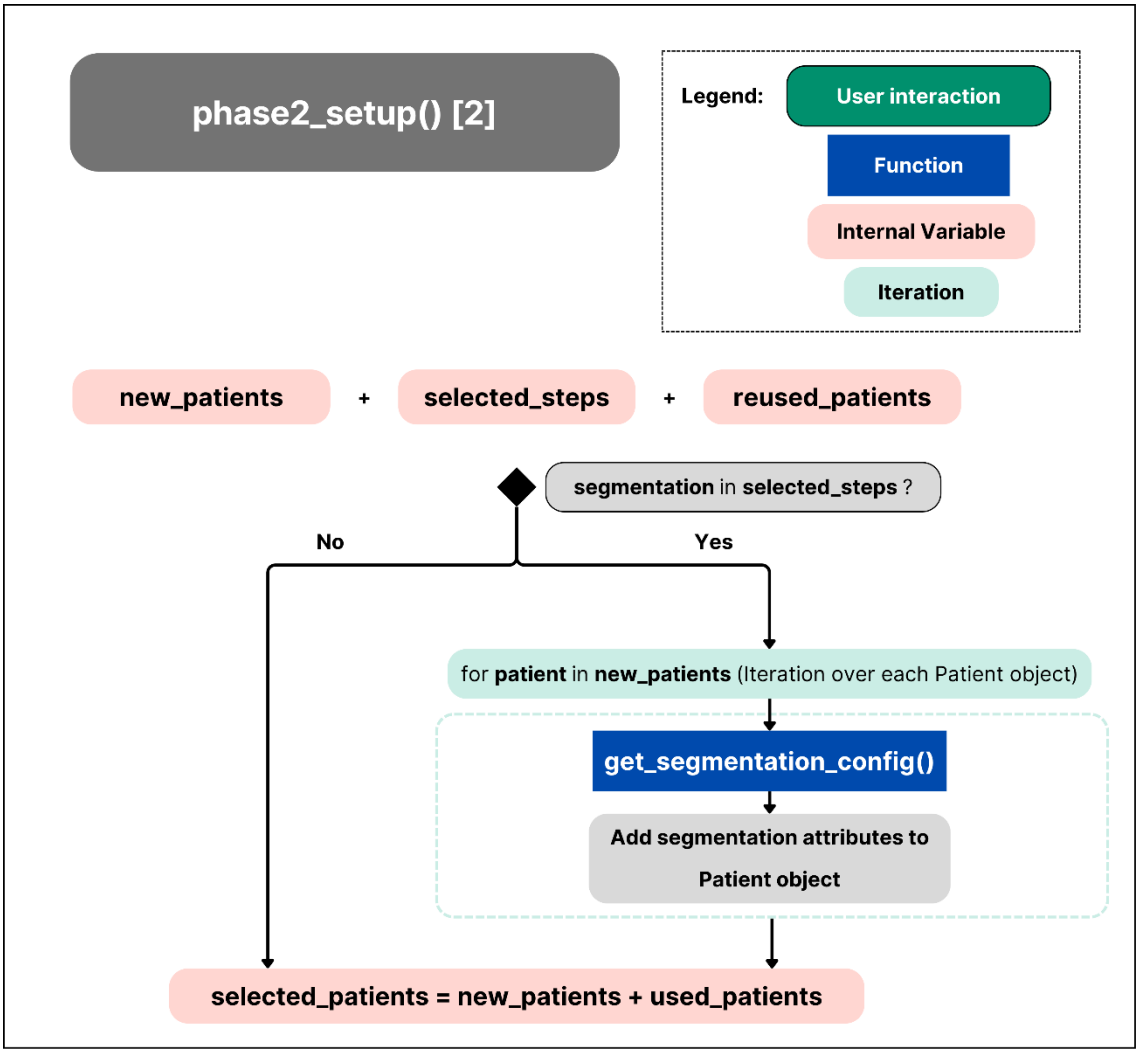


Figure 54: Flowchart 2 of phase2_setup()

Then the user selects the desired configuration for the selected steps. Because the segmentation configuration is saved individually within the patient object, only new patients are passed through the ***get_segmentation_config()*** function as reused patients will have its original configuration.

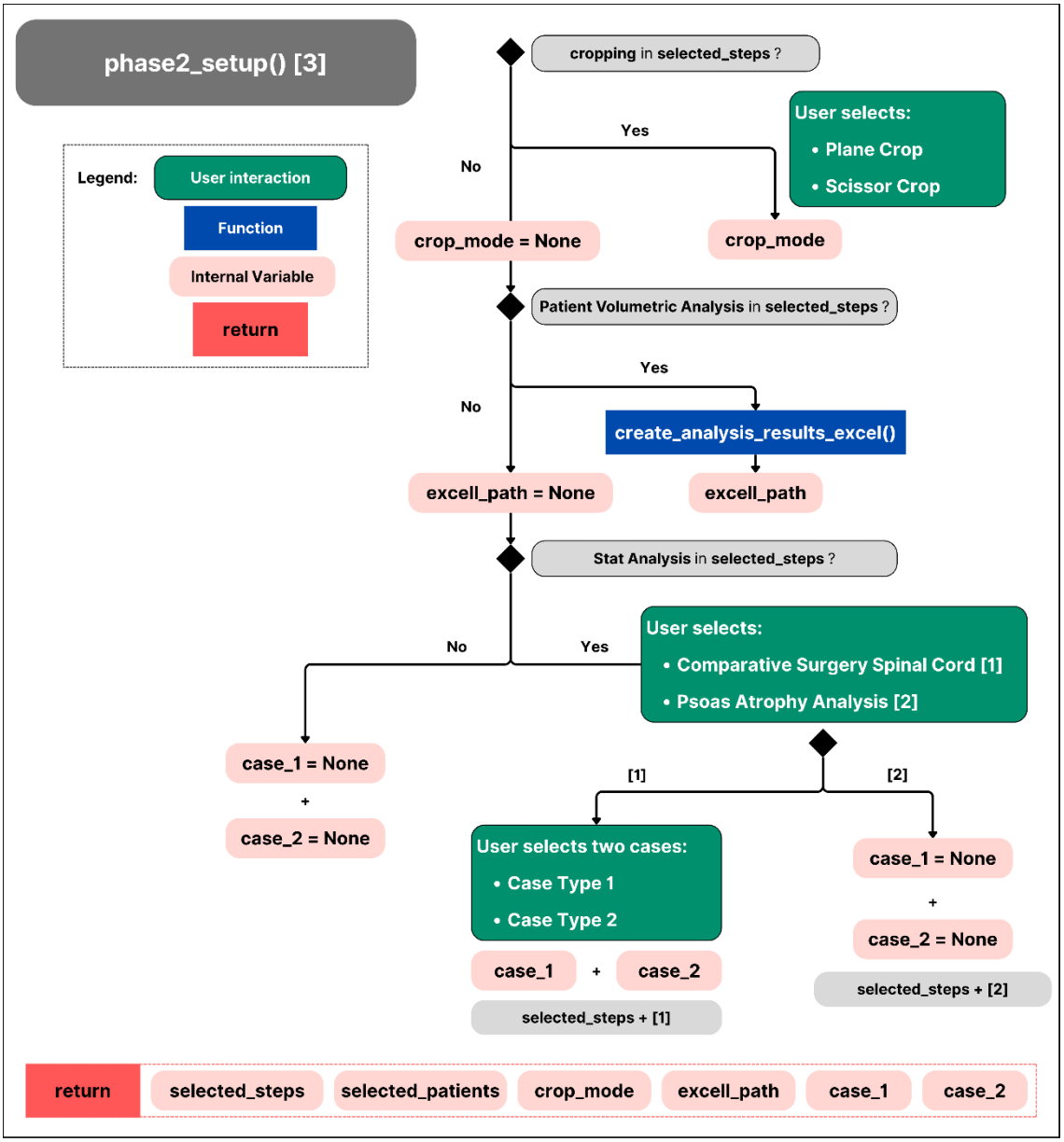


Figure 55: Flowchart 3 of phase2_setup()

The selected patients, steps and their configuration are returned and then used as inputs for ***execute_analysis()***, the execution phase high-level orchestrator function.

Code Structure Overview: phase3_execution()

In the ***execution_phase.py*** script, the ***execute_analysis()*** high level orchestration function runs all the selected processing steps for each patient selected. It also uses the helper functions ***is_step_done()*** and ***_check_paths_exist()*** which are defined locally and check if each step is done and skips completed steps to avoid redundancies. The functions called all belong to the Functional Modules as they carry out to the segmentation, mesh generation, registration, crop process, patient volumetric analysis and global statistical analysis.

Functions used in execute_analysis():

Function / Method	Purpose	Location
segmentation_process()	Executes segmentation for one patient visit	segmentation.py
mesh_process()	Generates meshes from segmented files	Mesh.execute_mesh
registration_process()	Registers POST to PRE volume	registration.py
analysis_process()	Runs volumetric/morphology analysis	Patient_Analysis.execute_analysis
global_spinal_cord_process()	Executes spinal cord comparison between grc	Stat_Analysis.execute_StatAnalysis
global_psoas_atrophy_process()	Executes psoas comparison between groups	Stat_Analysis.execute_StatAnalysis
save_to_json()	Saves updated Patient object state to disk	Patient (class method)
is_step_done()	Checks if a given step is already completed	Defined locally in execution_phase.py
plane_crop_process()	Executes plane-based cropping	Crop.execute_crop (dynamically imported)
scissor_crop_process()	Executes interactive scissor cropping	Crop.execute_crop (dynamically imported)
auto_crop_process()	Executes automatic cropping	Crop.execute_crop (dynamically imported)
_check_paths_exist()	Checks whether outputs exist for a step	Defined locally in execution_phase.py

Table 26: Overview of functions used in execute_analysis()

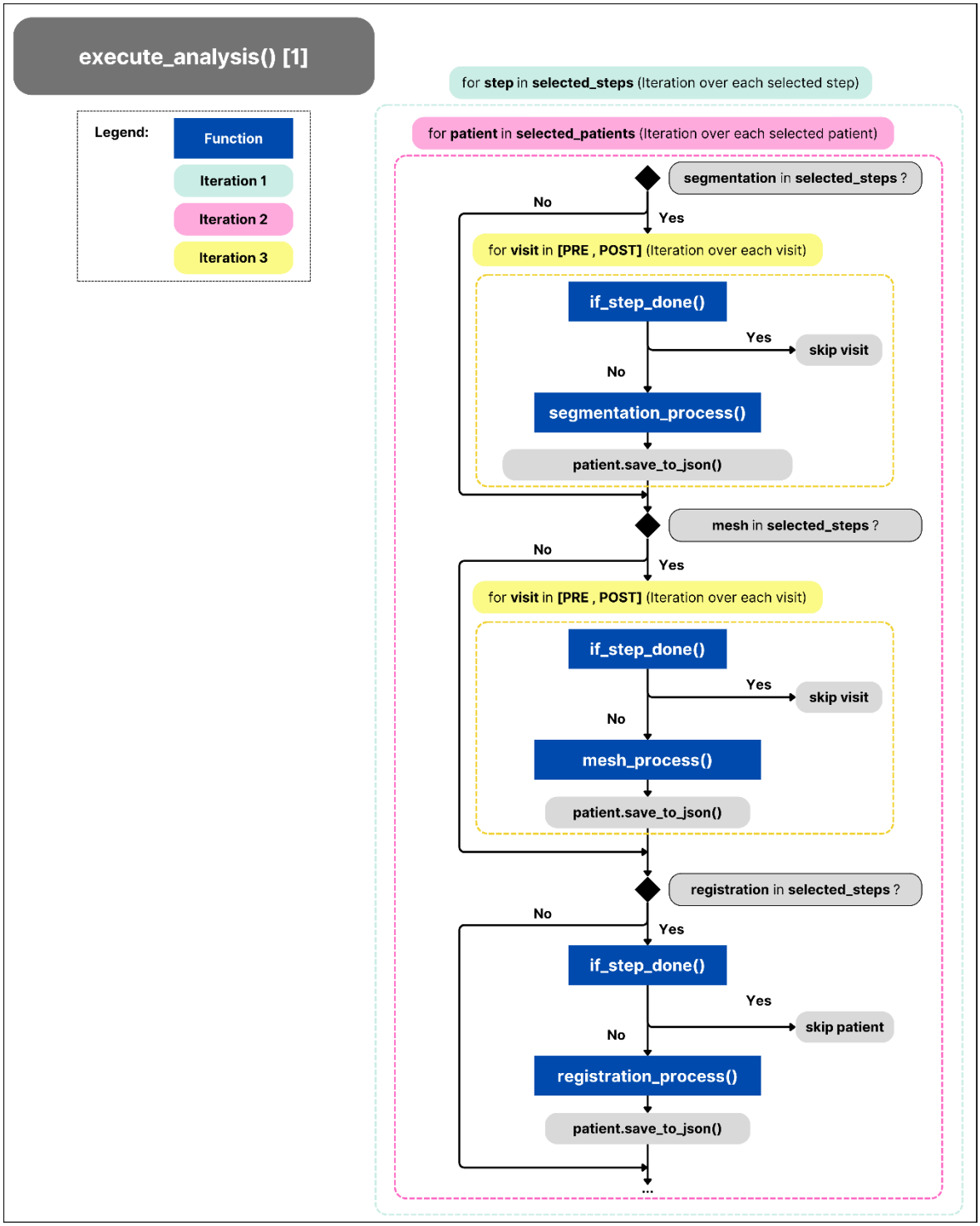


Figure 56: Flowchart 1 of execute_analysis()

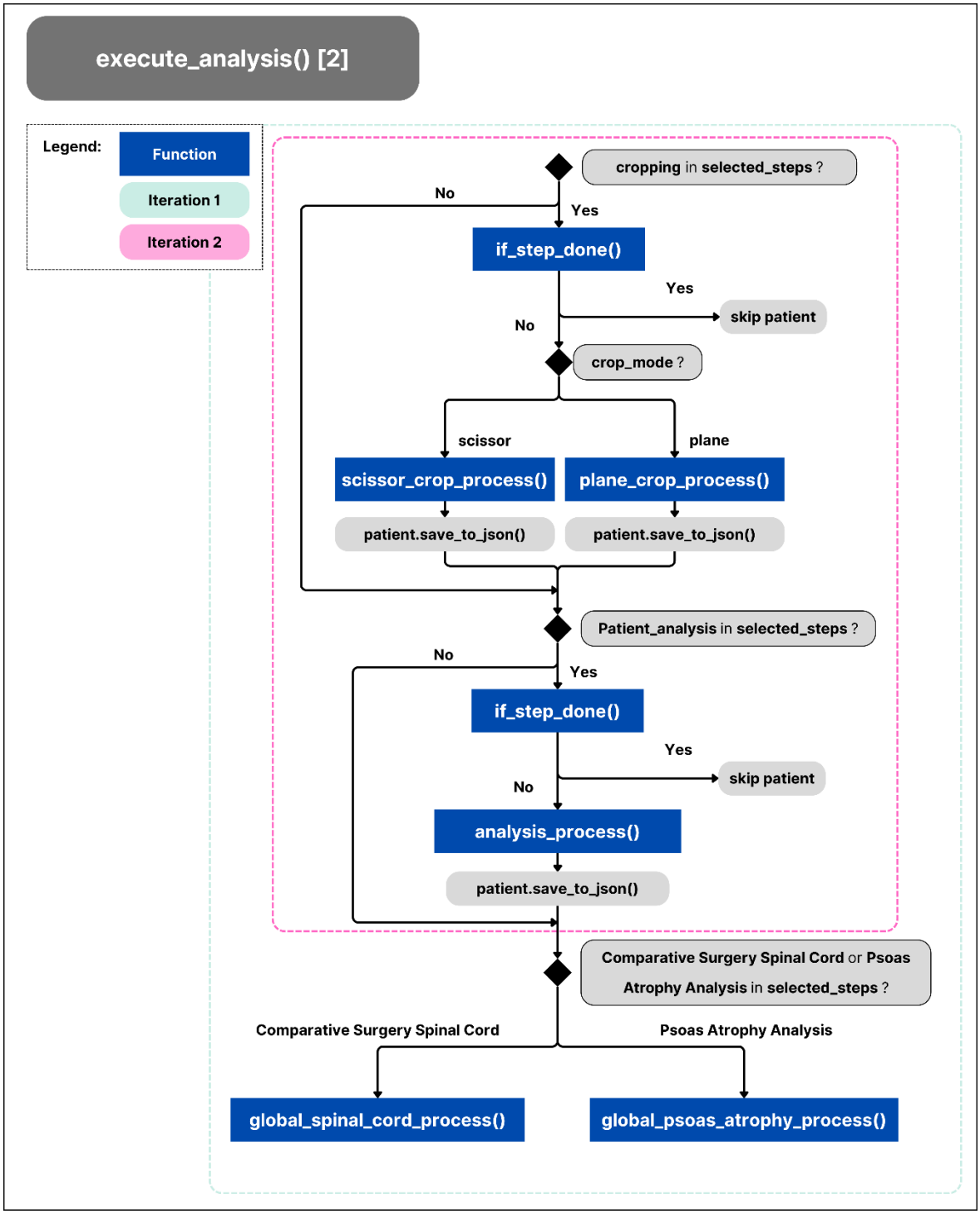


Figure 57: Flowchart 2 of execute_analysis()

Annex 3: Defined Functions In Functional Modules

Segmentation Module

Functions defined in segmentation.py script:

Function / Method	Purpose	Location
segmentation_process()	Main orchestration of the segmentation step per patient visit	segmentation.py
run_totalsegmentator()	CLI call to TotalSegmentator with dynamic ROI list	segmentation.py
combine_segmentations_spinal_cord()	Combines spinal cord + two vertebrae into a 3-label mask	segmentation.py
combine_segmentations_iliopsoas()	Combines iliopsoas L/R + two vertebrae into 4-label mask	segmentation.py
smooth_segmentation()	Smooths labeled image regions (per label ID)	segmentation.py
save_segmentation()	Saves .nii.gz file to disk and releases memory	segmentation.py
visit.log_segmentation()	Logs segmentation paths to VisitData	Patient class (via visit)

Table 27: Overview of functions defined in segmentation.py script.

Meshing Module

Functions defined in *execute_mesh.py* script:

Function / Method	Purpose	Location
mesh_process()	Main entry function for meshing per patient and visit	execute_mesh.py
subprocess.run()	Launches Slicer with MeshGeneration.py and input .json	Python Standard Library
visit.log_mesh()	Stores output mesh folder path in VisitData	VisitData class inside Patient

Table 28: Overview of functions defined in execute_mesh.py script.

Functions defined in *MeshGeneration.py* script:

Function	Purpose
main()	Entry point: parses input .json, loads segmentation, and triggers export
rename_segments()	Renames each segment using the label dictionary (e.g., Segment_1 → SpinalCord)
export_meshes()	Converts Slicer segments into .obj meshes using SubjectHierarchy
clean_output_folder()	Removes non-target .obj and .mtl files after export

Table 29: Overview of functions defined in MeshGeneration.py script.

Registration Module

Functions defined in registration.py script:

Function / Method	Purpose	Location
registration_process()	High-level orchestrator for registration	registration.py
extract_mesh_info()	Extracts vertebrae and ROI mesh paths from patient structure	registration.py
combine_vertebrae()	Merges vertebrae meshes into a single .obj for ICP	registration.py
spinal_cord_registration()	Registers spinal cord mesh using vertebrae alignment	registration.py
iliopsoas_registration()	Registers left and right iliopsoas meshes using vertebrae alignment	registration.py
patient.log_registration()	Logs output registration folder(s) into Patient object	Patient class

Table 30: Overview of functions defined in registration.py script.

Crop Module

Functions defined in execute_crop.py script:

Function	Purpose
plane_crop_process()	Launches **Slicer** in interactive mode using the PlaneCrop.py script, automatically loads the patient volume and meshes, and initializes a scene where the user can **manually place cutting planes** (via markup lines). It's a **semi-automatic cropping** process tailored for spinal cord or iliopsoas ROIs.
scissor_crop_process()	Also launches **Slicer** using the ScissorCrop.py script, which loads the required segmentations and volume, and sets up the **Scissors tool** for the user to manually apply the crop. Like the above, it's **semi-automatic**, combining scripted setup with manual execution.

Table 31: Overview of functions defined in execute_crop.py script

Functions defined in ScissorCrop.py script:

Function	Purpose
crop_roi()	Performs mesh-based cropping using Slicer's SegmentEditor. Combines the .obj meshes, crops with scissors, saves outputs.
import_model_as_segmentation()	Loads a mesh file and imports it into a segmentation node. Used inside crop_roi.
find_file_by_keywords()	Searches for a .obj file in the folder whose filename contains **all** specified keywords.
main()	Entry point when executed in Slicer CLI. Parses input JSON, resolves file paths, runs crop_roi() and exits Slicer.

Table 32: Overview of functions defined in ScissorCrop.py script

Functions defined in PlaneCrop.py script:

Function	Purpose
main()	Entry point for the script; parses input JSON and starts the process by calling load_data_and_prepare_editor.
load_data_and_prepare_editor()	Loads the reference volume and ROI meshes, sets up segmentation nodes, and prepares the Markups editor.
create_and_configure_line()	Creates a markups line node (used to define cropping planes) and configures its display.
show_popup_window()	Shows a popup window prompting the user to place control points before continuing.
save_line()	Called once the user confirms the line placement; handles logic for continuing to next line or performing cropping.
activate_line_tool()	Activates placement mode in Slicer for a given line node.
generate_planes_from_lines()	Converts two user-defined lines into VTK planes and visualizes them in the 3D scene.
crop_segments_between_planes()	Uses VTK clippers to crop the original segment meshes between two planes.
save_cropped_segments_as_nrrd()	Saves the cropped segments (PRE and POST) as .nrrd label maps into the specified output folder.

Table 33: Overview of functions defined in PlaneCrop.py script.

Patient Volumetric Analysis Module

Functions defined in execute_analysis.py and ShapeAnalysis.py script:

Function / Method	Purpose	Location
analysis_process()	Orchestrates analysis steps depending on ROI	execute_analysis.py
volume_value_from_patient()	Calculates volume (cm^3) from PRE/POST .nrrd masks and logs into Excel	execute_analysis.py
spinal_cord_shape_analysis()	Runs interpolation + area/perimeter metrics after rotating segmentations	execute_analysis.py
psoas_atrophy_analysis()	Calculates and saves psoas muscle stats (PRE/POST/increment) per level	execute_analysis.py
patient.log_analysis()	Stores path to analysis results in patient JSON	Patient class
rotate_segmentations()	Prepares masks for shape statistics by standardizing orientation	ShapeAnalysis.py
shape_statistics()	Computes interpolated path length, area, and perimeter for spinal cord	ShapeAnalysis.py

Table 34: Overview of functions defined in execute_analysis.py and ShapeAnalysis.py script.

Stat Analysis Module

Functions defined in StatAnalysis.py script:

Function / Method	Purpose	Location
global_spinal_cord_process()	Runs comparative spinal shape analysis between two case types	execute_StatAnalysis.py
global_psoas_atrophy_process()	Runs comparative volume analysis of psoas muscle	execute_StatAnalysis.py
general_spinal_cord_morphology_analysis()	Loads interpolated shape arrays and runs statistical comparison (area/perimeter)	execute_StatAnalysis.py
global_shape_statistics()	Computes group-wise mean/std, global stats, and plots (±std ribbons)	execute_StatAnalysis.py
general_psoas_atrophy_analysis()	Aggregates per-patient .json stats into global stats for Control/Intervened groups	execute_StatAnalysis.py
compute_stats()	Calculates mean, std, median, IQR, n for a numeric list	execute_StatAnalysis.py
cliffs_delta()	Computes Cliff's Delta effect size between two groups	execute_StatAnalysis.py
plot_histogram()	Plots histogram for a single group (volume change %)	execute_StatAnalysis.py
plot_boxplot()	Compares volume changes via boxplot (Intervened vs Control)	execute_StatAnalysis.py
save_spinal_cord_global_excel()	Saves spinal cord stats to Excel	Prep_and_Tools.results_structure
save_comparison_figures()	Saves spinal cord comparison figures	Prep_and_Tools.results_structure
save_psoas_global_excel()	Saves psoas stats to Excel	Prep_and_Tools.results_structure
save_psoas_figures()	Saves psoas histogram/boxplot figures	Prep_and_Tools.results_structure
create_analysis_results_excel()	Creates or appends a general results Excel file	Prep_and_Tools.results_structure

Table 35: Overview of functions defined in StatAnalysis.py script.

Annex 4: Prep_and_Tools Defined Functions

prep_workflows.py

This script defines the function that takes on the loading of multiple patients at the same time ***batch_patient_prep()*** and the function that loads a single patient ***individual_patient_prep()***. Because these are high-level orchestrator functions they use many other functions within the Prep_and_Tools modules. The functions used within this script are:

Function / Method	Purpose	Location
batch_patient_prep()	High-level orchestrator for loading multiple patients from spreadsheet	prep_workflows.py
individual_patient_prep()	Handles patient creation manually for a single case	prep_workflows.py
load_csv()	Loads patient metadata spreadsheet	Prep_and_Tools.io_tools
copy_csv()	Copies the CSV/Excel file into the Results folder for reference	Prep_and_Tools.io_tools
extract_patient_info()	Parses rows from the metadata file into structured patient dicts	Prep_and_Tools.patient_tools
organize_patient_files()	Sorts raw input files into SORTED folder by visit and series	Prep_and_Tools.patient_tools
create_results_structure()	Initializes patient folder inside the Results directory	Prep_and_Tools.results_structure
get_user_selection()	CLI menu to choose between automatic/manual image selection	Prep_and_Tools.ui_tools
select_series()	Prompts user to pick a DICOM/NIfTI series from visual preview	Prep_and_Tools.ui_tools
visualize_nifti_series()	Displays axial/sagittal/coronal mid-slices for a NIfTI series	Prep_and_Tools.visualization_tools
visualize_dicom_series()	Displays mid-slice preview of a DICOM series	Prep_and_Tools.visualization_tools
list_all_series()	Scans a folder and lists all available NIfTI and DICOM series	Prep_and_Tools.imaging_tools
convert_dicom_to_nifti()	Converts selected DICOM folder to .nii.gz	Prep_and_Tools.imaging_tools
automatic_image_selection()	Automatically selects the most relevant series using heuristics	Prep_and_Tools.imaging_tools

Table 36: Overview of functions used within the batch_patient_prep() and individual_patient_prep() functions.

patient_tools.py

These are the functions defined within the ***patient_tools.py*** script:

Function / Method	Purpose	Location
extract_patient_info()	Parses a pandas DataFrame to extract structured patient metadata from the input CSV (NHC, ID, visits, levels, etc.)	patient_tools.py
organize_patient_files()	Sorts raw DICOM and NIfTI files by visit and series, storing them under SORTED/ folders using SeriesDescription or SeriesNumber	patient_tools.py
find_patients()	Recursively searches the Results folder for patient _config.txt files	patient_tools.py
show_patients()	Displays a quick summary of found patients, distinguishing between new and partially processed ones	patient_tools.py
load_case_config()	Parses the .txt config file and returns a dict of the patient metadata (used in setup phase and display)	patient_tools.py
clean_patient_analysis_files()	Deletes all analysis-related output for a patient (segmentation, mesh, registration, crop, stats). Useful when overwriting/restarting	patient_tools.py

Table 37: Overview of the functions defined within the patient_tools.py script.

io_tools.py

These are the functions defined within the ***io_tools.py*** script:

Function / Method	Purpose	Location
load_csv()	Loads a CSV or Excel file into a pandas DataFrame; used in prep_workflows.py to load the master metadata table	io_tools.py
copy_csv()	Copies the metadata file to the Results/ folder with a new name (COPY of ...) for traceability	io_tools.py
list_folder_contents()	Utility to print the contents of a folder, with formatting and error handling	io_tools.py

Table 38: Overview of the functions defined within the io_tools.py script

imaging_tools.py

These are the functions defined within the ***imaging_tools.py*** script:

Function / Method	Purpose	Location
convert_dicom_to_nifti()	Converts DICOM folder to NIfTI .nii.gz format using dicom2nifti	imaging_tools.py
list_nifti_series()	Returns list of valid .nii/.nii.gz files loaded with nibabel for visualization	imaging_tools.py
is_dicom_folder()	Quick check if a folder contains DICOM files based on extension	imaging_tools.py
list_all_series()	Lists and prints metadata of all DICOM/NIfTI series in a folder (shape, voxel size)	imaging_tools.py
automatic_image_selection()	Automatically selects the best image series based on anatomical coverage and resolution	imaging_tools.py

Table 39: Overview of the functions defined within the imaging_tools.py script.

visualization_tools.py

These are the functions defined within the ***visualization_tools.py*** script:

Function	Purpose	Location
visualize_nifti_series()	Displays axial, sagittal, and coronal mid-slices of NIfTI files using adaptive contrast enhancement.	visualization_tools.py
visualize_dicom_series()	Loads and shows a representative mid-slice from a DICOM folder, robust to corrupted or unreadable slices.	visualization_tools.py

Table 40: Overview of the functions defined within the visualization_tools.py script

ui_tools.py

These are the functions defined within the ***ui_tools.py*** script:

Function	Purpose	Location
select_main_directory()	Prompts user to select the main directory for results and input.	ui_tools.py
select_slicer_path()	Guides user to specify the path to the 3D Slicer executable.	ui_tools.py
get_user_selection()	Displays a list of options and returns user selection(s).	ui_tools.py
select_series()	Asks user to pick one series from a list (used after previewing).	ui_tools.py
get_segmentation_config()	Guides user through segmentation configuration: tool, modality, ROI, and speed mode.	ui_tools.py

Table 41: Overview of the functions defined within the ui_tools.py script

results_structure.py

These are the functions defined within the ***results_structure.py*** script:

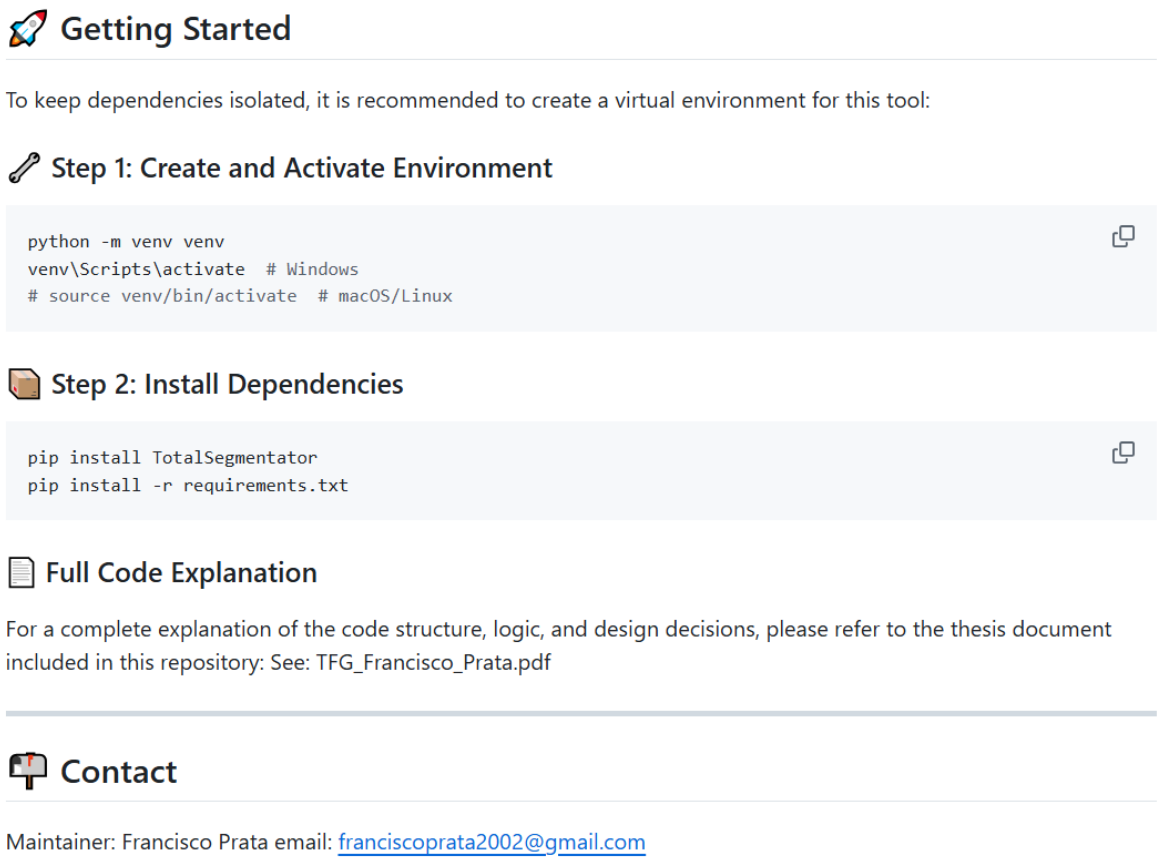
Function	Purpose	Location
create_results_structure()	Creates or overwrites patient folder structure dynamically by visit.	results_structure_tools.py
export_patient_list_to_excel()	Scans patient progress (based on config and JSON state), and exports status table to Excel with visual flags.	results_structure_tools.py
create_analysis_results_excel()	Prompts user to create an empty Excel file to store analysis results.	results_structure_tools.py
save_spinal_cord_global_excel()	Saves global spinal cord shape comparison (area & perimeter) to a specific sheet in the Excel.	results_structure_tools.py
save_comparison_figures()	Saves matplotlib figures of shape comparison to a designated output folder.	results_structure_tools.py
save_psoas_global_excel()	Appends psoas atrophy statistics (mean, median, Cliff's delta, etc.) to Excel.	results_structure_tools.py
save_psoas_figures()	Saves psoas analysis visualizations (histograms, boxplots) into a structured figures folder.	results_structure_tools.py

Table 42: Overview of the functions defined within the results_structure.py script.

Annex 5: Modular Comparative Copilot GitHub

A GitHub repository has been created with all the developed code and necessary material to install and run the Modular Comparative Copilot tool. It can be accessed by requesting permission to the author, through the link: https://github.com/franciscoprata1/Modular-Comparative-Copilot.

The README file has been developed with the necessary steps to install the tool correctly.



🚀 Getting Started

To keep dependencies isolated, it is recommended to create a virtual environment for this tool:

🔧 Step 1: Create and Activate Environment

```
python -m venv venv
venv\Scripts\activate  # Windows
# source venv/bin/activate  # macOS/Linux
```

📦 Step 2: Install Dependencies

```
pip install TotalSegmentator
pip install -r requirements.txt
```

📄 Full Code Explanation

For a complete explanation of the code structure, logic, and design decisions, please refer to the thesis document included in this repository: See: TFG_Francisco_Prata.pdf

📪 Contact

Maintainer: Francisco Prata email: franciscoprata2002@gmail.com

Figure 58: Snapshot of the README file available in the GitHub repository that gives a basic explanation of how to install and use the Modular Comparative Copilot tool.